



第5章 数据库管理

主讲教师：陈安龙

电子科技大学 信息与软件学院

学习目的和要求

- 了解典型关系DBMS数据库系统结构、数据库存储结构、数据库文件组织、数据库索引原理、B+树索引、哈希索引。
- 理解数据库事务、并发控制、事务调度、基于锁的共享数据访问、锁协议、事务隔离级别。
- 掌握数据库管理的基本操作，如数据库维护、数据库用户管理、角色管理、权限管理等。

5.1 数据库管理

数据库管理(Database Management)是指对数据库系统中的数据进行有效组织、存储、管理和维护的过程。它涉及到DBA对数据库的创建、更新、查询、备份、恢复等操作，旨在确保数据的安全性、完整性、一致性和可靠性。

1、为什么需要数据库管理

- ① 数据库系统随规模增大,系统会变得异常复杂
- ② 多用户数据库应用带来数据库访问复杂性
- ③ 数据安全和数据隐私对机构和用户都非常重要
- ④ 数据库系统随数据量增加和使用时间增长其性能会降低
- ⑤ 系统遭遇意外事件, 数据库损坏或数据丢失

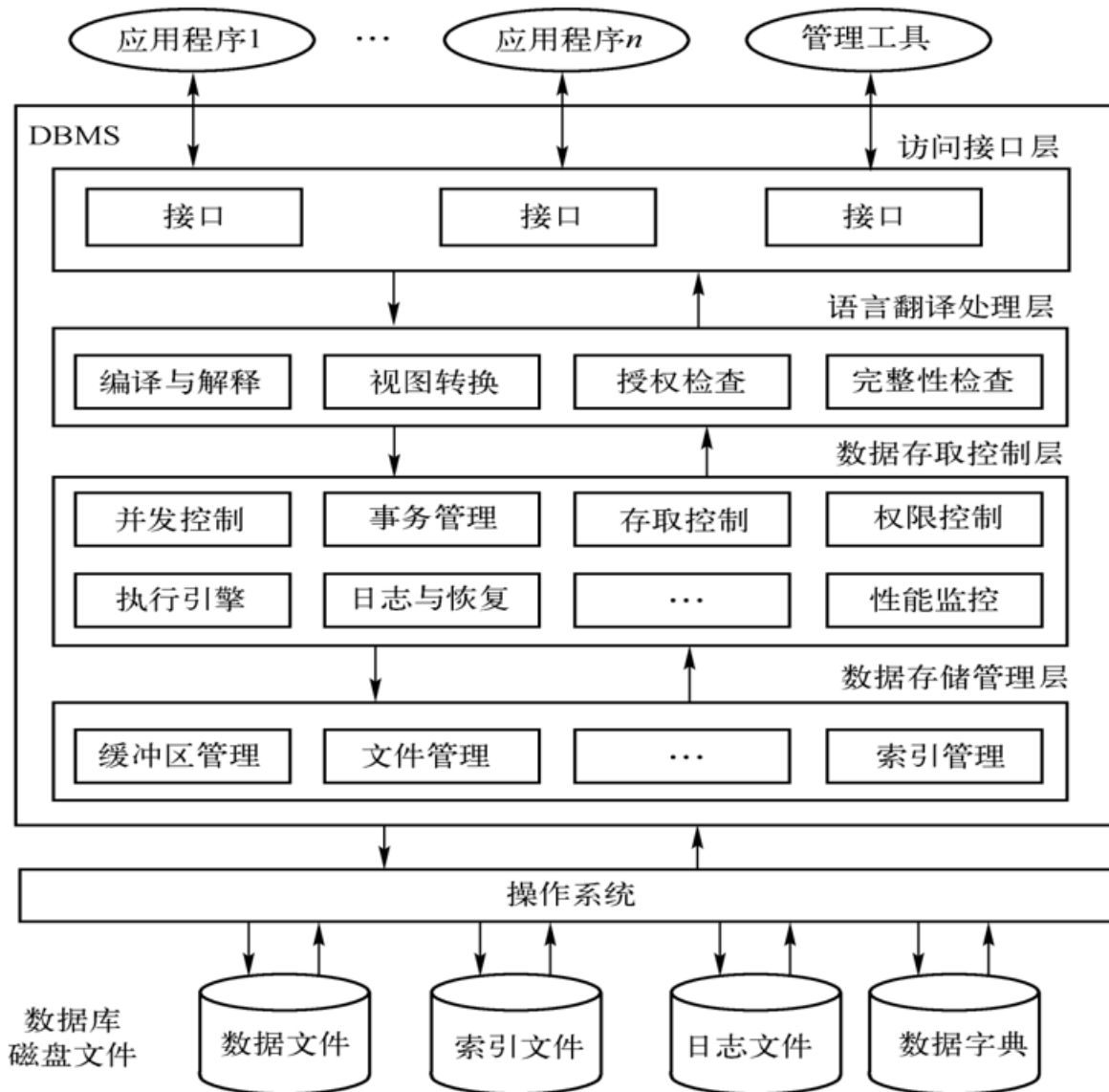
2、数据库管理目标

- ① 保障数据库系统正常稳定运行和数据库存取访问性能；
- ② 充分发挥数据库系统的软硬件处理能力；
- ③ 确保数据库系统安全和用户数据隐私性；
- ④ 有效管控数据库系统用户及其角色权限；
- ⑤ 解决数据库系统性能优化、系统故障与数据损坏等问题；
- ⑥ 最大限度地发挥数据库对其所属机构的信息资产作用。

3、数据库管理内容

- ① DBMS系统运行管理与数据库性能监控
- ② 数据库存储、索引与查询优化
- ③ 数据库事务并发控制
- ④ 数据库角色、用户管理
- ⑤ 数据库对象权限管理
- ⑥ 数据库安全管理
- ⑦ 数据库备份与恢复

4、数据库管理系统(DBMS)的系统结构



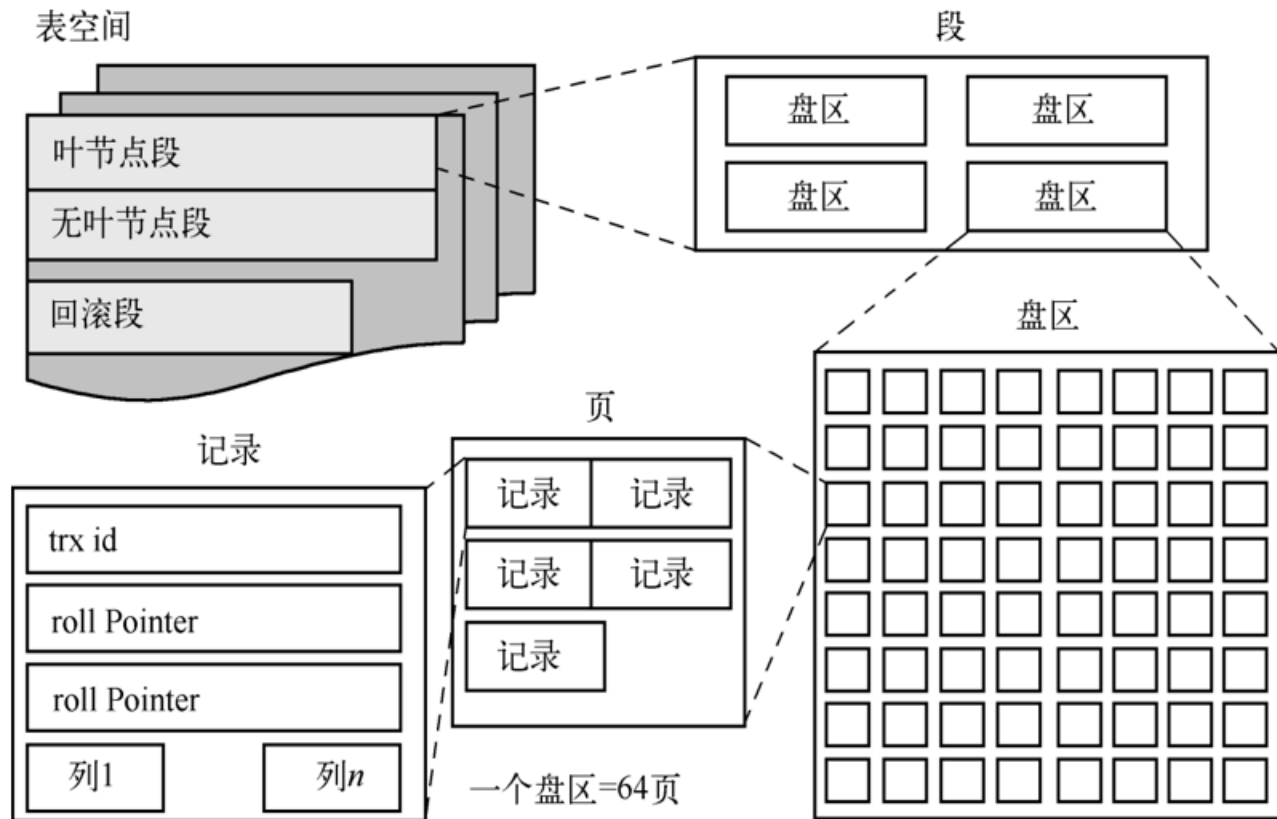
5.2、数据库存储管理

数据库存储管理是数据库管理系统（DBMS）中的一个重要组成部分，主要负责管理数据库中数据的存储和组织。定义了数据在磁盘上的存储结构，包括数据缓冲区、数据库物理文件（数据文件、索引文件、日志文件、控制文件）等。它确定了数据在物理存储介质上的组织方式，以便高效地存取数据。

1、数据库存储结构

- 数据库存储结构可以分为逻辑存储结构和物理存储结构。
- 数据库逻辑存储结构是面向数据库编程用户的存储结构。
- 物理存储结构是指数据在磁盘上的实际存储方式，包括数据文件、索引文件、日志文件等物理文件。

逻辑存储结构是指用户看到和使用的数据库对象的组织方式，包括**表空间**、**段**、**页面**、**记录**等逻辑结构。**逻辑存储结构与数据库的物理存储无关**



① 表空间 (Tablespace)

表空间是数据库中用于存储数据库对象（如表、索引等）的逻辑结构。它是一个或多个物理文件的集合，数据库对象在这些文件中存储。主要功能：

- I. **数据管理：**表空间允许数据库管理员将不同的数据库对象存储在不同的物理位置，以便于管理和优化性能。
- II. **存储分离：**可以将不同类型的数据（如用户数据、临时数据、索引等）存储在不同的表空间中，以提高性能和安全性。

示例：在 PostgreSQL 中，可以创建多个表空间，每个表空间可以指定不同的物理存储位置。

② 段 (Segment)

段是表空间中的一个逻辑结构，通常用于存储特定类型的数据库对象。每个段可以包含一个或多个页面。主要类型有：

- I. 表段：用于存储表的数据。
- II. 索引段：用于存储索引的数据。
- III. 临时段：用于存储临时表的数据。

段的使用使得数据库可以更好地管理和组织数据，尤其是在处理大规模数据时。段的大小和数量可以根据需要进行调整，以优化存储和性能。

③ 页面 (Page)

页面是数据库中**存储数据的基本单位**，通常是**固定大小的块（如 8KB）**。每个页面可以存储多个记录。主要功能有：

- I. 数据存储：**页面是实际存储数据的地方，数据库在读取和写入数据时以页面为单位进行操作。
- II. I/O 操作：**数据库系统通常以页面为单位进行输入/输出操作，这样可以提高数据访问的效率。

示例：在 PostgreSQL 中，**页面的大小通常为 8KB**，数据库会将数据分配到这些页面中。

④ 记录 (Record)

记录是数据库表中的一行数据，包含多个字段（列）的值。每个记录对应于表中的一行。主要功能有：

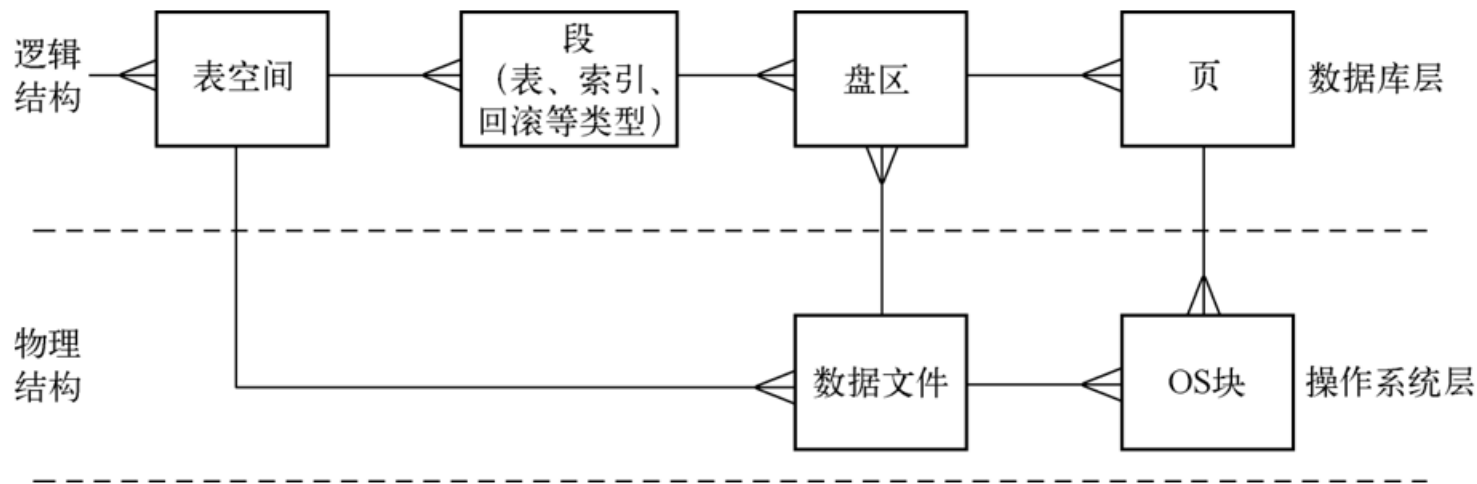
- I. 记录是数据库中存储的基本数据单元，所有的查询和操作都是围绕记录进行的。
- II. 每个记录在页面中占用一定的空间，多个记录可以存储在同一个页面中。

示例：在一个用户表中，每个用户的信息（如用户名、密码、邮箱等）都可以视为一条记录。

逻辑结构之间的关系：

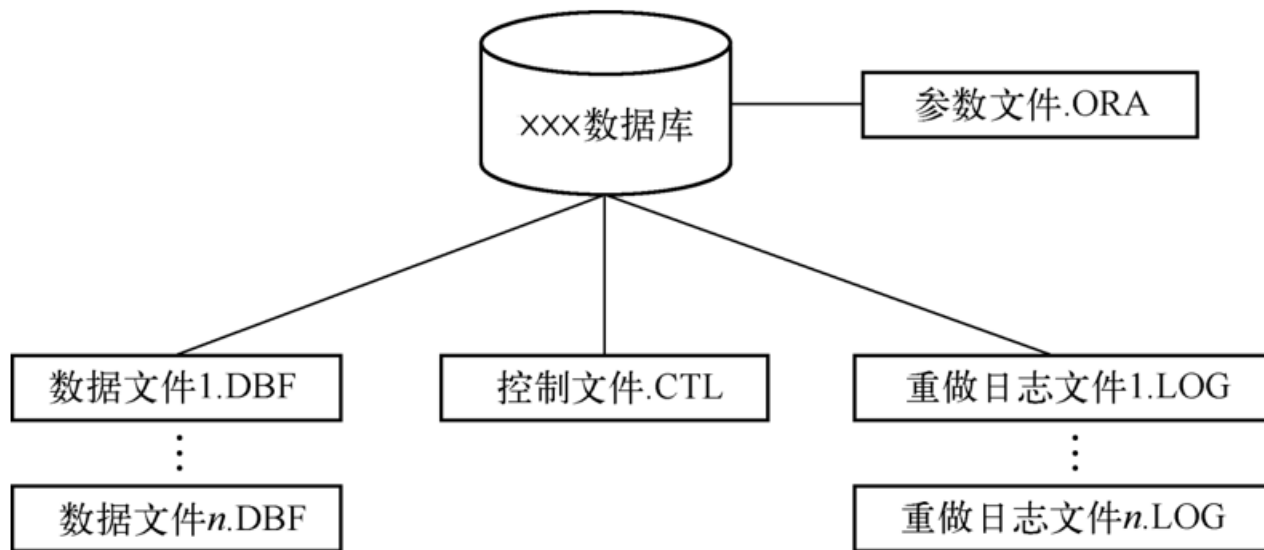
- ① **表空间**：是数据库的逻辑容器，包含多个段。
- ② **段**：是表空间中的逻辑结构，通常用于存储特定类型的对象（如表或索引），每个段由多个页面组成。
- ③ **页面**：是存储数据的基本单位，多个记录可以存储在一个页面中。
- ④ **记录**：是表中的一行数据，包含多个字段的值，存储在页面中。

逻辑存储结构和物理存储结构之间通过数据库管理系统（DBMS）进行映射和管理，用户通过逻辑结构与数据库交互，而DBMS负责将逻辑结构转换为物理结构进行实际的数据存储和管理。合理设计和管理逻辑存储结构和物理存储结构可以提高数据库系统的性能和可靠性，确保数据的安全性和完整性。



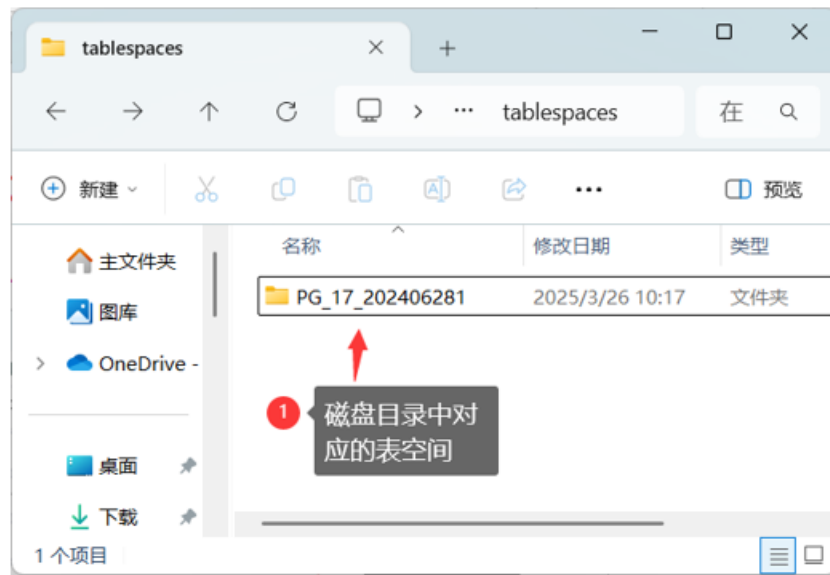
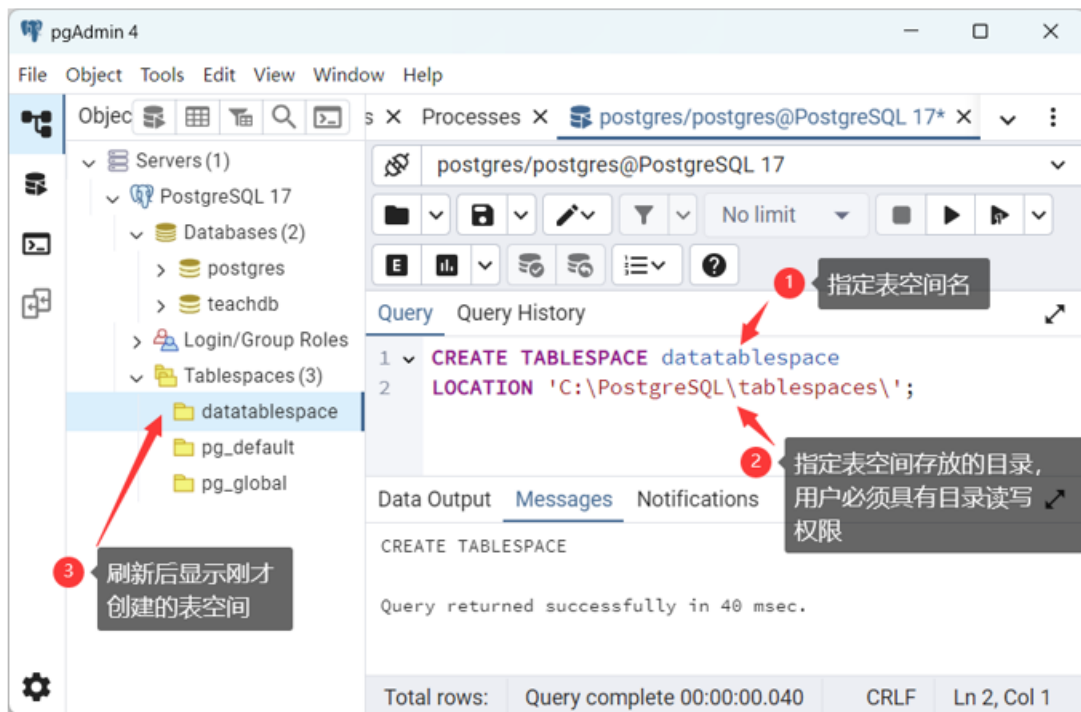
关系数据库的物理存储结构，至少应该包括：

- ① 数据文件：数据文件是数据库中存储实际数据记录的文件，包含表中的数据记录；
- ② 日志文件：日志文件用于记录数据库系统的操作日志，用于数据库的恢复和故障处理；
- ③ 控制文件：控制文件包含数据库系统的元数据信息，如数据库的结构信息、表空间信息等。



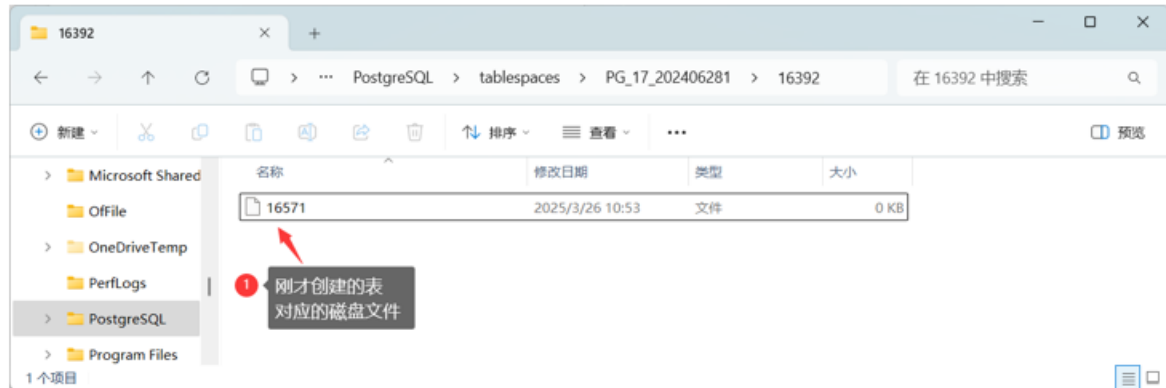
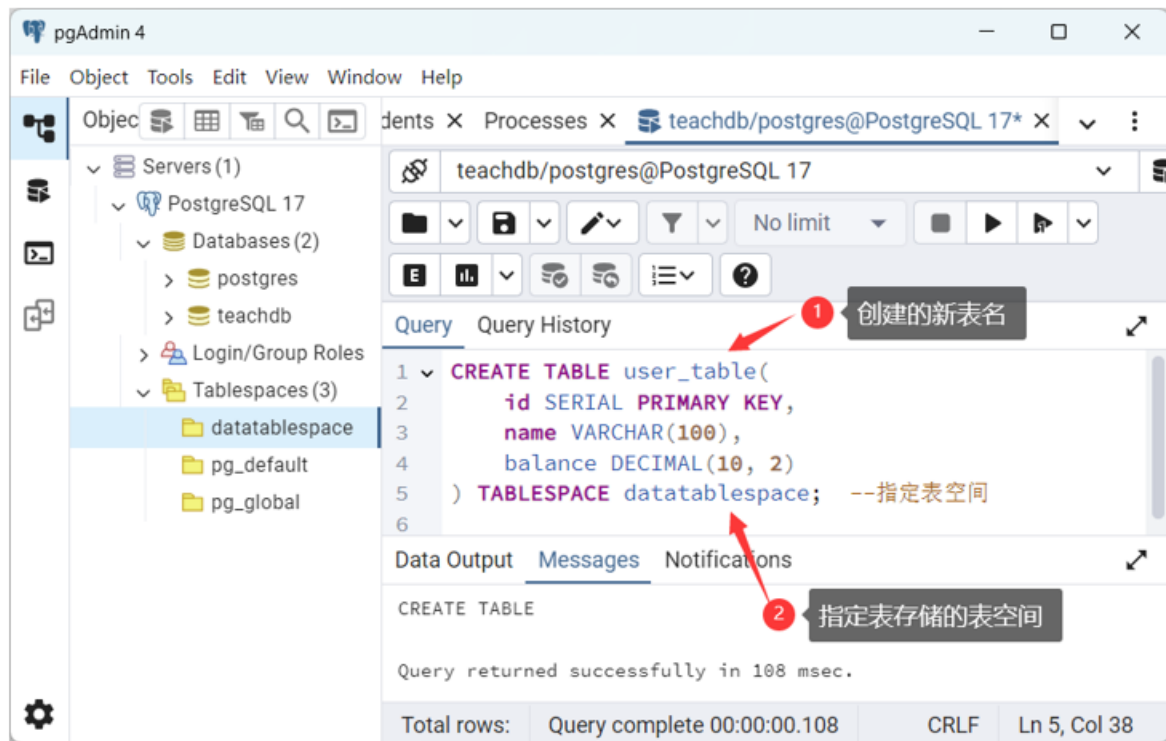
创建表空间的示例

CREATE TABLESPACE **datatablespace** **LOCATION** 'C:\PostgreSQL\tablespace\';



创建表并指定表空间的示例

```
CREATE TABLE user_table(  
    id SERIAL PRIMARY KEY,  
    name VARCHAR(100),  
    balance DECIMAL(10, 2)  
) TABLESPACE datatablespace;  
--指定表空间
```



向表中插入一些记录，记录将存储在指定的表空间中的示例

INSERT INTO user_table (name, balance) **VALUES** ('Alice', 1000.00), ('Bob', 500.00);

The screenshot displays the pgAdmin 4 interface. The left sidebar shows the database structure: Servers (1) > PostgreSQL 17 > Databases (2) > postgres > tablespaces (3) > datatablespace. The main window shows the 'Query' tab with the following SQL statement:

```
1 INSERT INTO user_table (name, balance) VALUES ('Alice', 1000.00), ('Bob', 500.00);
2
```

The 'Messages' tab shows the execution result:

```
INSERT 0 2

Query returned successfully in 46 msec.
```

Total rows: Query complete 00:00:00.046

An inset window shows the 'Query' tab with the following SQL statement:

```
1 select * from user_table;
2
```

The 'Data Output' tab shows the resulting table data:

	id [PK] integer	name character varying (100)	balance numeric (10,2)
1	1	Alice	1000.00
2	2	Bob	500.00

Total rows: 2 Query complete 00:00:00.075 CRLF Ln 1, Col 1

查看表空间信息的示例

SELECT * FROM pg_tablespace;

The screenshot shows the pgAdmin 4 interface. On the left, the 'Servers' tree is expanded to 'PostgreSQL 17' > 'Databases (2)' > 'teachdb' > 'Tablespaces (3)', with 'datatablespace' selected. The main pane shows the 'Query' tab with the SQL query `SELECT * FROM pg_tablespace;`. A red arrow points to the query text with a callout box saying '这是系统表，用于存储表空间信息' (This is a system table, used for storing tablespace information). Below the query, the 'Data Output' tab shows the results of the query. A red arrow points to the first column 'oid' with a callout box saying '创建的表空间的ID' (ID of the created tablespace). The results table has 3 rows and 6 columns: oid, spcname, spcowner, spcacl, and spcoptions. The status bar at the bottom indicates 'Total rows: 3' and 'Query complete 00:00:00.103'.

	oid [PK] oid	spcname name	spcowner oid	spcacl aclitem[]	spcoptions text[]
1	1663	pg_default	10	[null]	[null]
2	1664	pg_global	10	[null]	[null]
3	16569	datatablespace	10	[null]	[null]

查看表的存储信息的示例

SELECT relname AS table_name, pg_tablespace.spcname AS tablespace_name

FROM pg_class JOIN pg_tablespace ON pg_class.reltablespace = pg_tablespace.oid

WHERE relkind = 'r' AND relname = 'user_table'; -- 'r' 表示用户创建的表

The screenshot shows the pgAdmin 4 web interface. On the left, the 'Servers' tree is expanded to 'PostgreSQL 17' > 'Databases (2)' > 'postgres' > 'teachdb' > 'Tables (3)' > 'datatablespace'. The main panel displays a SQL query in the 'Query' tab:

```
1 SELECT relname AS table_name, pg_tablespace.spcname AS tablespace_name
2 FROM pg_class
3 JOIN pg_tablespace ON pg_class.reltablespace = pg_tablespace.oid
4 WHERE relkind = 'r' AND relname = 'user_table'; -- 'r' 表示用户创建的表
5
```

Below the query editor, the 'Data Output' tab shows the results of the query. The results are displayed in a table with two columns: 'table_name' and 'tablespace_name'. The first row shows 'user_table' in the 'datatablespace' tablespace.

	table_name	tablespace_name
1	user_table	datatablespace

At the bottom of the interface, the status bar indicates 'Total rows: 1', 'Query complete 00:00:00.075', and 'Ln 3, Col 7'.

删除表空间的示例

DROP TABLESPACE *datatablespace*;

注意：在删除表空间之前，需要先删除或移动该表空间中的所有对象（如表和索引）。

2、数据文件组织

- 从物理存储结构来看，存取数据文件的基本单位是操作系统文件块 (OS 块)；
- DBMS 通过操作系统以块为单位存取访问数据文件中的数据记录，大多数数据库默认的数据文件块大小为 4KB 到 8KB，磁盘扇区大小一般为 4KB；
- 通常将数据文件块大小设置为扇区大小 $\times 2^n (n \geq 1)$ ，一个数据文件由多个块组成，每个块存储多条数据记录；
- 如果一个数据记录中包含了大对象类型数据，如图像类型数据，它在数据文件块中就无法直接存放。大对象数据通常采用单独文件方式存储，并将其指针放入数据库的数据文件块中存储。

数据文件记录结构

- 在关系数据库中，不同关系表的数据记录长度是不一样的；
- 如果关系表的记录中包含变长数据项，该关系表的数据记录长度也会不一样。

(1) 定长记录

- 在数据文件中，所有记录均分配相同长度的存储空间；
- 定长记录一般分为记录头和记录数据区两个部分；

记录头	字段1	字段2	...	字段 n
-----	-----	-----	-----	--------

- 每个文件块除了存放数据库表的记录外，还有一个块头区和可能存在的空闲区；
- 块头区用于存放与其他块的链接指针、块最近修改时间戳等信息；
- 空闲区为文件块中没有存放记录的区域。

块头区	记录1	记录2	...	记录 n	空闲区
-----	-----	-----	-----	--------	-----

1) 当数据文件块的大小不是定长记录大小的整数倍时，如何办？

- 只有在文件块中有足够空间时，才能将定长记录存储到当前文件块内；
- 若当前文件块没有存储完整记录的空间，则需将该记录存储到另一空闲的文件块内。

【例 5-1】 在选课管理数据库CurriculaDB中， 学生信息 (student)表的创建 SQL语句如下：

```
CREATE TABLE Student (  
    StudentID      char(13)      PRIMARY KEY,  
    StudentName    varchar(10)   NOT NULL,  
    StudentGender  char(2)       NULL,  
    BirthDay       date          NULL,  
    Major          varchar(30)   NULL,  
    StudentPhone   char(11)      NULL  
);
```

假设数据文件块大小为4KB， 块头区占用 12 字节，记录头占用12字节，若 Student表要存放8120个学生记录，请问需要文件块存储学生表的学生信息？

Student 表的数据文件定长记录存储结构的示意图



2) 在数据文件中删除记录,如何处理?

- ① 当表中一个记录被删除时, 可以将紧随其后的记录移动到被删除记录先前占用的存储空间中, 其他记录也依次移动; 当表中插入一个记录时, 只需在文件末尾添加。

上述方法有什么缺点? 如何解决?

2) 在数据文件中删除记录,如何处理?

- ② 改进方案是在记录被删除时，将数据文件的最后一条记录移动到被删除记录先前占用的存储空间中，这种方案仅涉及一条记录移动，其开销较小。
- 上述方法还有缺点吗？如何改进？

2) 在数据文件中删除记录,如何处理?

- ③ 在数据文件的第一个块头区定义文件头中, 存储自由链表指针。该自由链表指针指向被删除的第一个记录的地址。该记录的块头区中也存储第二个被删除记录的地址。以此类推, 在数据文件中, 形成一个被删除记录的自由链表。

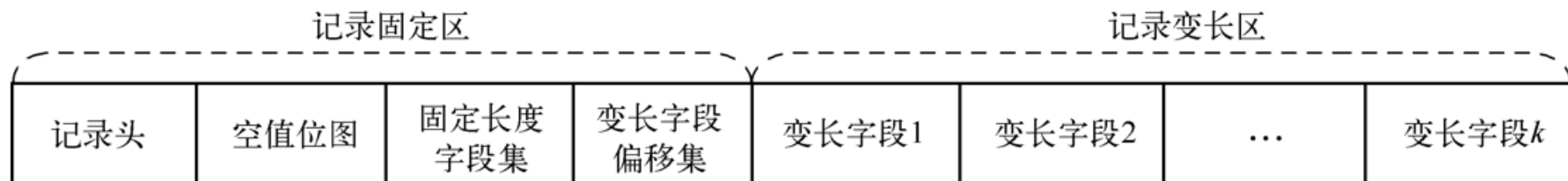
20230812011	王成	男	2002-01-20	计算机	139***	←
20230812016	赵倩	女	2002-05-11	软件	133***	←
20230812019	李明	男	2002-02-25	软件	135***	←
20230812020	齐静	女	2002-06-08	软件	138***	

(2) 变长记录

- 数据文件存储的各数据记录长度是不一样的；
- 按照实际存储需求被分配存储空间，分配给每个数据记录的存储空间不同；
- 必须解决如下两个问题：
 - ①如何设计记录格式，使得记录的各字段可以直接定位访问；
 - ②如何在数据文件块中存储变长记录，使得文件块中的记录易于访问。

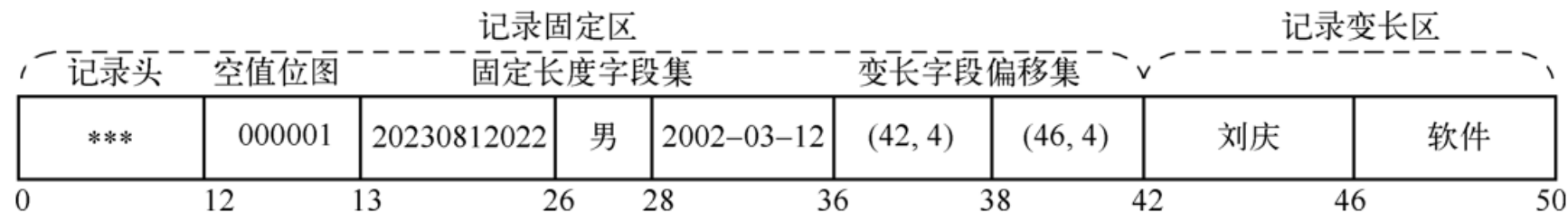
变长记录格式

- 可以将变长记录格式分为记录固定区和记录变长区。
- 所有固定长度的字段放到记录固定区存储空间， 各变长字段在记录中的偏移量和长度信息也被放入记录固定区存储空间。



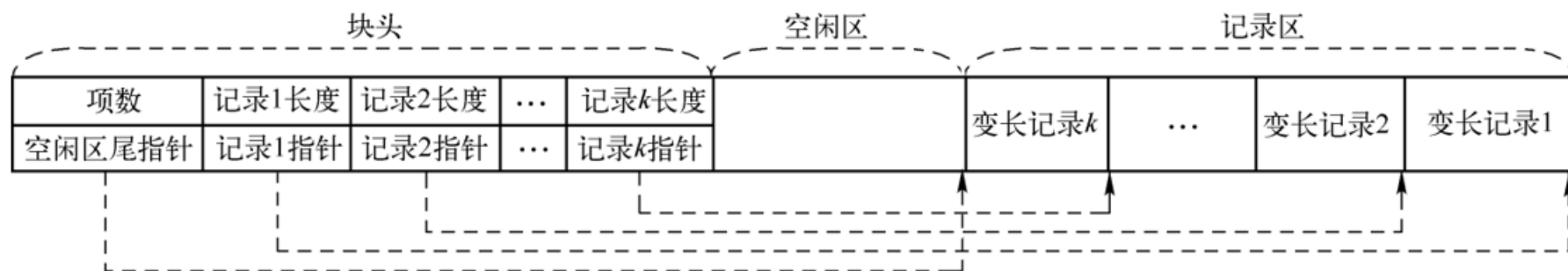
变长记录格式示例

- 假设student表采用变长记录格式存储一个学生数据（20230812022，刘庆，男，2002-03-12，软件，NULL）
- 在访问记录字段时，可以通过记录头信息在记录存储空间中定位访问到各固定长度字段数据，同样通过记录固定区中的各变长字段偏移值和长度值，可以访问到各变长字段数据。



如何在数据文件块中存储变长记录

- 文件块中采用分槽结构组织变长记录存储;
- 分槽(slot)结构指的是将文件块中的空间划分为多个槽, 每个槽可以存储一个记录或记录的指针。每个槽通常包含两部分信息: 记录长度和记录内容; 记录长度用于标识记录的实际长度, 记录内容存储记录的数据;
- 在文件块分槽结构中, 块头的“项数”单元记录文件块中存储的记录数。



文件块分槽结构插入、删除记录

- 当插入新记录时，在空闲区的尾部开始存放该变长记录数据，并将该记录的长度和初始位置添加到块头区中；同时块头区的修改空闲区尾指针；
- 当删除某记录时，将块头区中的项数减1，并在块头区移动其后的指针单元和长度单元，同时在记录区移动其后的变长记录，重新调整空闲区尾指针；
- 为了降低删除记录带来的系统处理开销，可以仅将删除记录在块头区的记录长度标记为特殊值(如-1)，并不把该记录在记录区占用的空间释放。

数据文件的记录组织

- 数据文件的记录组织方法是指在文件中如何组织和存储记录数据，以便有效地对数据进行访问和管理。以下是一些常见的数据文件记录组织方法：

(1). 顺序文件组织：

特点：记录按照顺序存储在文件中，每个记录的位置取决于前一个记录的位置。

优点：适合顺序访问，易于维护和备份。

缺点：插入和删除记录可能需要移动大量数据。

数据文件的记录组织

(2). 索引文件组织:

特点: 文件中包含一个索引表, 索引表中存储记录的关键字和对应的位置信息。

优点: 支持快速随机访问, 插入和删除记录相对高效。

缺点: 索引表可能占用额外空间, 需要维护索引表的一致性。

(3). 散列文件组织:

特点: 根据记录的关键字直接计算记录在文件中的位置, 避免了顺序查找。

优点: 支持快速访问, 适合大规模数据集。

缺点: 可能存在冲突, 需要解决散列冲突问题。

数据文件的记录组织

(4). 堆文件组织:

特点: 记录按照任意顺序存储在文件中, 每个记录包含指向下一个记录的指针。

优点: 支持随机访问和插入, 适合动态数据集。

缺点: 可能存在碎片化空间, 需要维护指针信息。

(5). 多级索引文件组织:

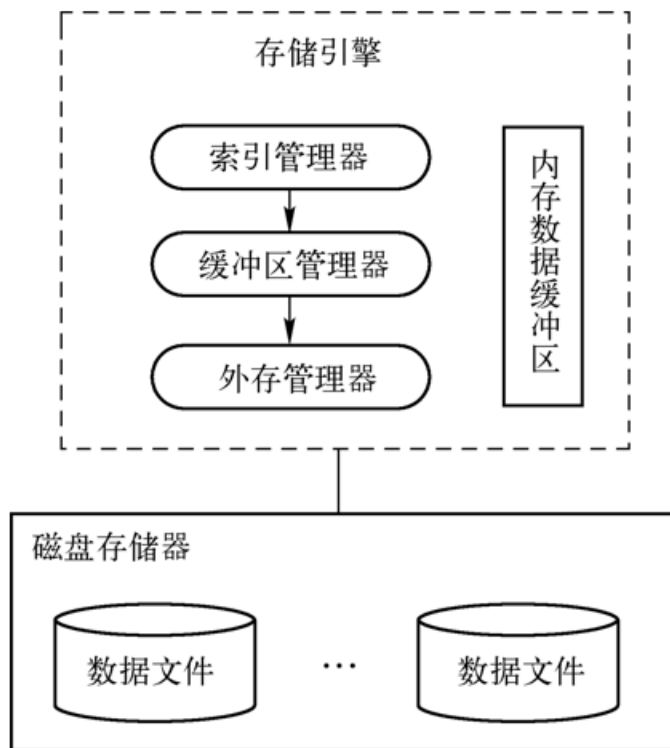
特点: 索引分为多级, 每一级索引指向下一级索引或记录。

优点: 支持高效的随机访问, 适合大型数据文件。

缺点: 需要额外的索引维护, 可能影响性能。

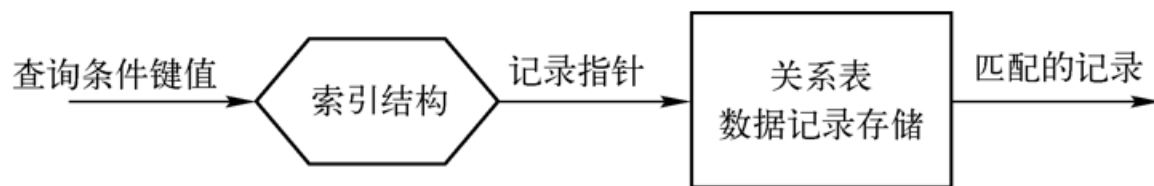
3、数据库存储引擎

数据库存储管理功能是由DBMS软件的存储引擎来实现的。数据库存储引擎也称存储管理器，它负责数据库中数据存储、数据检索和数据更新等处理。



5.3 数据库索引

索引是一种数据记录定位结构，它将查询条件键值作为输入，能快速从索引结构(如索引表)获取该条件键值对应数据记录的位置指针，通过位置指针从关系表找出结果集记录

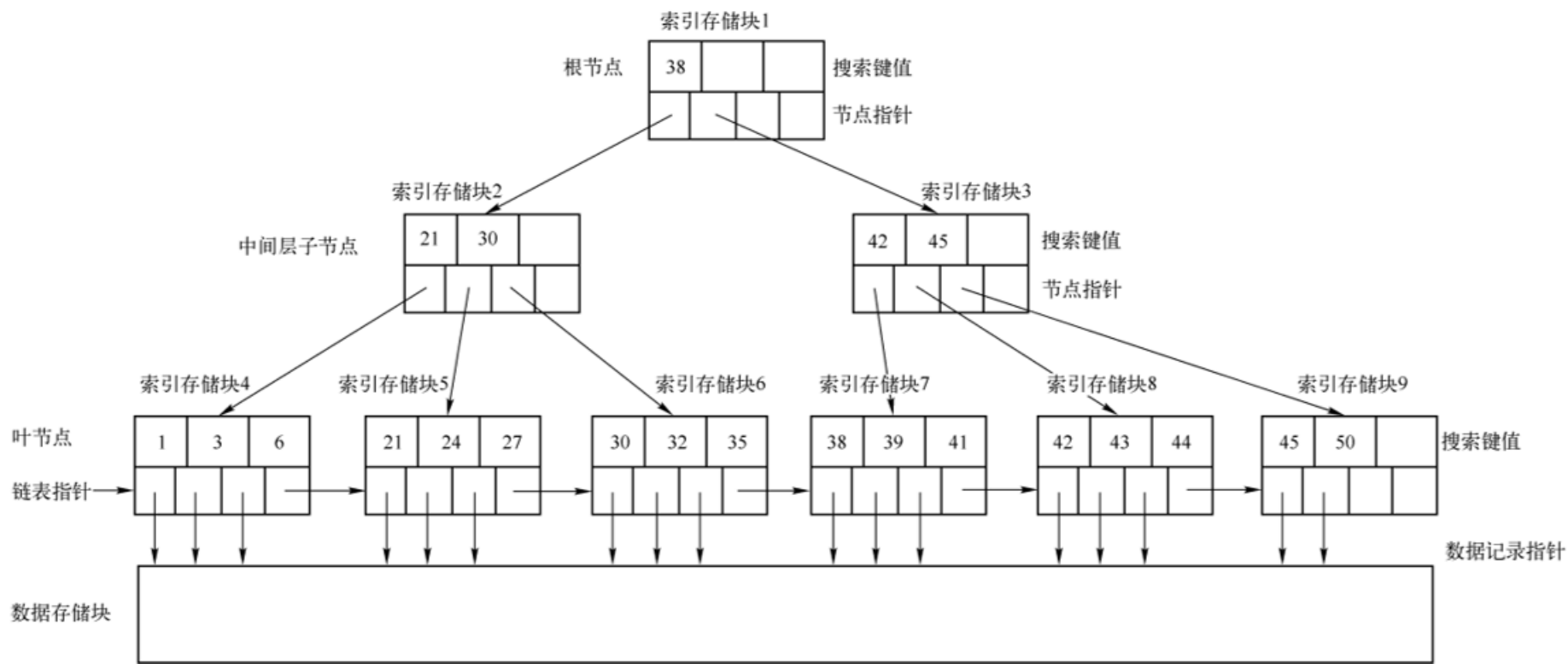


索引的作用

- ① 可以大大加快数据的查询速度，这也是创建索引的最主要动因。
- ② 可以加快表与表之间的连接速度，特别是在实现多表关联查询方面意义重大。
- ③ 在使用分组或排序子句进行数据查询时，同样可以显著缩短查询中分组和排序的时间，提高查询处理效率。

B+树索引

B+树是一种广泛用于数据库索引的平衡多叉树索引结构。它是 B-树的变体，B+树的根节点和中间层子节点只保存索引的搜索键值以及下层节点指针，而叶节点不但保存索引的搜索键值，还保存对应的数据记录(Rowid) 指针和链表指针。



B+树结构

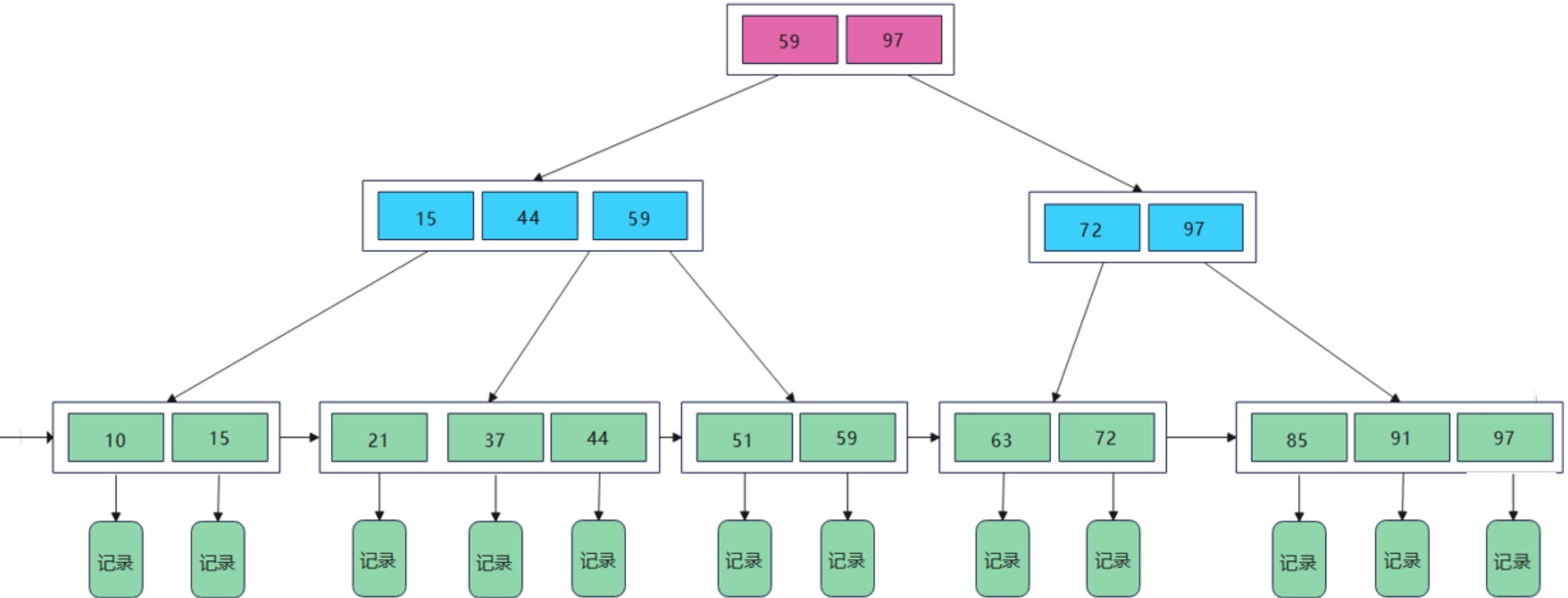
B+树结点的最大孩子个数称为树的阶，通常用 m 表示。一棵 m 阶B+树要么是空树，要么满足以下定义：

- ① 一个结点的结构是： $(K_1, P_1, K_2, \dots, K_n, P_n)$ ，其中 P 为指向子树的指针， K 为关键字；每个分支节点最多有 m 棵子树（孩子节点）；
- ② 所有分支节点（可视为索引的索引）中仅包含他的各个子节点（即下一级的索引块）中关键字的最大值及指向其子节点的指针。
- ③ 对于非根的所有结点，至少拥有 $\lceil m/2 \rceil$ 棵子树，即至少拥有 $\lceil m/2 \rceil$ 个关键字
- ④ 为了满足阶的定义，任何结点最多拥有 m 个子树，即最多拥有 m 个关键字
- ⑤ B+树的所有叶节点包含全部关键字及指向相应记录的指针，叶节点中将关键字按大小顺序排列，并且相邻叶节点按大小顺序相互链接起来

B+树的数据结构

```
typedef struct BPlusTreeNode {  
    int numKeys; //关键字个数  
    int keys[m - 1]; //结点关键字  
    struct BPlusTreeNode *ptr[m];  
    //结点指针， m-1个指向子结点， 另一个为NULL或指向下一个叶子结点  
} BPlusTreeNode;
```

3阶B+树示例



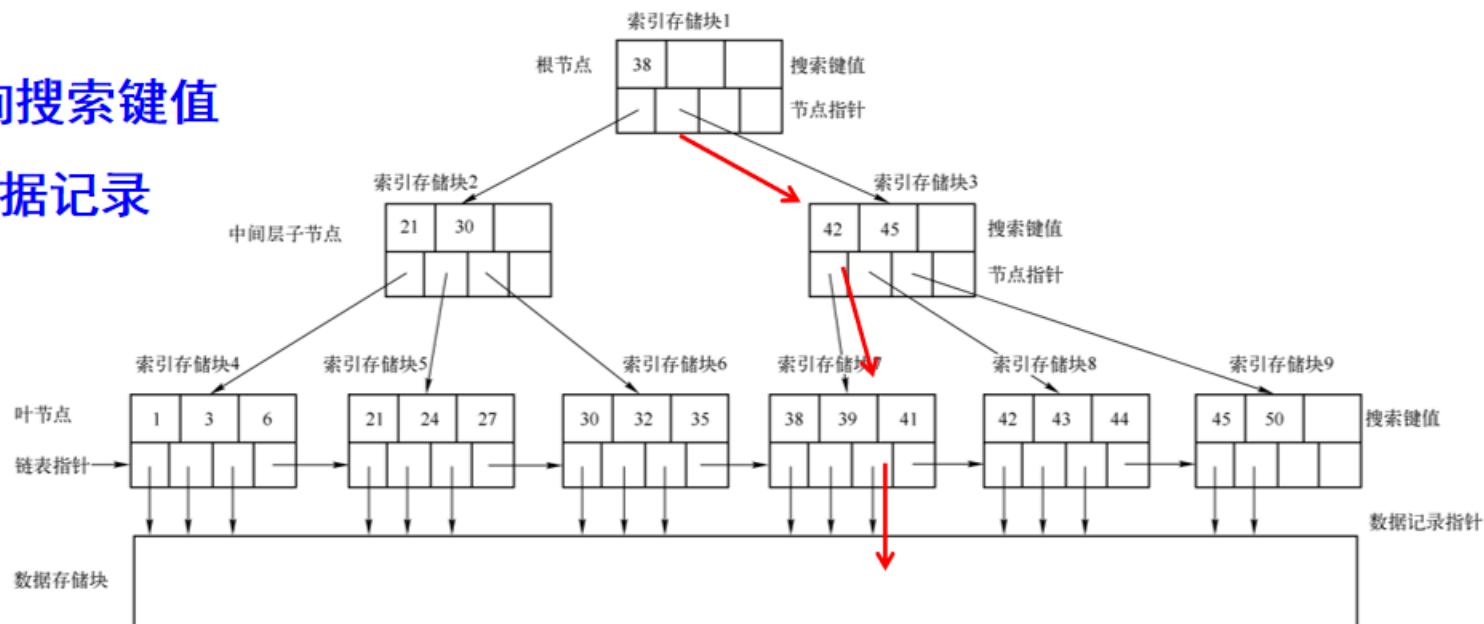
B+树记录查询

在B+树索引结构中，查找搜索键值为k的数据记录。

- ① 首先读取根节点索引数据，判断搜索键值为k的数据记录在哪个中间层子树分支。在读取中间层子节点索引数据后，继续判断搜索键值为k的数据记录在哪个叶节点。
- ② 在读取叶节点索引数据后，找出搜索键值为k的数据记录指针。通过记录指针从数据存储块中读取结果数据记录

例5-3：查询搜索键值

等于42的数据记录

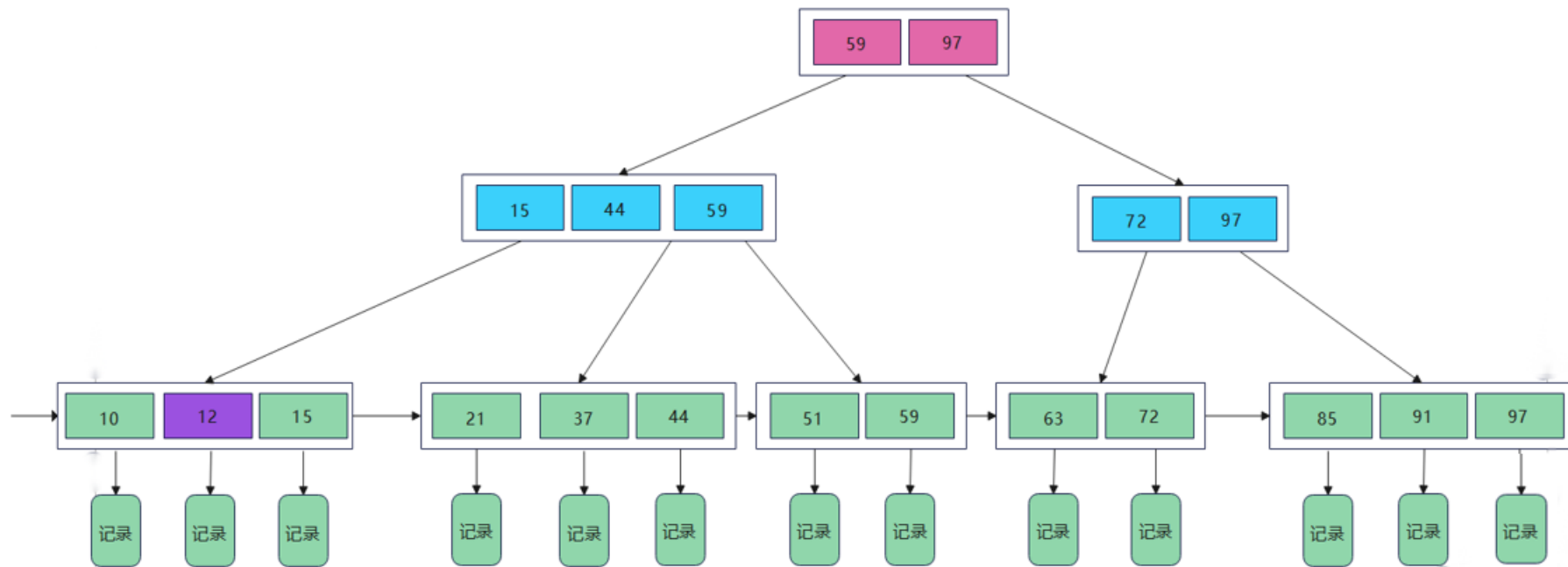


B+树记录插入

- 插入的操作全部都在叶子结点上进行，且不能破坏关键字自小而大的顺序；
- 当插入关键字后结点的关键字个数大于 m ，需要进行“分裂”。
- B+树的插入有四种情况：
 - ①若被插入关键字所在的结点，其含有关键字数目小于 m ，则直接插入；
 - ②若被插入关键字所在的结点，其含有关键字数目等于 m ，则需要将这个结点分为左右两部分，中间的结点放到父节点中。假设其双亲结点中包含的关键字个数小于 m ，则插入操作完成。
 - ③在第2种情况中，如果上移操作导致其双亲结点中关键字个数大于 M ，则应继续分裂其双亲结点。
 - ④若插入的关键字比当前结点中的最大值还大，破坏了B+树中从根结点到当前结点的所有索引值，此时需要及时修正后，再做其他操作。

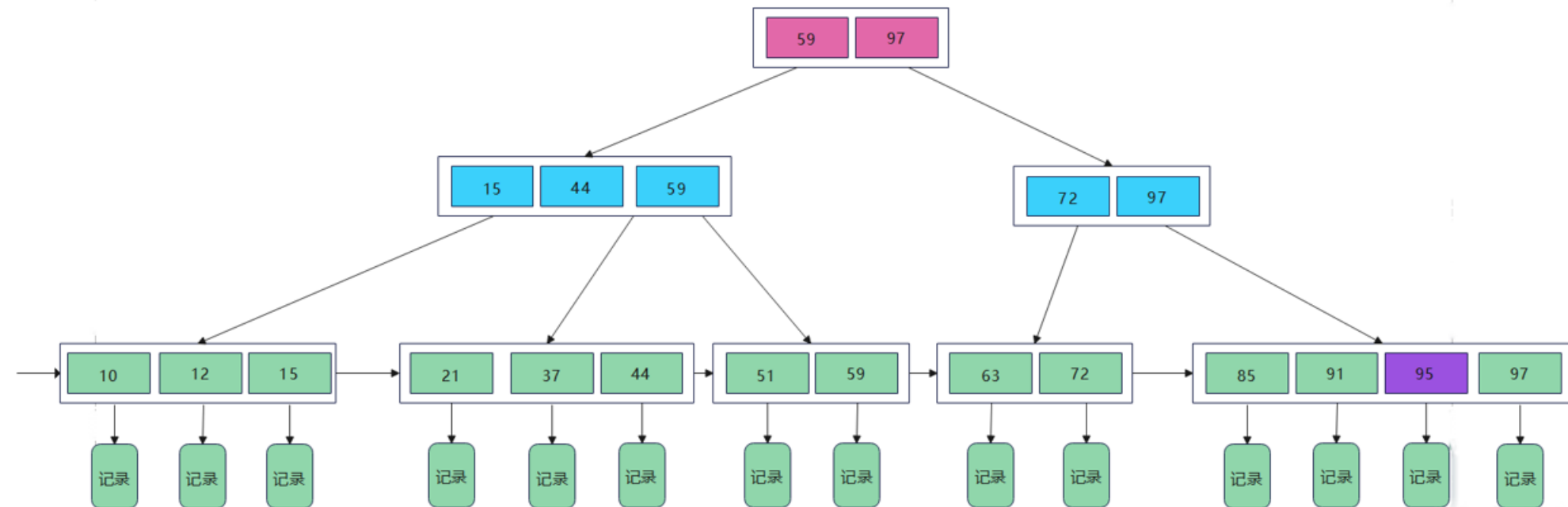
B+树记录插入

(1) 插入关键字12，此时第一个叶子节点部分[10, 15]关键字的个数 $< m$ ，可以直接插入：
(紫色代表插入的元素)



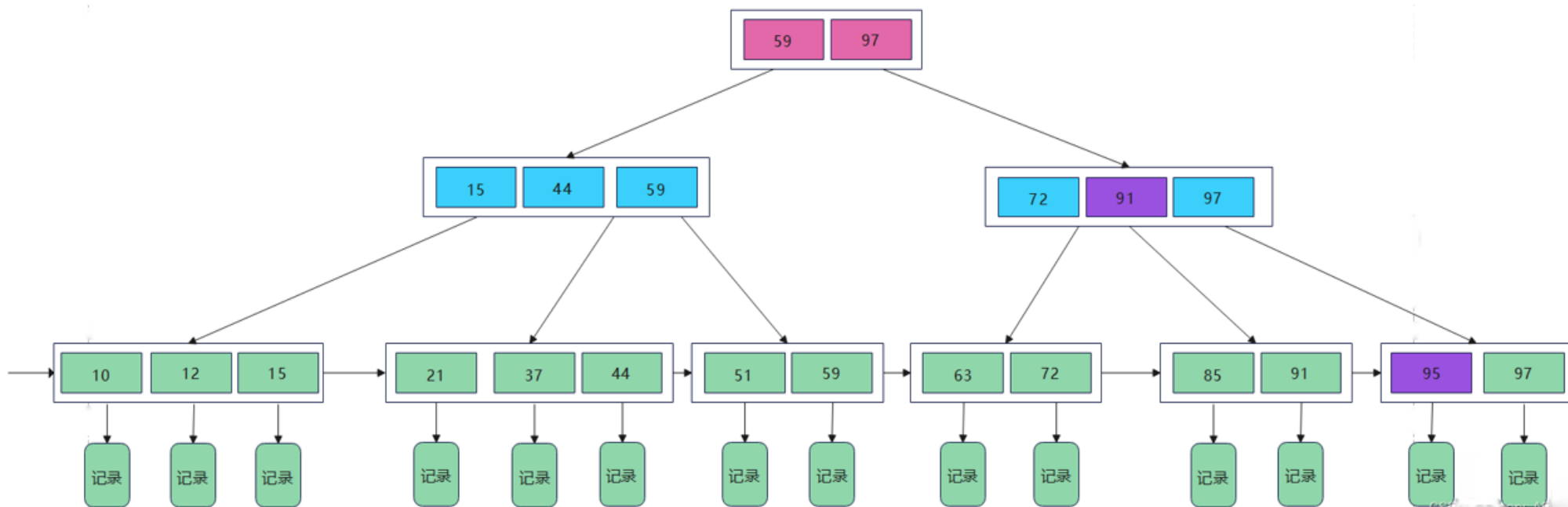
B+树记录插入

(2) 插入95，需要插入到最后一个叶子节点部分[85, 91, 97]:



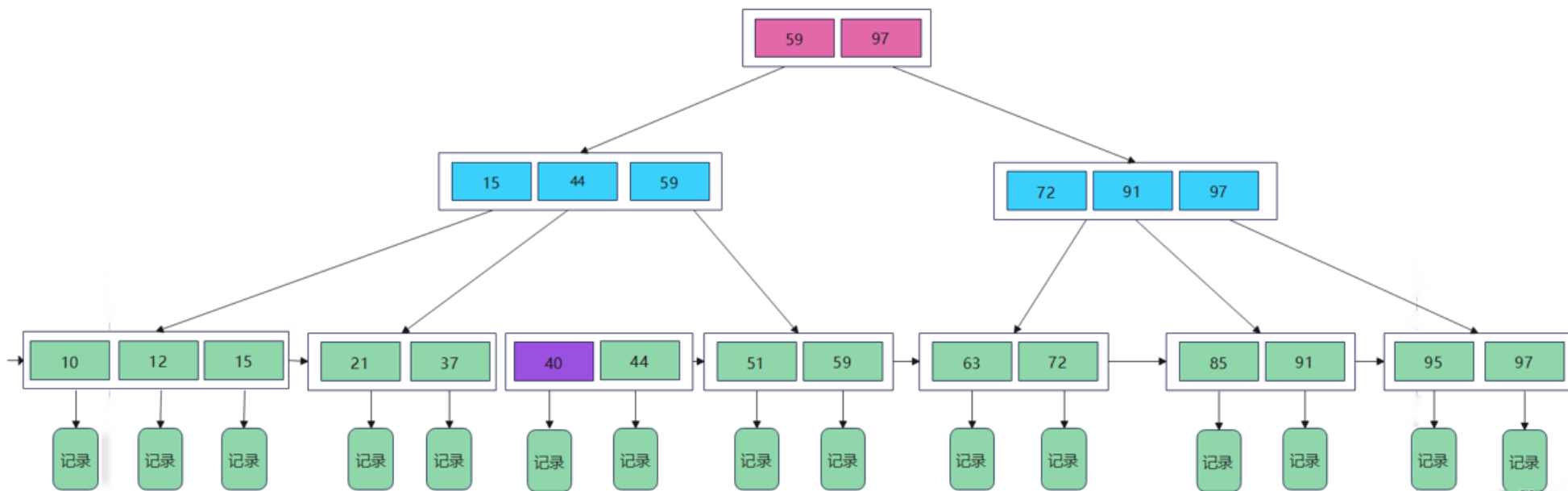
B+树记录插入

该节点的关键字个数大于 m ，需要进行分裂操作，并且父节点需要插入一个新的关键字：



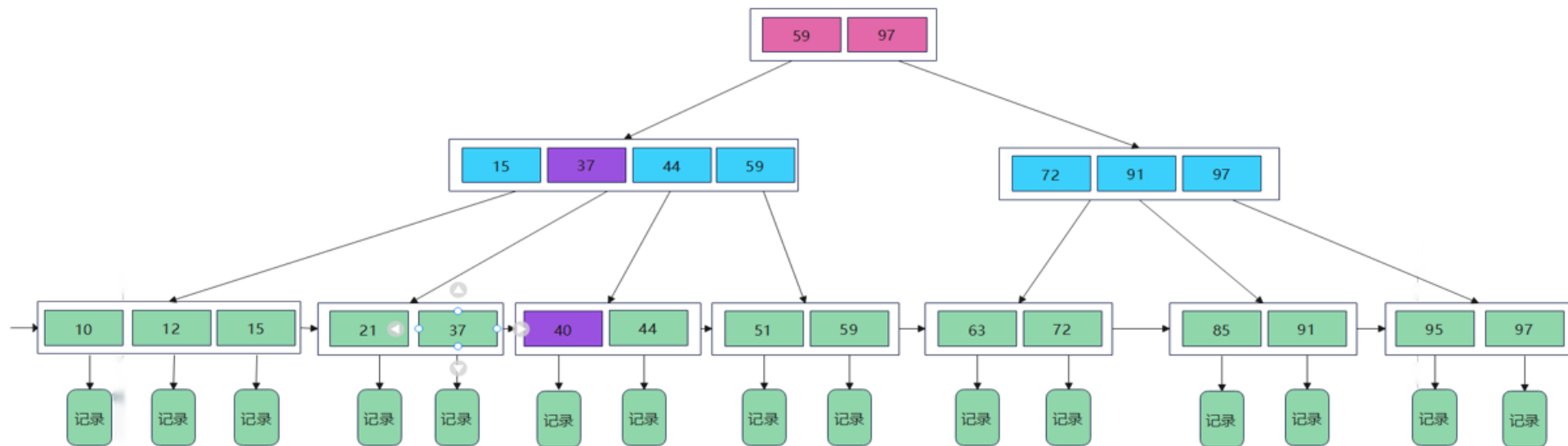
B+树记录插入

(3) 插入40，需要插入到第二个叶子节点部分[21, 37, 44]:



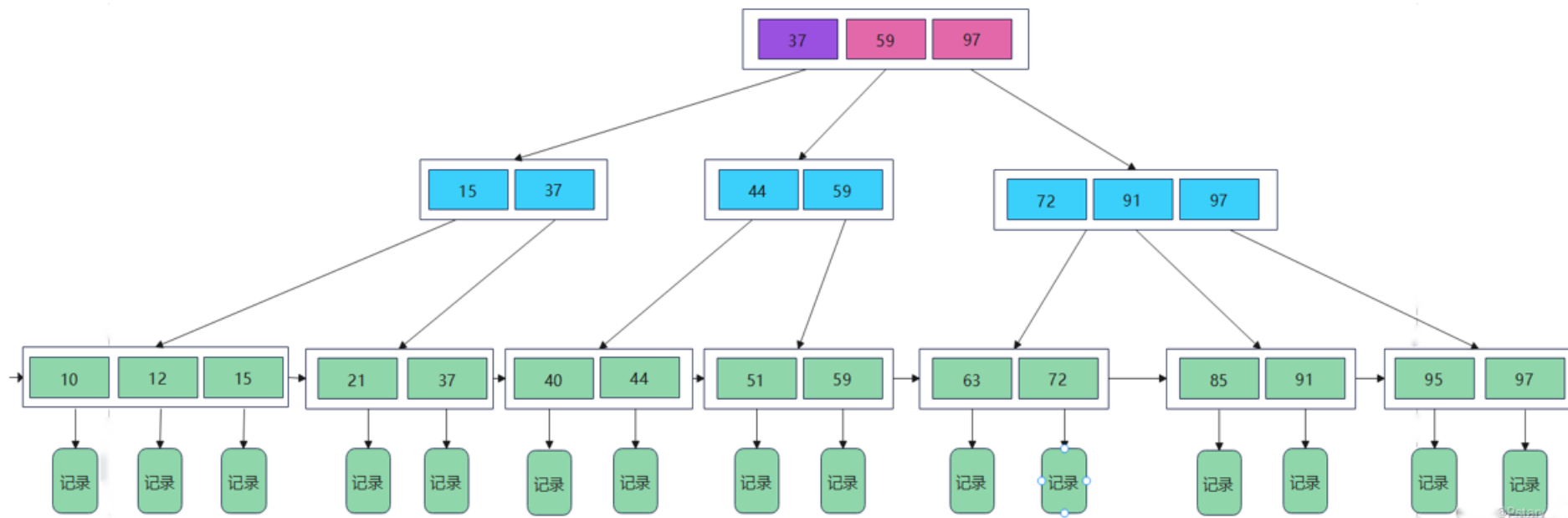
B+树记录插入

该节点的关键字个数大于 m ，需要进行分裂操作，并且父节点需要插入一个新的关键字：



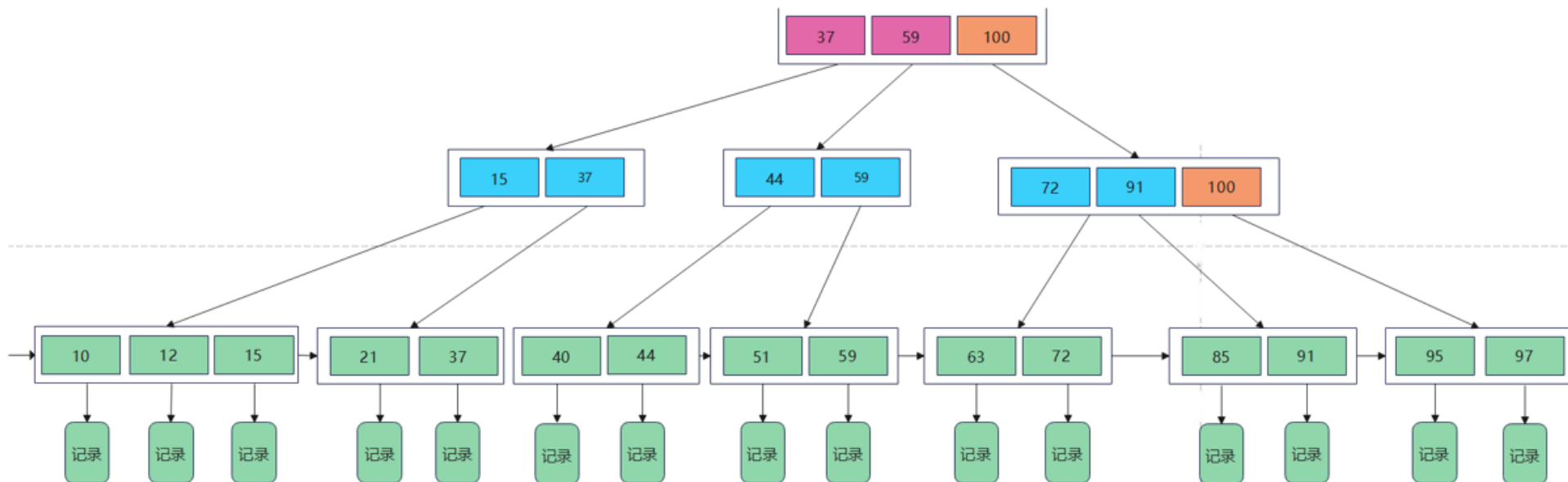
B+树记录插入

父节点插入新的关键字之后，关键字的个数大于 m ，也需要进行分裂：



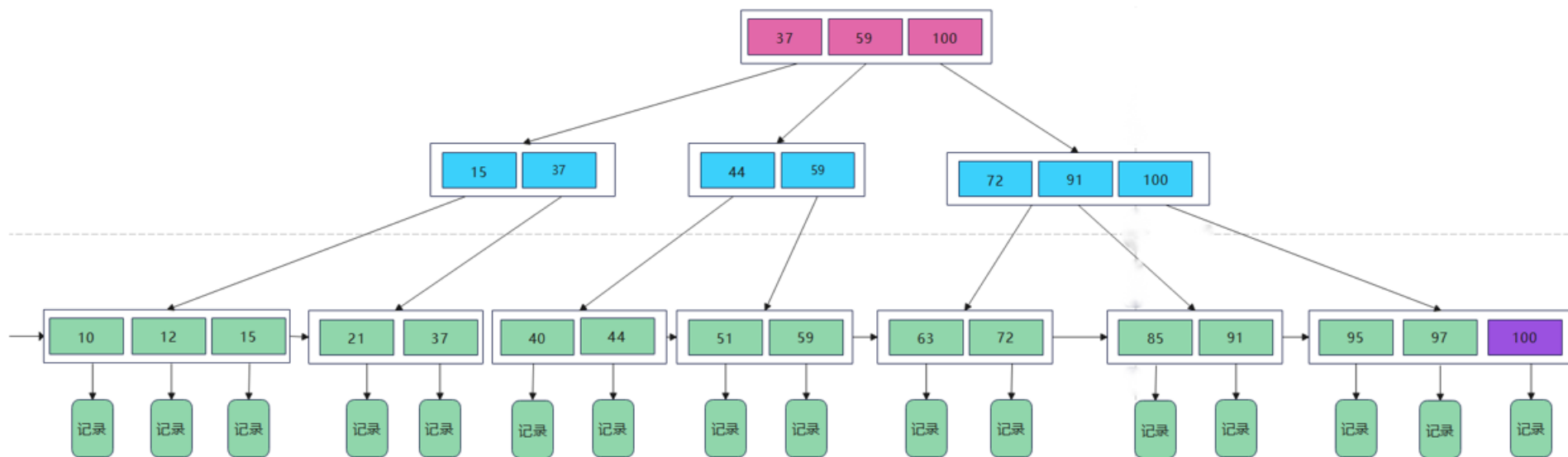
B+树记录插入

(4) 插入100，由于其值比最大值 97 还大，插入之后，从根结点到该结点经过的所有结点中的所有值都要由 97 改为 100。（橙色为修改之后的）



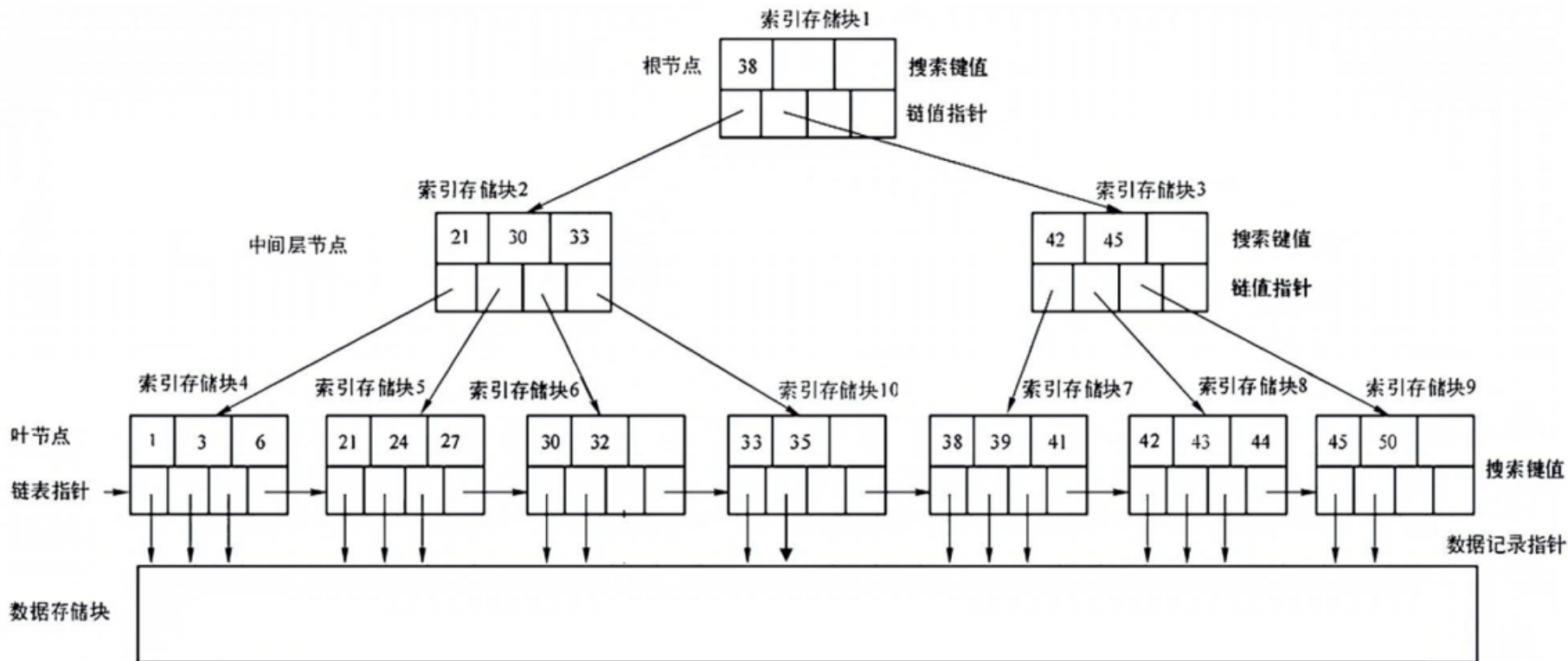
B+树记录插入

修改完最大值之后，在最后一个节点处插入100：



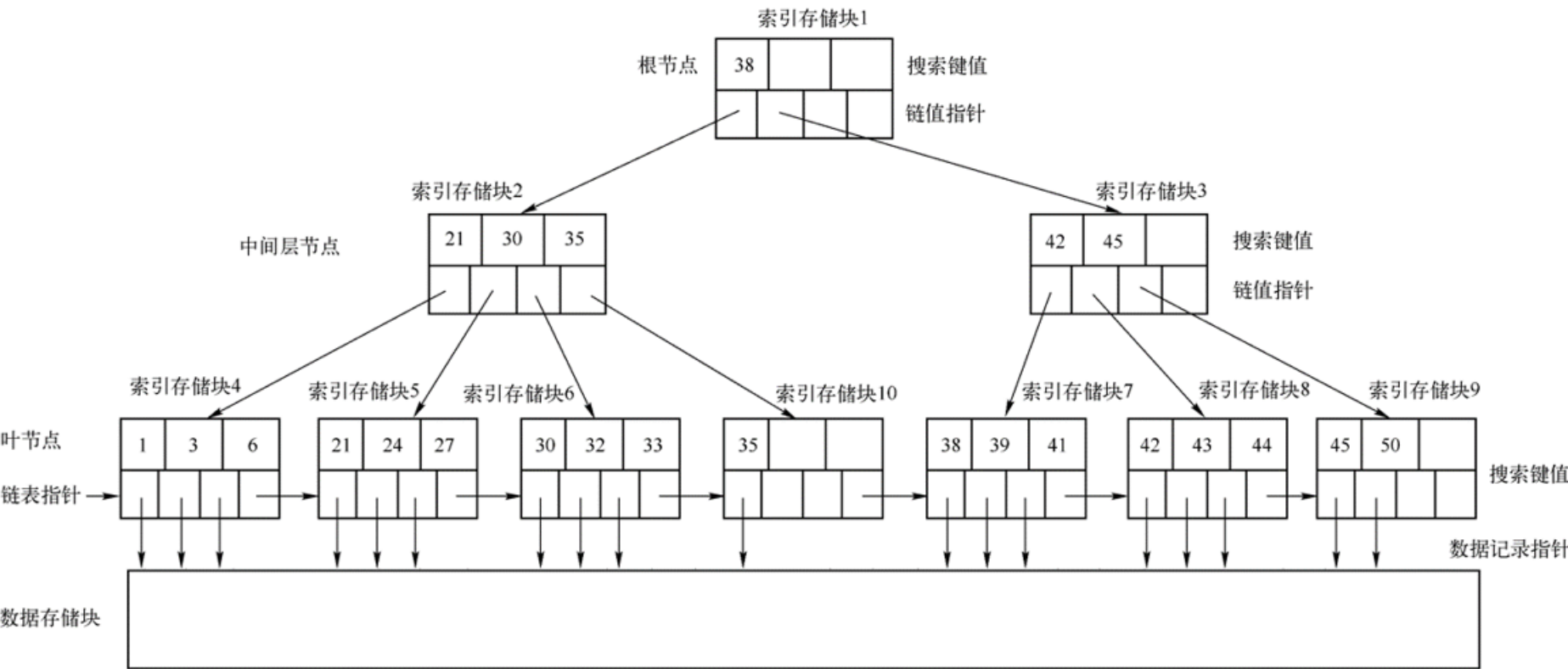
B+树记录插入(数据结构的方法)

在书上例 5-3 的3 阶B+树索引结构中， 假设插入键值为33的数据记录后的B+树如下：



B+树记录插入(教材的方法可以商榷)

在例 5-3 的3 阶B+树索引结构中， 假设插入键值为33的数据记录后的B+树如下：

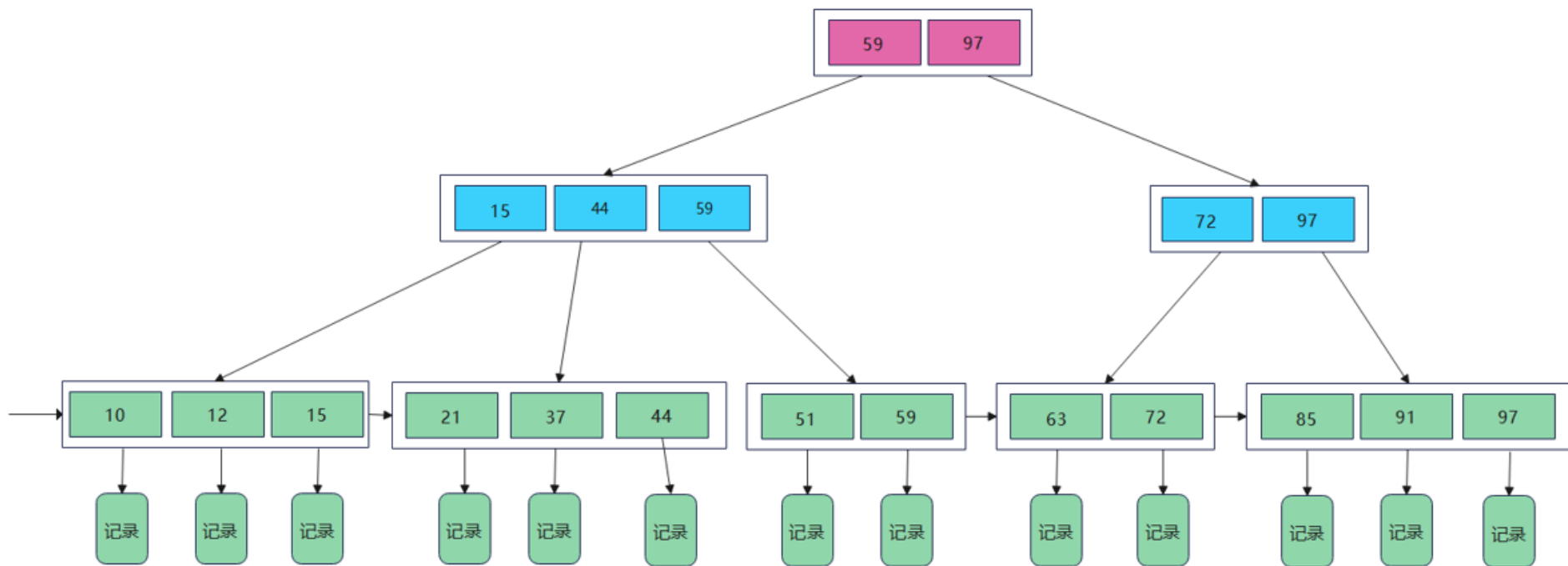


B+树记录删除

B+树的删除也和B树十分类似，它有下面几种情况：

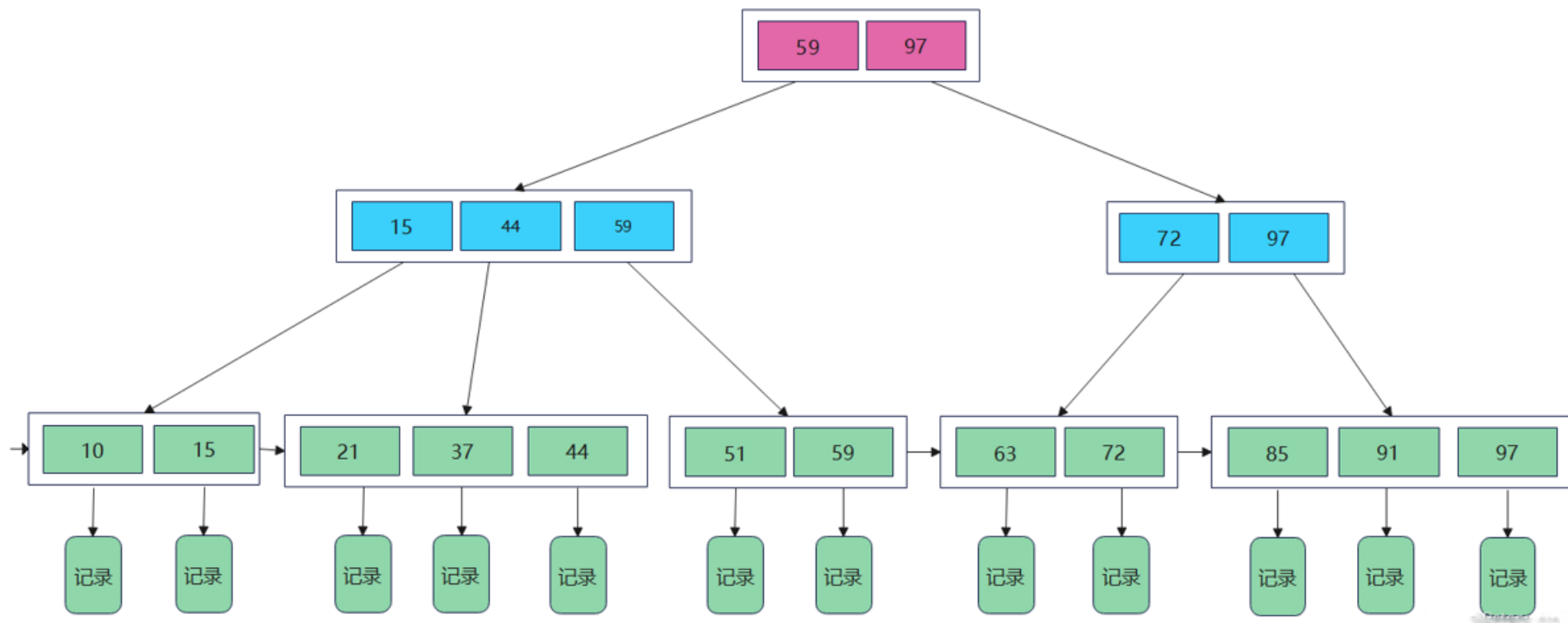
- ① 找到存储有该关键字所在的结点时，由于该结点中关键字个数 $\geq \lceil m/2 \rceil$ ，做删除操作不会破坏 B+树，则可以直接删除；
- ② 当删除某结点中最大或者最小的关键字，就会涉及到更改其双亲结点一直到根结点中所有索引值的更改。
- ③ 当删除该关键字，导致当前结点中关键字个数小于 $\lceil m/2 \rceil$ ，若其兄弟结点中含有多余的关键字，可以从兄弟结点中借关键字完成删除操作。
- ④ 第 3 种情况中，如果其兄弟结点没有多余的关键字，则需要同其兄弟结点进行合并。
- ⑤ 当进行合并时，可能会产生因合并使其双亲结点破坏 B+树的结构，需要依照以上规律处理其双亲结点。

B+树删除记录的示例



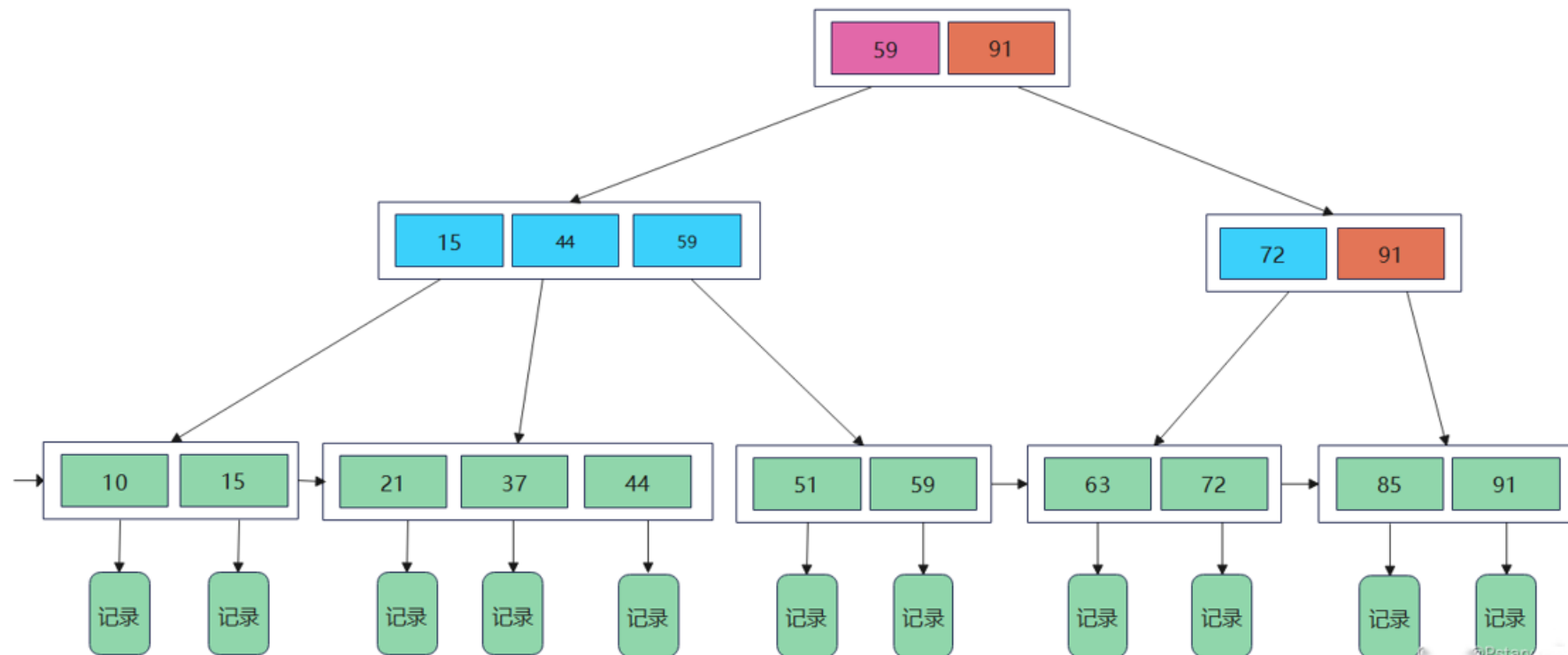
B+树记录删除

(1) 删除12, 包含12的叶子节点的关键字个数为3, 当删除12时, 关键字个数仍然大于等于 $\lceil m/2 \rceil$, 可以直接删除:



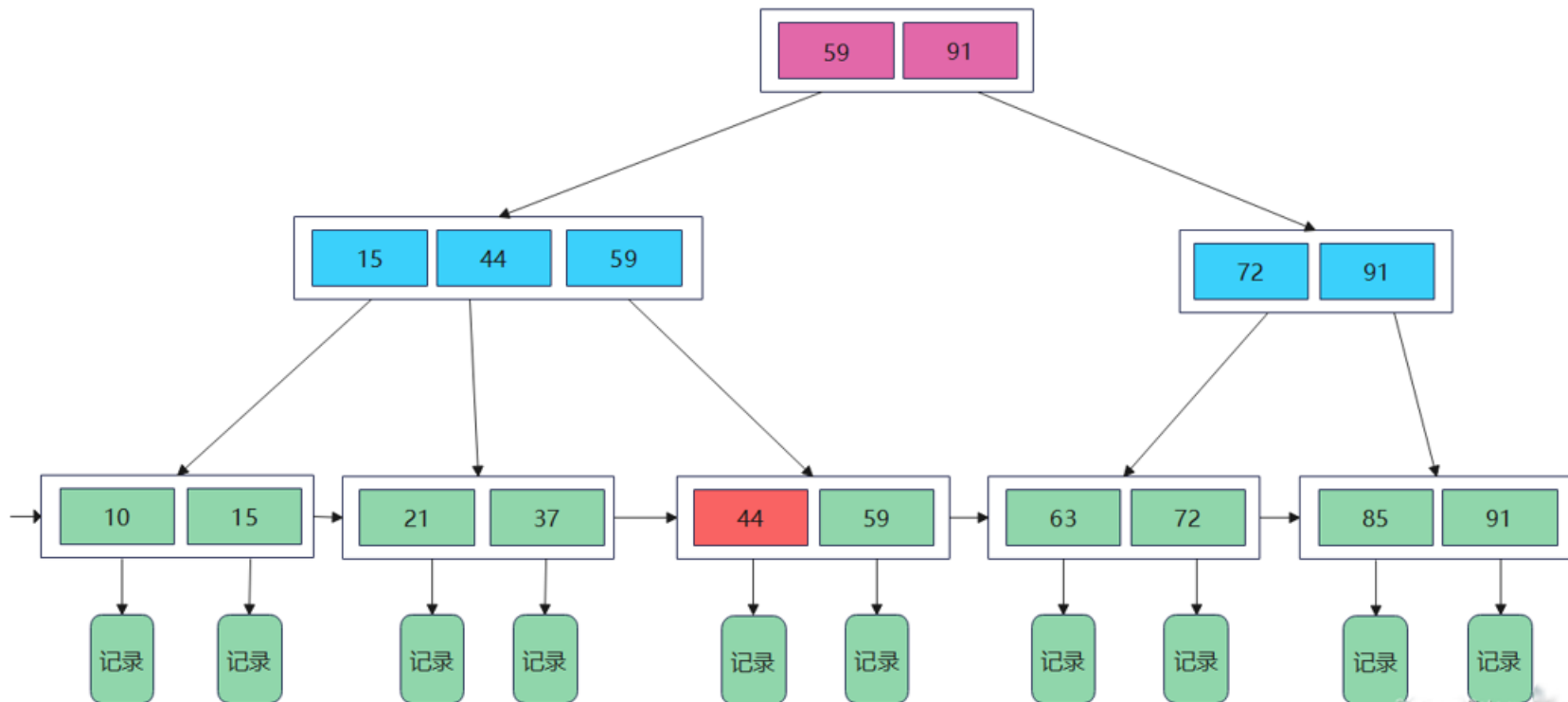
B+树记录删除

(2) 删除97, 97为整个B+树中元素最大的值, 当删除这个元素时, 需要修改从97的父节点到根节点中所有涉及到97的值, 将其修改为第二大的元素值 (在这个例子中第二大的元素为91) :



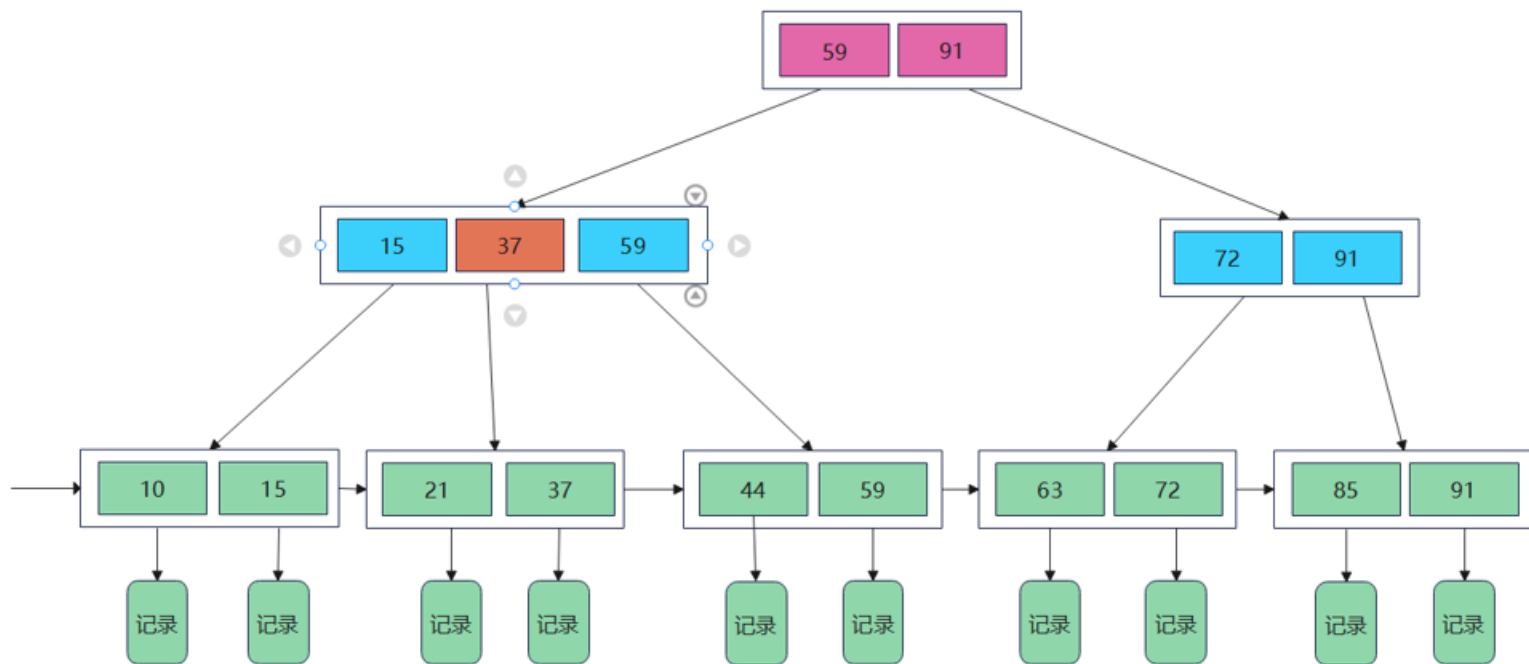
B+树记录删除

(3) 删除51, 此时51所在的结点只有59一个元素, 该节点的关键字个数小于 $\lceil m/2 \rceil$, 而它的兄弟结点元素个数大于 $\lceil m/2 \rceil$, 可以给该节点借一个:



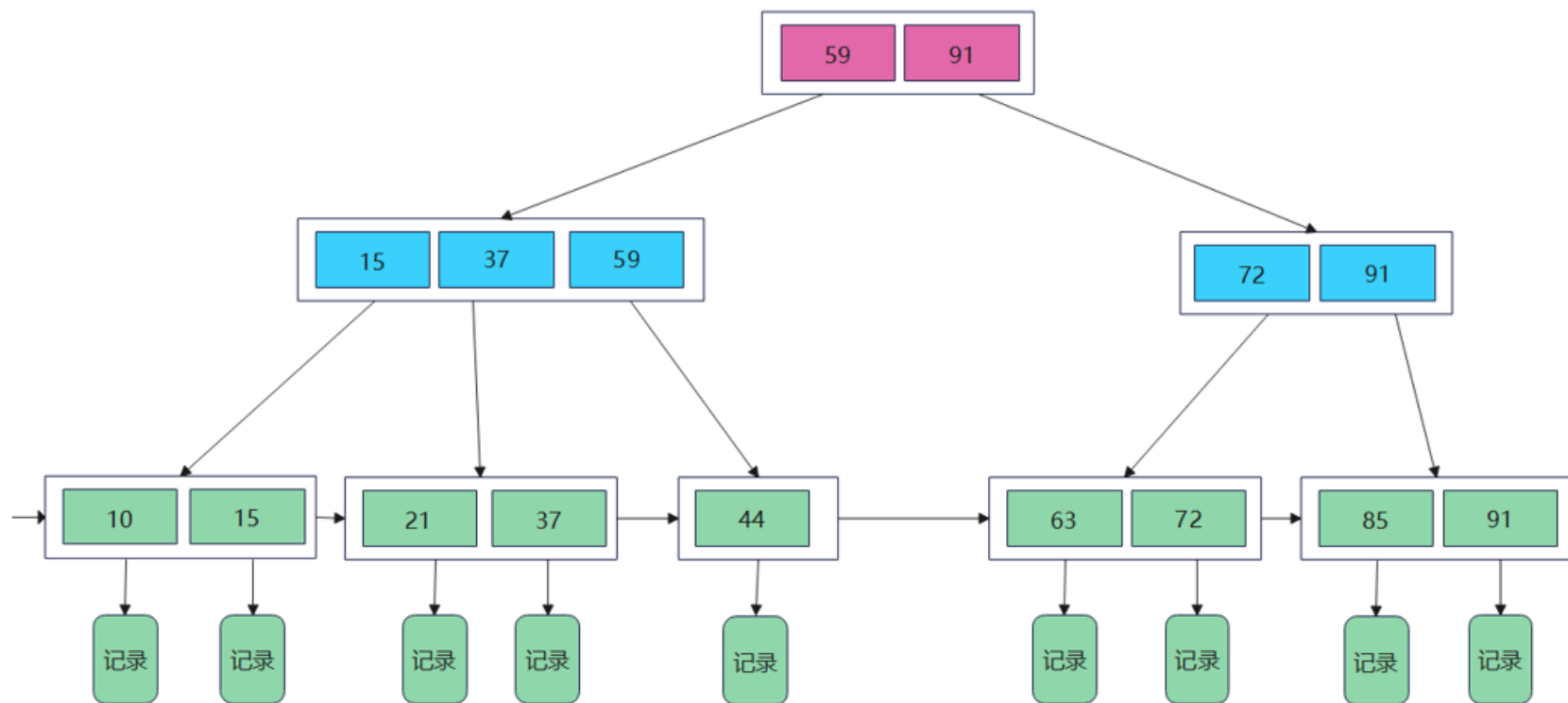
B+树记录删除

此时第二个叶子节点中的最大关键字为37，因此需要修改其父节点的值：



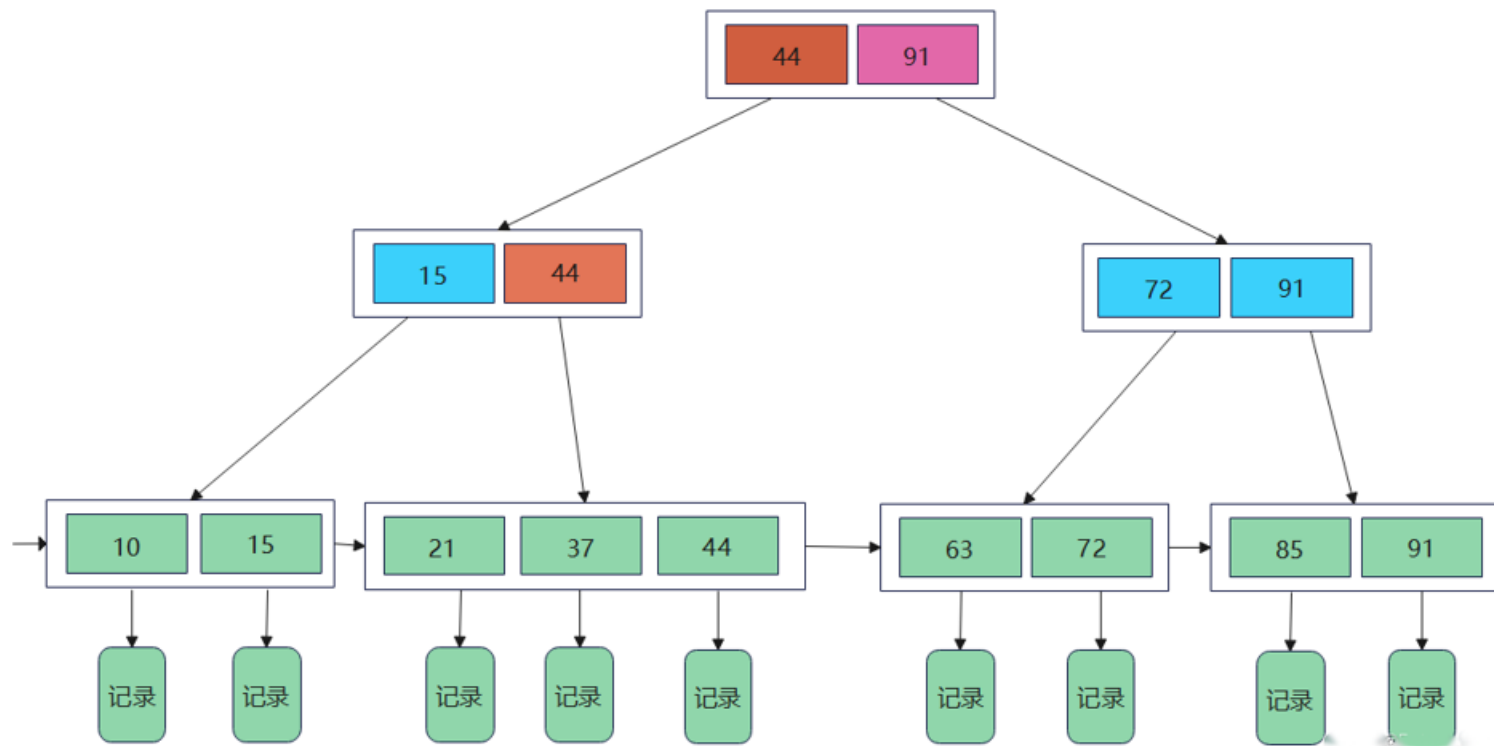
B+树记录删除

(4) 删除59，此时59所在结点的关键字个数小于 $\lceil m/2 \rceil$ ：



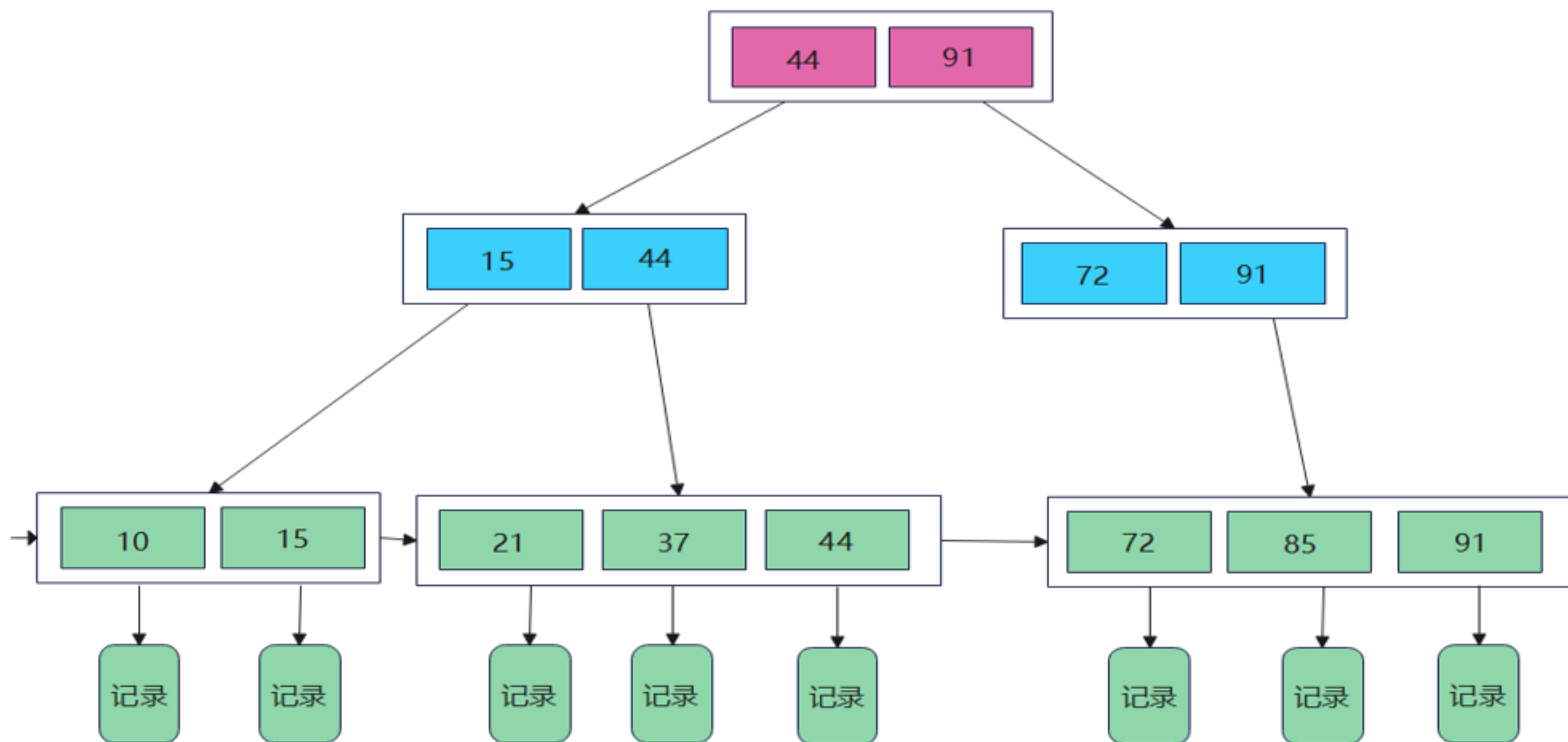
B+树记录删除

并且该结点的兄弟结点的个数都为 $\lceil m/2 \rceil$ ，无法给它借关键字，因此将该结点与兄弟结点进行合并，结点合并之后，需要注意修改祖先节点相关的值：



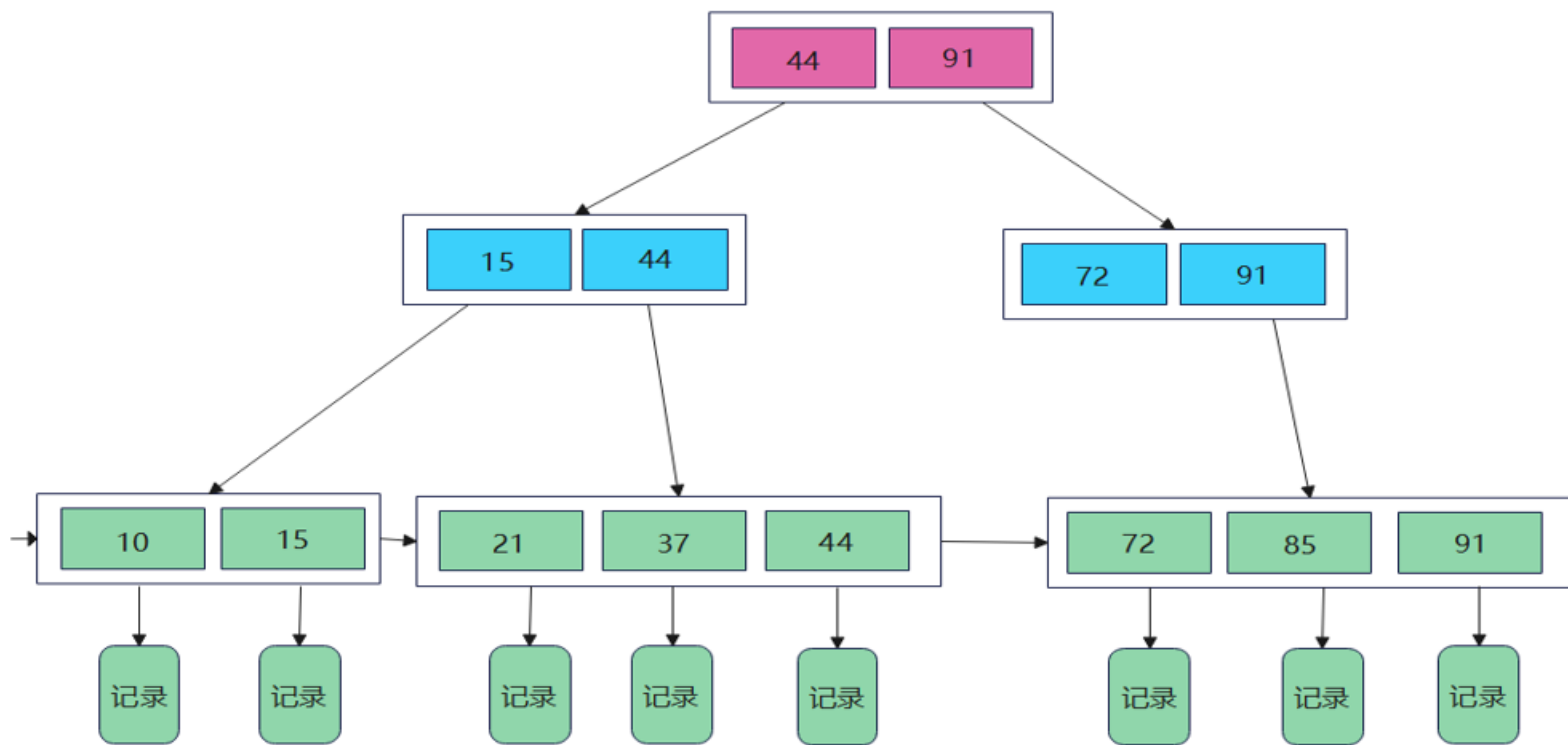
B+树记录删除

(5) 删除63，此时63所在的结点关键字个数小于 $\lceil m/2 \rceil$ ，并且兄弟节点无法给该节点提供关键字，因此该节点和其兄弟节点进行合并：



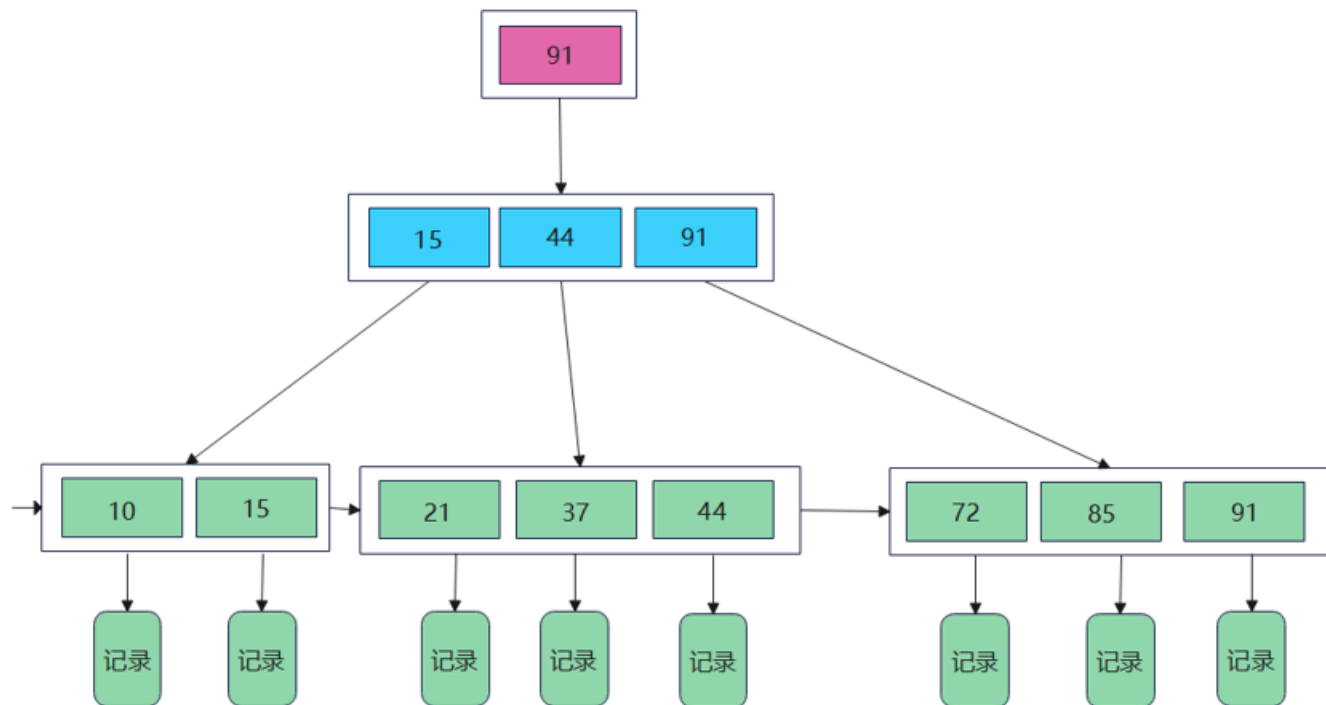
B+树记录删除

从图中可以发现，合并后的结点的父节点应该删除72，这时父节点关键字的个数小于 $\lceil m/2 \rceil$ ：



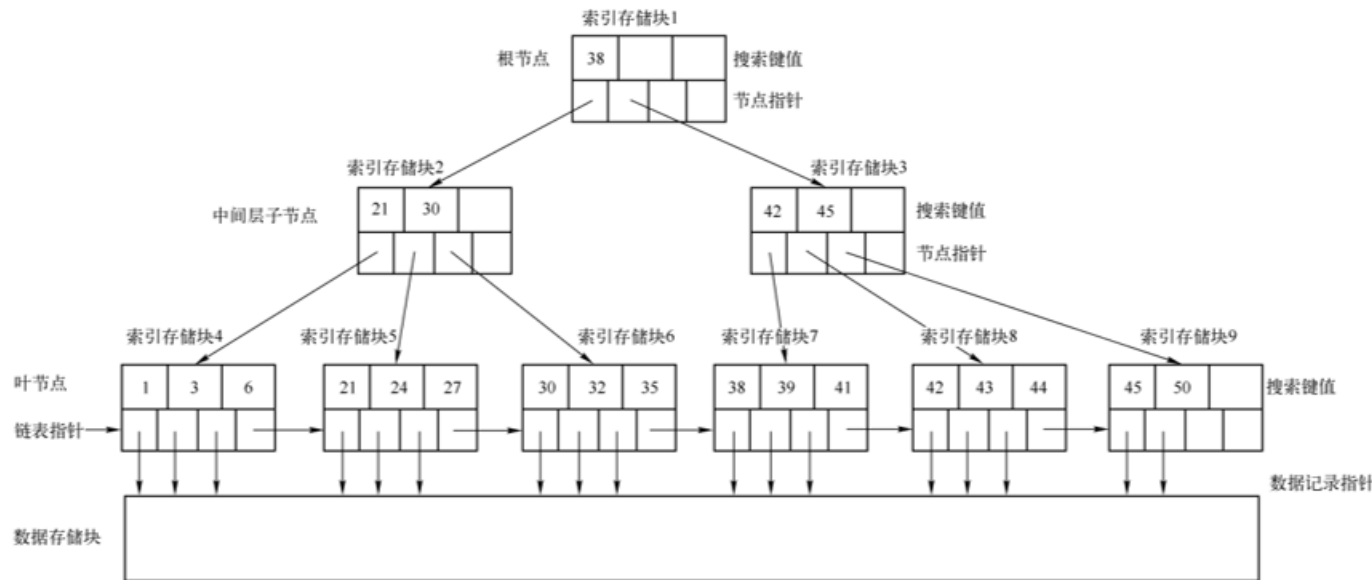
B+树记录删除

并且此时91所在结点的兄弟节点无法给该节点提供关键字，因此该结点和兄弟节点合并，并且需要修改合并后的节点的父节点的关键字值，合并后的结点的关键字个数满足条件。



B+树的使用效率

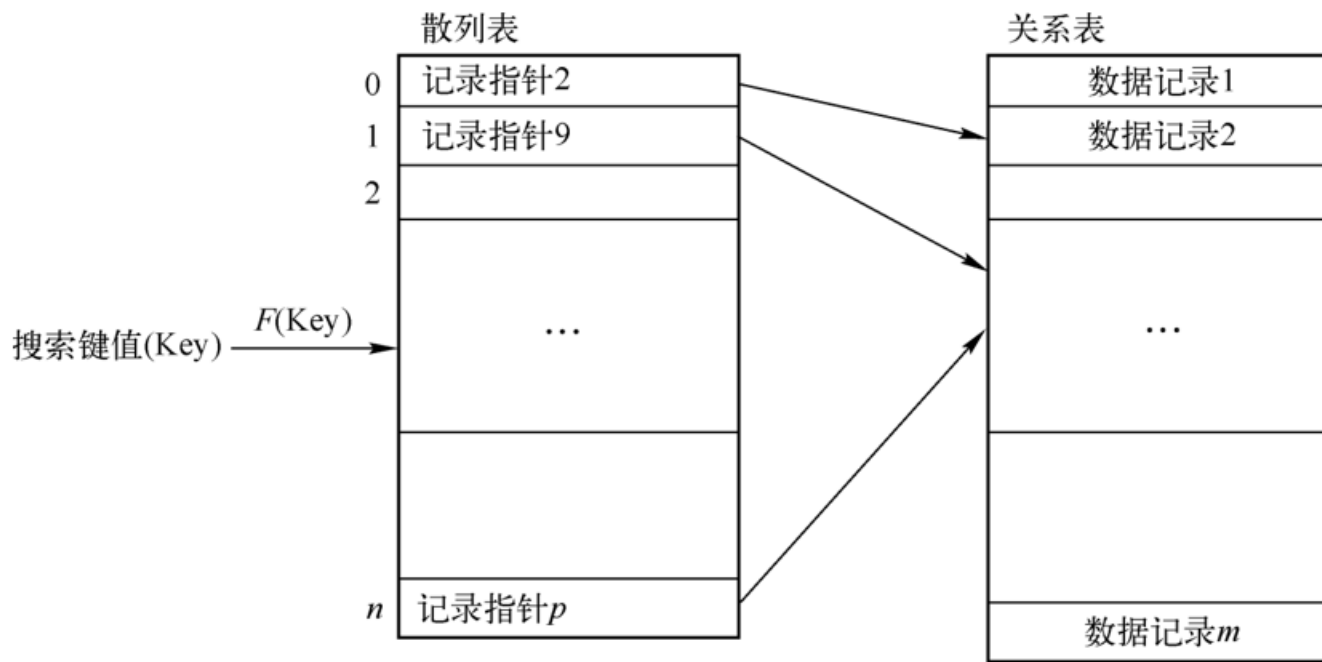
- 使用B+树实现关系表数据查询、插入、删除操作的时间开销主要与B+树的高度成正比
- 一般来讲，在给定关键字数量下，索引存储块越大，B+树的高度越低。



- ① 假设当索引存储块容量为4KB，搜索键为4字节整数，键值指针为8字节时，从公式 $4n+8(n+1) \leq 4096$ ，可以得到索引节点的最大阶数 $n=340$ ；则B+树索引节点存储块能够容纳的键-指针对最大数目为340；
- ② 为了确保B+树索引效率，通常要求索引节点存储块容纳的键-指针对最小数目为 $n/2=170$ ；
- ③ 假设当前B+树中各索引节点均容纳键-指针对数目为255。那么，该B+树索引结构有255个中间层节点、 255^2 个叶节点、 255^3 个数据记录指针，即3层B+树索引结构可以访问 255^3 个数据记录的关系表。

5、散列索引（Hash索引）

- 散列（哈希）索引是一种数据库索引结构，它通过将索引键值经过哈希函数计算后映射到一个固定大小的散列地址上，从而实现快速的数据检索。
- 存放哈希值对应记录指针的存储单元称为“桶” (Bucket)。 在内存的散列索引结构中，每个桶存储记录指针或记录链表。 在磁盘文件的散列索引结构中， 每个桶则存储实际数据记录。



散列函数的选择

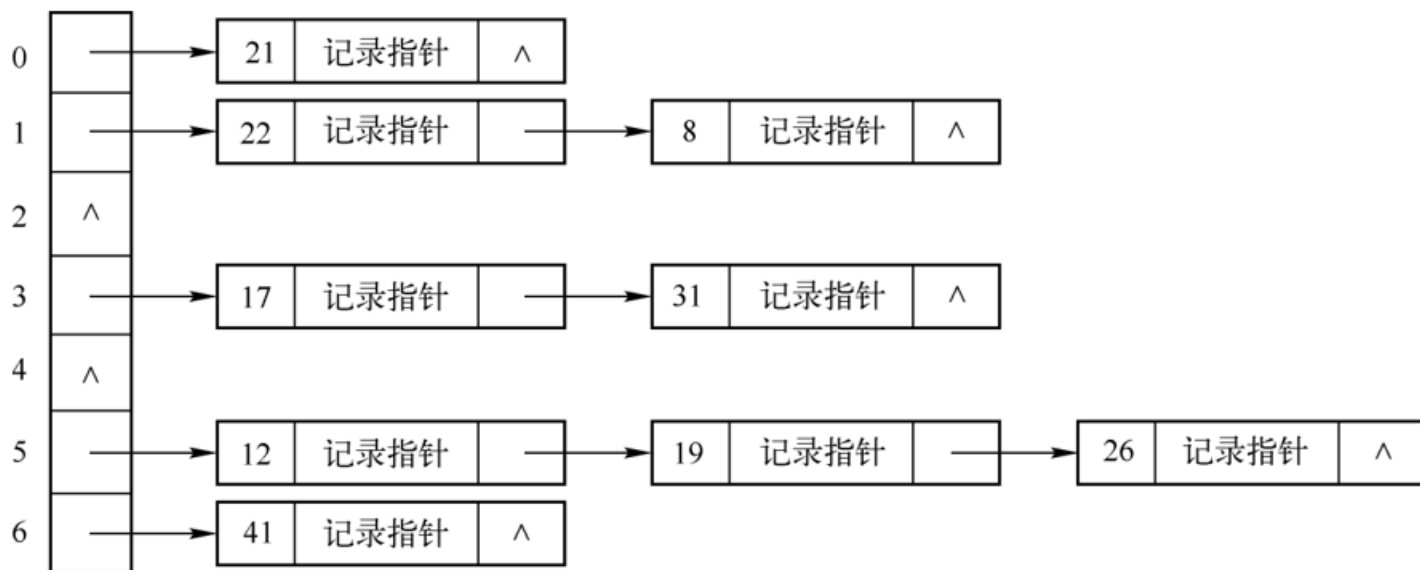
- ① 直接寻址法
- ② 数字分析法
- ③ 平方取中法
- ④ 取随机数法
- ⑤ 除留取余法

散列冲突解决

- ① 开放寻址法
- ② 再哈希法
- ③ 公共溢出区
- ④ 链地址法

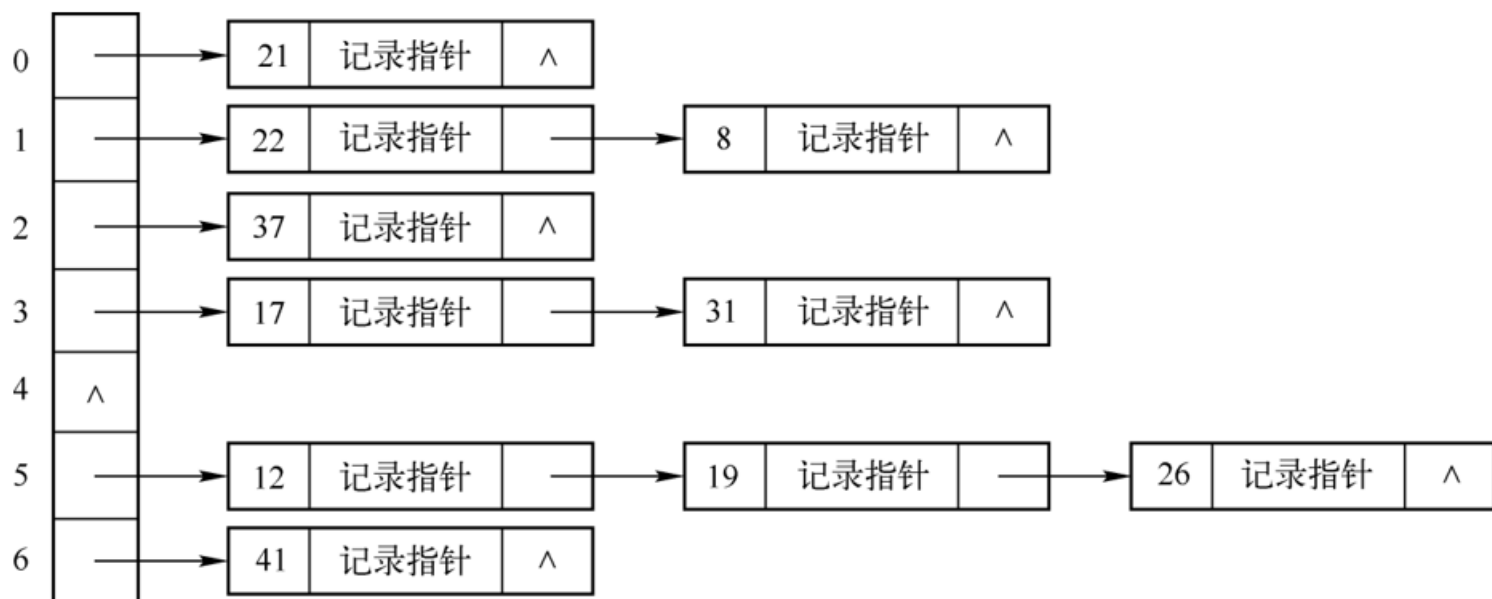
链表地址法的散列索引示例

假设散列表桶数为7，每个桶仅存储一个链表首指针，链表节点存储(键值,数据记录指针,链表指针)索引。当有多个键值映射到同一个桶时，将各键值—记录指针节点放入该桶链表中。例如，有一组键值key为{12,17,22,8,19,31,41,26,21}，其散列函数为 $F(\text{key}) = \text{key} \bmod 7$ ，使用链地址法所构建的散列表如图



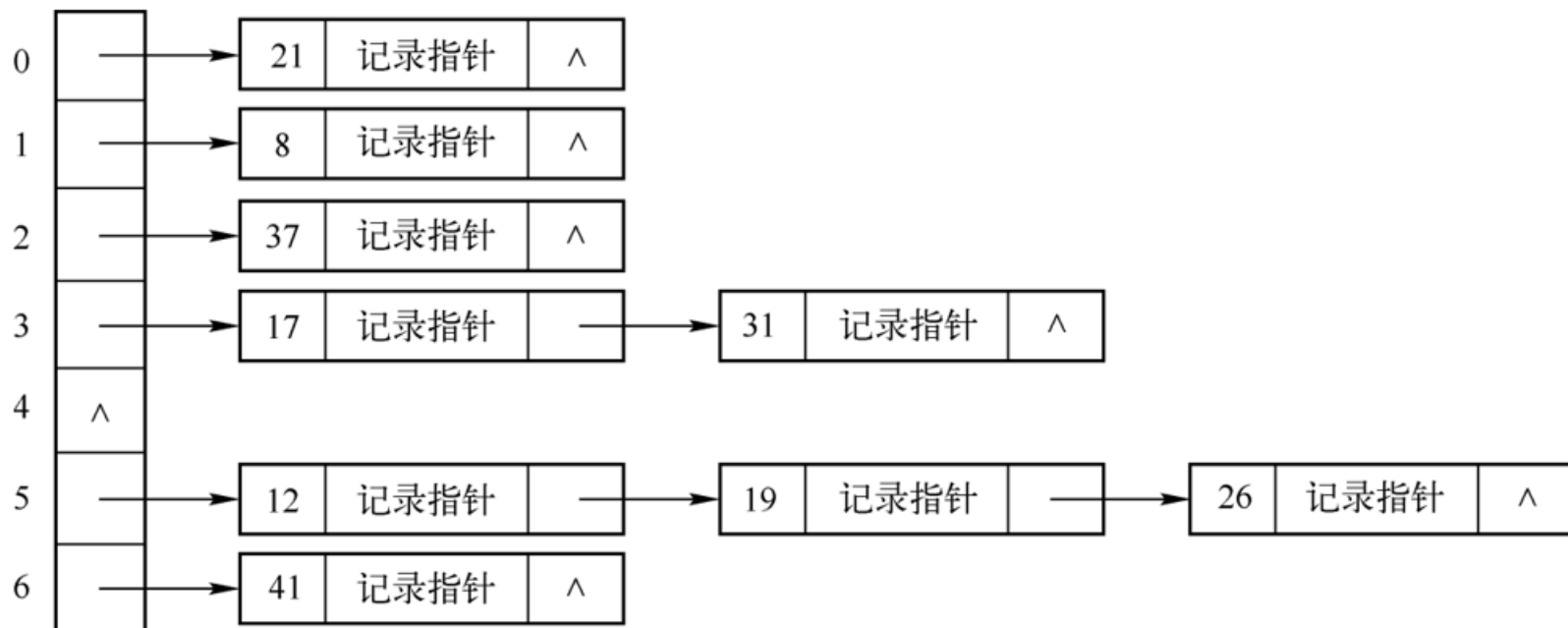
散列索引插入示例

在使用上图散列表索引的关系表中，当插入键值 $\text{key}=37$ 的数据记录时，同步计算新插入记录键值的散列值 $F(37)=2$ 。在散列表中该值对应的桶单元为空，则将新记录的索引数据 $(37, \text{记录指针}, \wedge)$ 放入一个新链表节点中，并将该节点的指针放入该桶单元；该记录插入后，散列表索引结构如图。



散列索引删除示例

在使用上图散列索引的关系表中，**当删除键值key=22的数据记录时**，同步计算该删除记录的**键值散列值 $F(22)=1$** 。在散列表偏移值为1的桶单元中获取链表指针，通过该指针找到含有键值22的节点，删除该节点，并将该节点的后继节点指针放入该桶单元。删除该记录后，散列表索引结构如图所示。



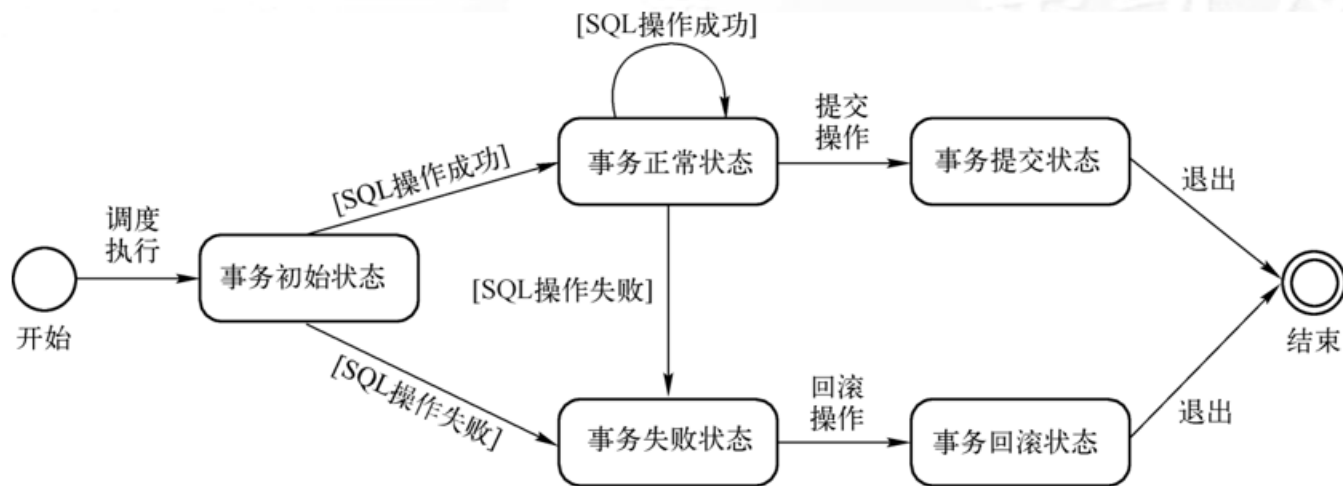
5.4 事务管理

1、事务 (transaction)

主要是更新操作

一个数据库操作序列，是数据库应用程序的基本逻辑单元。这些操作要么都做，要么都不做，是一个不可分割的执行单位。

事务状态：根据事务状态变迁，事务状态有：初始状态、正常状态、失败状态、回滚状态、事务提交、事务结束。



事务的两种结局：①提交而结束，其对DB的更新才能被其它事务访问；
②事务失败，事务所作的DB更新必须回滚。

对故障事务处理的依据：事务是已提交，还是未提交。

2、事务的特征

(1) 原子性 (Atomicity) : 原子性指事务是一个不可分割的工作单元, 要么全部执行成功, 要么全部执行失败, 不存在部分执行的情况。

特点: 当事务中的所有操作都成功完成时, 整个事务被提交 (Commit) ; 如果任何一个操作失败, 整个事务将被回滚 (Rollback) 到事务开始前的状态。

2、事务的特征

(2) 一致性(Consistency): 指事务执行前后, 数据库从一个一致性状态转变到另一个一致性状态, 保证数据的完整性和约束。

特点: 事务执行过程中, 数据库的约束和规则不会被破坏, 确保数据的正确性和完整性。

例如: 一个帐号的收支之差应等于余额。飞机订票系统, 事务执行前后, 座位与订出座位等信息必须一致。

2、事务的特征

(3). 隔离性(Isolation): 隔离性指多个事务并发执行时, 每个事务的操作应该与其他事务的操作相互隔离, 互不干扰。

特点: 通过隔离级别(如读未提交、读提交、可重复读、串行化)来控制事务之间的影响, 避免并发执行时出现脏读、不可重复读、幻读等问题。

2、事务的特征

(4). 持久性(Durability): 持久性指一旦事务提交, 其所做的修改将永久保存在数据库中, 即使系统发生故障也不会丢失。

特点: 数据库系统会将事务的提交结果持久化到磁盘中, 确保即使系统崩溃或断电, 数据也能够恢复到提交后的状态。

3、事务并发执行的主要原因

- ① **提高系统吞吐量：** 并发执行可以使系统在单位时间内处理更多的事务，提高系统的吞吐量。通过允许多个事务同时执行，系统可以更有效地利用资源，提高系统的处理能力。
- ② **减少平均等待时间：** 多个事务并发执行可以减少用户的平均等待时间，提高用户体验。用户可以更快地获取所需的数据或执行操作，从而提升用户满意度。
- ③ **提高资源利用率：** 允许多个事务并发执行可以更充分地利用系统资源，避免资源的闲置浪费。多个事务可以共享系统资源，提高系统的资源利用率。
- ④ **支持大规模应用：** 对于大规模高并发场景，必须支持多用户并发访问数据库。通过并发执行，系统可以满足大量用户的需求，保证系统的稳定性和可靠性。
- ⑤ **提高系统性能：** 并发执行可以提高系统的性能，减少系统的平均响应时间。通过允许多个事务同时执行，系统可以更快地处理请求，提高系统的性能。

4、事务SQL编程

- 事务定义语句

BEGIN

SQL 语句1

SQL 语句2

...

COMMIT

BEGIN

SQL 语句1

SQL 语句2

...

ROLLBACK

- ① **BEGIN** 或 **START TRANSACTION** 标明事务的开始;
- ② **Commit** 事务执行成功, 正常提交而结束
- ③ **Rollback** 事务执行失败, 撤销回滚而结束

事务SQL编程示例

假设有一个简单的银行系统，其中有两个表：
`accounts`（账户表）和
`transactions`（交易表），
希望实现一个转账功能，
即从一个账户转账到另一个账户。

首先，需要创建两个表：

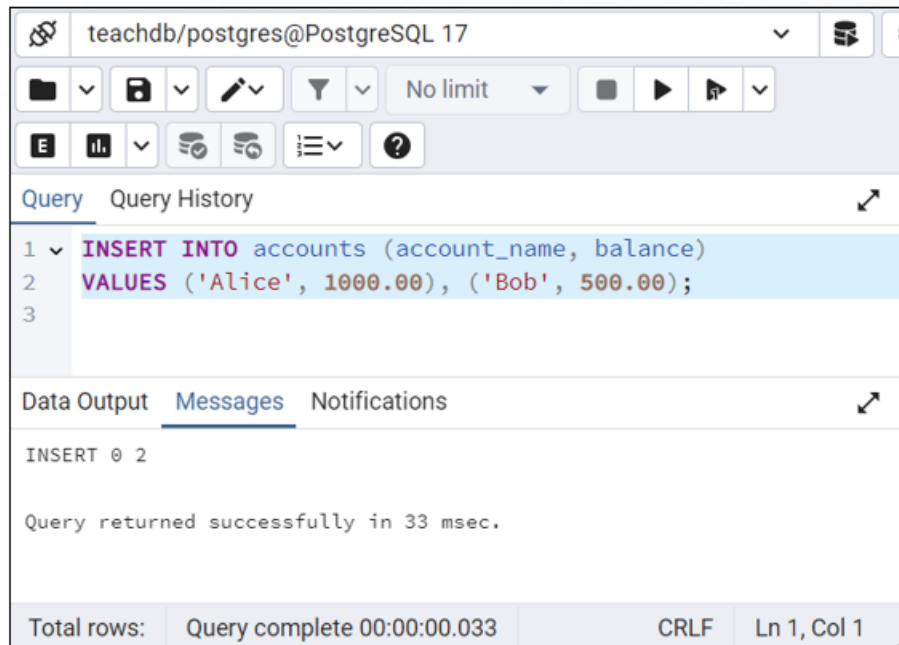
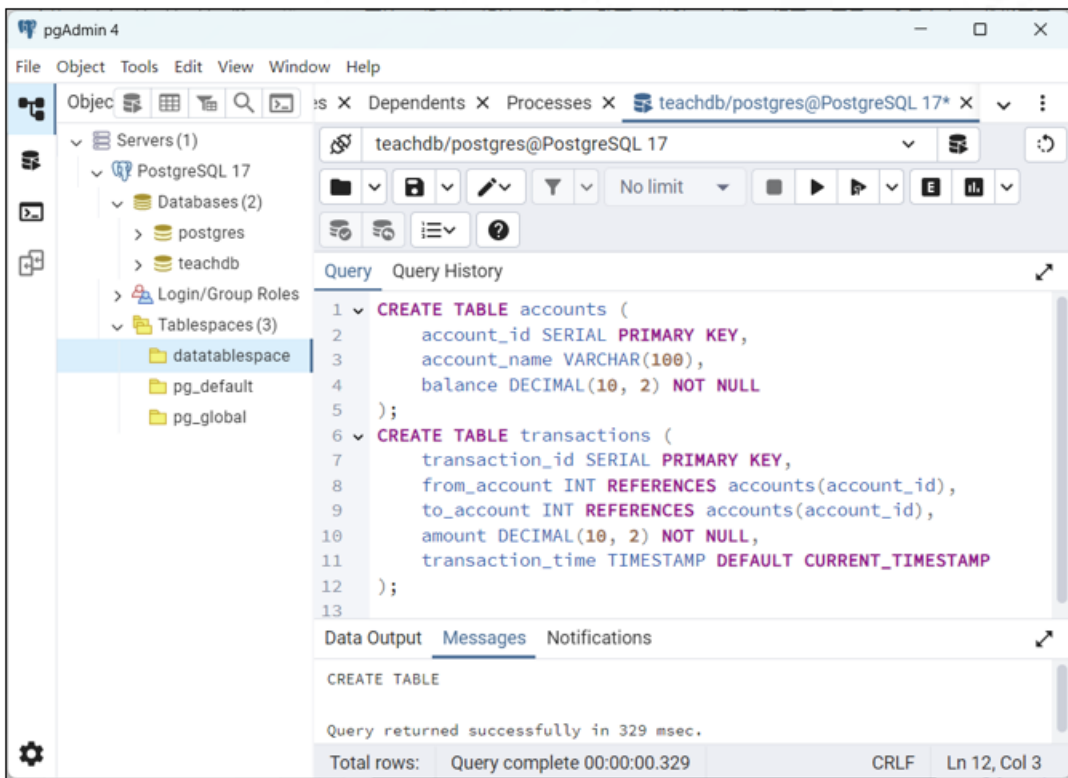
```
CREATE TABLE accounts (  
    account_id SERIAL PRIMARY KEY,  
    account_name VARCHAR(100),  
    balance DECIMAL(10, 2) NOT NULL  
);
```

```
CREATE TABLE transactions (  
    transaction_id SERIAL PRIMARY KEY,  
    from_account INT REFERENCES accounts(account_id),  
    to_account INT REFERENCES accounts(account_id),  
    amount DECIMAL(10, 2) NOT NULL,  
    transaction_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

插入示例数据到 accounts 表中

INSERT INTO accounts (account_name, balance) VALUES

('Alice', 1000.00), ('Bob', 500.00);



事务处理示例

将实现转账操作的事务处理程序。假设要将 200.00 从 Alice 的账户转账到 Bob 的账户。

BEGIN; -- 开始事务

```
UPDATE accounts -- 从 Alice 的账户中扣除金额
SET balance = balance - 200.00
WHERE account_name = 'Alice';
```

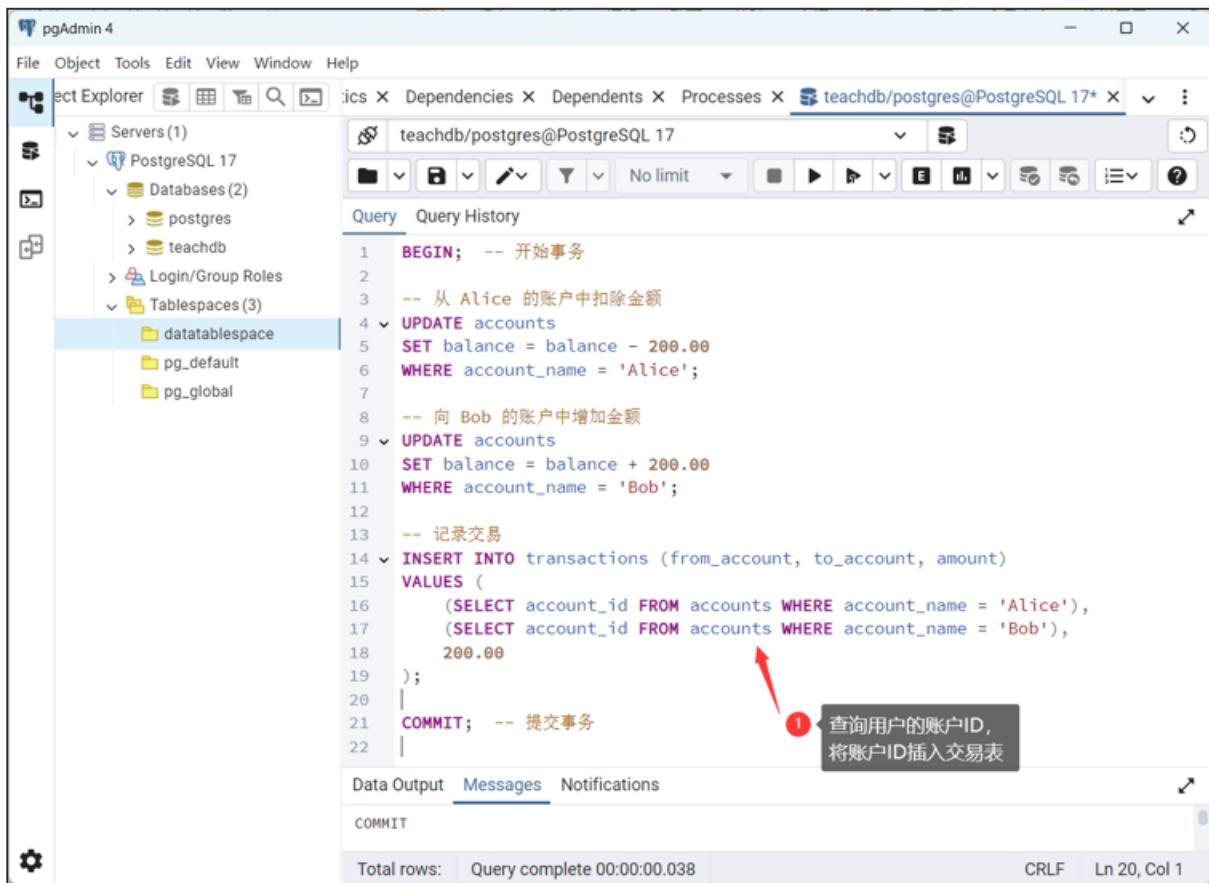
```
UPDATE accounts -- 向 Bob 的账户中增加金额
SET balance = balance + 200.00
WHERE account_name = 'Bob';
```

```
INSERT INTO transactions (from_account, to_account, amount) -- 记录交易
VALUES (
    (SELECT account_id FROM accounts WHERE account_name = 'Alice'),
    (SELECT account_id FROM accounts WHERE account_name = 'Bob'),
    200.00
);
```

COMMIT; -- 提交事务

事务处理示例

将实现转账操作的事务处理程序。假设要将 200.00 从 Alice 的账户转账到 Bob 的账户。



错误处理与回滚

在实际应用中，需要处理可能出现的错误，并在出现错误时回滚事务。以下是改进的示例，使用 PL/pgSQL 进行错误处理：

```
DO $$
DECLARE
    alice_balance DECIMAL(10, 2);
    bob_exists BOOLEAN;

BEGIN -- 开始事务
    -- 检查 Alice 的账户是否存在并获取余额
    SELECT balance INTO alice_balance
    FROM accounts
    WHERE account_name = 'Alice';

    -- 如果 Alice 的账户不存在，抛出异常
    IF alice_balance IS NULL THEN
        RAISE EXCEPTION 'Account for Alice does not exist.';
    END IF;
```

错误处理与回滚

-- 检查 Alice 的账户余额是否足够

IF alice_balance < 100.00 THEN

RAISE EXCEPTION 'Insufficient balance in Alice's account.';

END IF;

-- 检查 Bob 的账户是否存在

SELECT EXISTS (

SELECT 1 FROM accounts WHERE account_name = 'Bob'

) INTO bob_exists;

-- 如果 Bob 的账户不存在，抛出异常

IF NOT bob_exists THEN

RAISE EXCEPTION 'Account for Bob does not exist.';

END IF;

-- 从 Alice 的账户中扣除金额

UPDATE accounts

SET balance = balance - 100.00

WHERE account_name = 'Alice';

错误处理与回滚

-- 向 Bob 的账户中增加金额

UPDATE accounts

SET balance = balance + 100.00

WHERE account_name = 'Bob';

-- 记录交易

INSERT INTO transactions (from_account, to_account, amount)

VALUES (

(SELECT account_id FROM accounts WHERE account_name = 'Alice'),

(SELECT account_id FROM accounts WHERE account_name = 'Bob'),

100.00

);

EXCEPTION

WHEN OTHERS THEN

-- 回滚事务

ROLLBACK;

RAISE NOTICE 'Transaction failed: %', SQLERRM;

END \$\$; --事务结束

错误处理与回滚

pgAdmin 4

File Object Tools Edit View Window Help

Object Explorer

- Servers (1)
 - PostgreSQL 17
 - Databases (2)
 - postgres
 - teachdb
 - Login/Group Roles
 - Tablespaces (3)
 - datatablespace
 - pg_default
 - pg_global

teachdb/postgres@PostgreSQL 17

Query Query History

```
27 -- 如果 Bob 的账户不存在, 抛出异常
28 IF NOT bob_exists THEN
29     RAISE EXCEPTION 'Account for Bob does not exist.';
30 END IF;
31
32 -- 从 Alice 的账户中扣除金额
33 UPDATE accounts
34 SET balance = balance - 100.00
35 WHERE account_name = 'Alice';
36
37 -- 向 Bob 的账户中增加金额
38 UPDATE accounts
39 SET balance = balance + 100.00
40 WHERE account_name = 'Bob';
41
42 -- 记录交易
43 INSERT INTO transactions (from_account, to_account, amount)
44 VALUES (
45     (SELECT account_id FROM accounts WHERE account_name = 'Alice'),
46     (SELECT account_id FROM accounts WHERE account_name = 'Bob'),
47     100.00
48 );
49
50 EXCEPTION
51 WHEN OTHERS THEN
52     -- 回滚事务
53     ROLLBACK;
54     RAISE NOTICE 'Transaction failed: %', SQLERRM;
55 END $$;
```

1 返回错误信息代码

2 前面处理出现错误时回滚事务所做的所有操作

3 事务正常结束时, 事务隐式自动提交

Data Output Messages Notifications

00

Query returned successfully in 34 msec.

Total rows: Query complete 00:00:00.034 CRLF Ln 42, Col 8

两次转事务处理后的数据库结果

teachdb/postgres@PostgreSQL 17

Query Query History

```
1 select * from accounts;
```

Data Output Messages Notifications

Showing rows: 1 to 2 Page No: 1 of 1

	account_id [PK] integer	account_name character varying (100)	balance numeric (10,2)
1	1	Alice	700.00
2	2	Bob	800.00

Total rows: 2 Query complete 00:00:00.116 CRLF Ln 1, Col 1

teachdb/postgres@PostgreSQL 17

Query Query History

```
1 select * from transactions;
```

Data Output Messages Notifications

Showing rows: 1 to 2 Page No: 1 of 1

	transaction_id [PK] integer	from_account integer	to_account integer	amount numeric (10,2)	transaction_time timestamp without time zone
1	1	1	2	200.00	2025-03-26 14:13:43.672266
2	6	1	2	100.00	2025-03-26 15:31:51.765906

Total rows: 2 Query complete 00:00:00.052 CRLF Ln 1, Col 1

SAVEPOINT创建保存点

在PostgreSQL中，SAVEPOINT语句用于创建一个保存点，可以在事务中设置保存点，以便在后续的操作中可以回滚到该保存点。这在处理复杂的事务逻辑时非常有用，可以将事务分成多个逻辑部分，并在需要时部分回滚。

- ① 创建保存点：SAVEPOINT sp1;
- ② 回滚到保存点：ROLLBACK TO SAVEPOINT sp1;
- ③ 释放保存点：RELEASE SAVEPOINT sp1;

SAVEPOINT保存点

下面是一个简单的示例，演示如何在PostgreSQL中使用SAVEPOINT来管理事务：

BEGIN; -- 开始事务

UPDATE accounts SET balance = balance - 100 WHERE account_id = 123; -- 扣除账户123的余额100

SAVEPOINT sp1; -- 创建保存点

UPDATE accounts SET balance = balance + 100 WHERE account_id = 456; -- 增加账户456的余额100

ROLLBACK TO SAVEPOINT sp1; -- 如果后续操作出现问题，可以回滚到保存点sp1

COMMIT; -- 提交事务

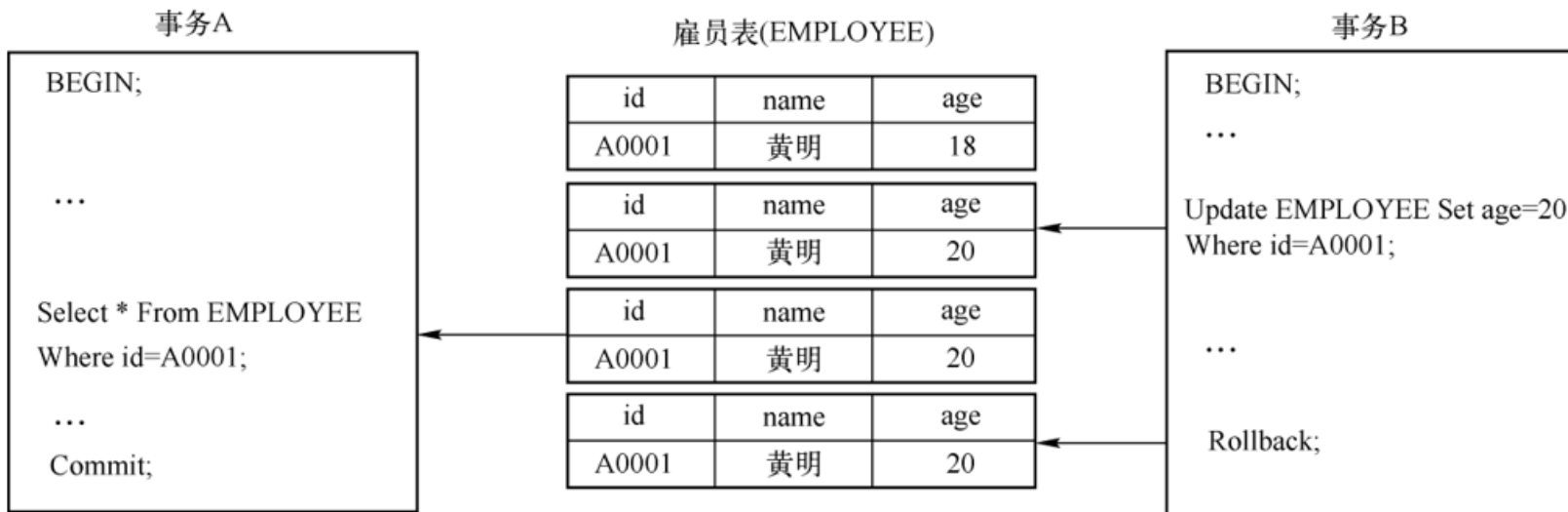
在上面的示例中，首先开始一个事务，然后扣除账户123的余额，接着创建一个保存点sp1，然后增加账户456的余额。如果后续操作出现问题，可以通过ROLLBACK TO SAVEPOINT sp1;回滚到保存点sp1，撤销增加账户456余额的操作。最后通过COMMIT;提交事务。

5.5 并发控制

数据库并发控制是指在多个用户同时访问数据库时，确保数据操作的正确性和一致性的一种机制。在数据库系统中，多个事务并发执行可能会导致一些问题，如丢失更新、脏读、不可重复读和幻读等，因此需要采取并发控制措施来解决这些问题。

1、并发执行可能会出现的问题

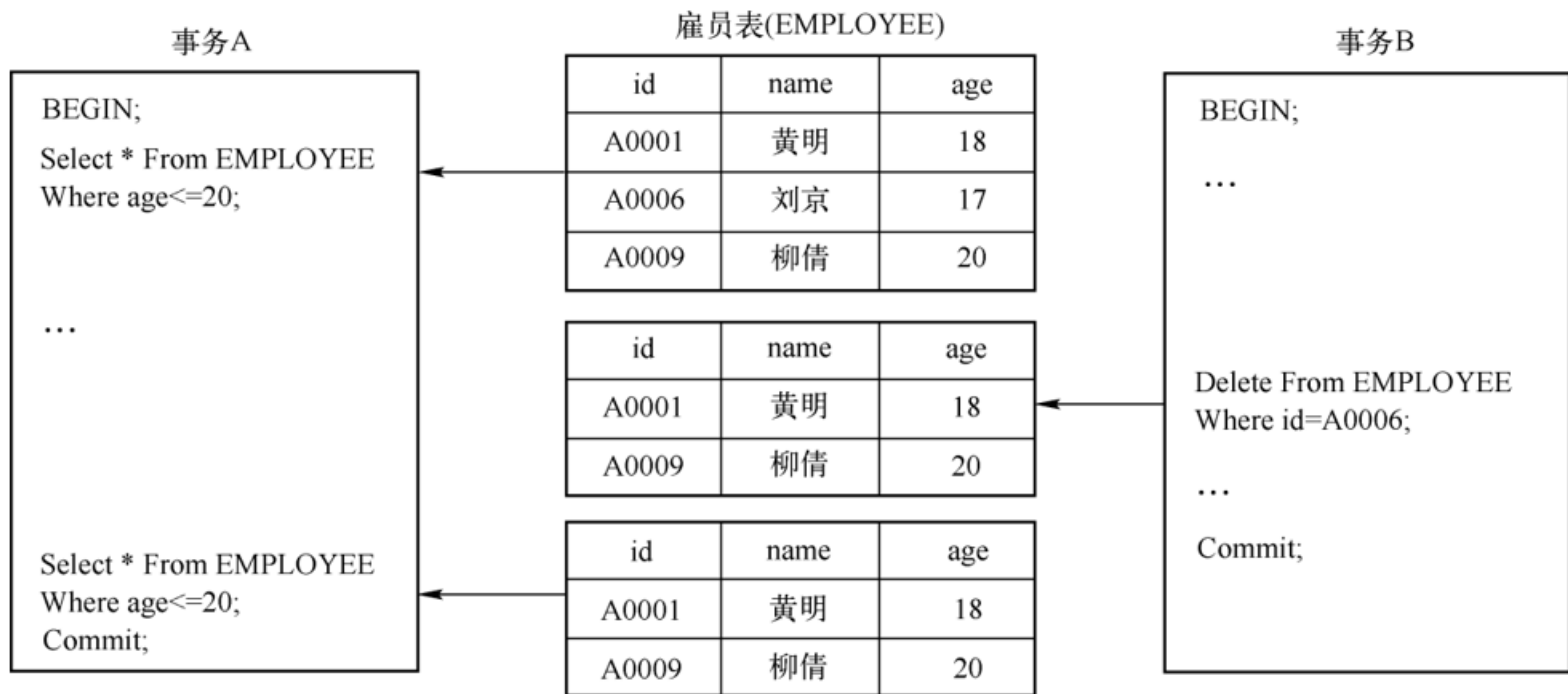
(1) 脏读 (Dirty Read) 是数据库并发控制中的一种问题，当一个事务对数据进行了修改但尚未提交时，另一个事务读取了这个未提交的数据，这就造成了脏读。脏读可能会导致数据的不一致性和错误，因为读取到的数据实际上是不完整或无效的。如果修改数据的事务最终回滚，那么读取到的数据就是无效的，这会对系统造成混乱和错误的结果。



如何避免脏读？

为了避免脏读，数据库系统通常会采取事务隔离级别中的“读已提交（Read Committed）”或更高级别的隔离级别，确保一个事务只能读取到已经提交的数据，避免读取到未提交的数据。通过合适的并发控制机制和隔离级别，可以有效地避免脏读问题，保证数据操作的正确性和一致性。

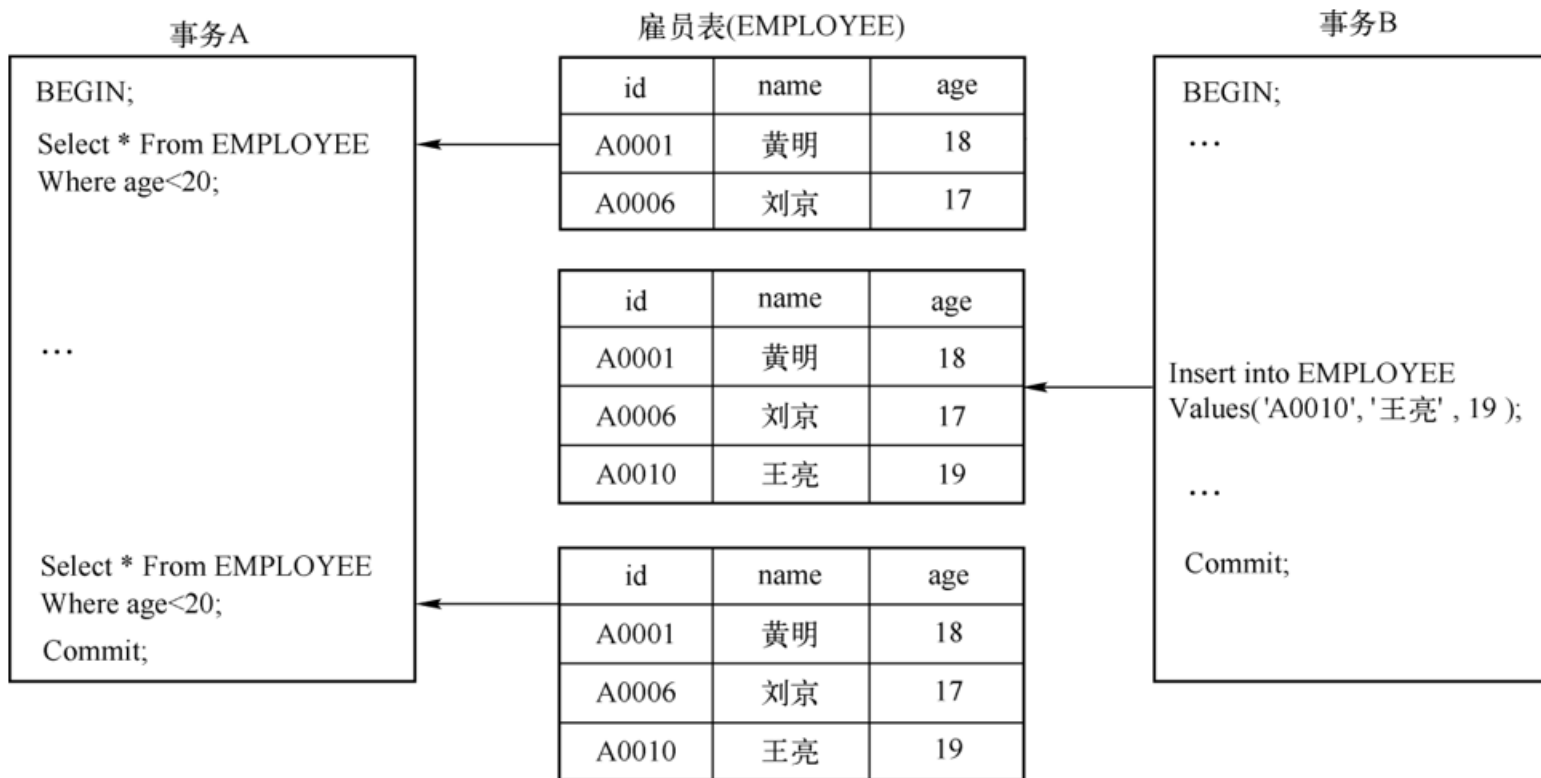
(2) 不可重复读 (Non-Repeatable Read) 是指在一个事务内，两次读取同一数据，但得到的结果却不一样。可能是因为在两次读取之间，另一个事务对数据进行了修改并提交，导致第一个事务在第二次读取时得到了不同的数据。



如何避免不可重复读？

不可重复读可能会导致数据的不一致性，因为同一数据在不同时间点的读取结果不一致。为了避免不可重复读问题，数据库系统通常会采取事务隔离级别中的“可重复读（Repeatable Read）”或更高级别的隔离级别，确保在一个事务内多次读取同一数据时，得到的结果是一致的，不会受到其他事务的影响。通过合适的并发控制机制和隔离级别，可以有效地避免不可重复读问题，保证数据操作的正确性和一致性。

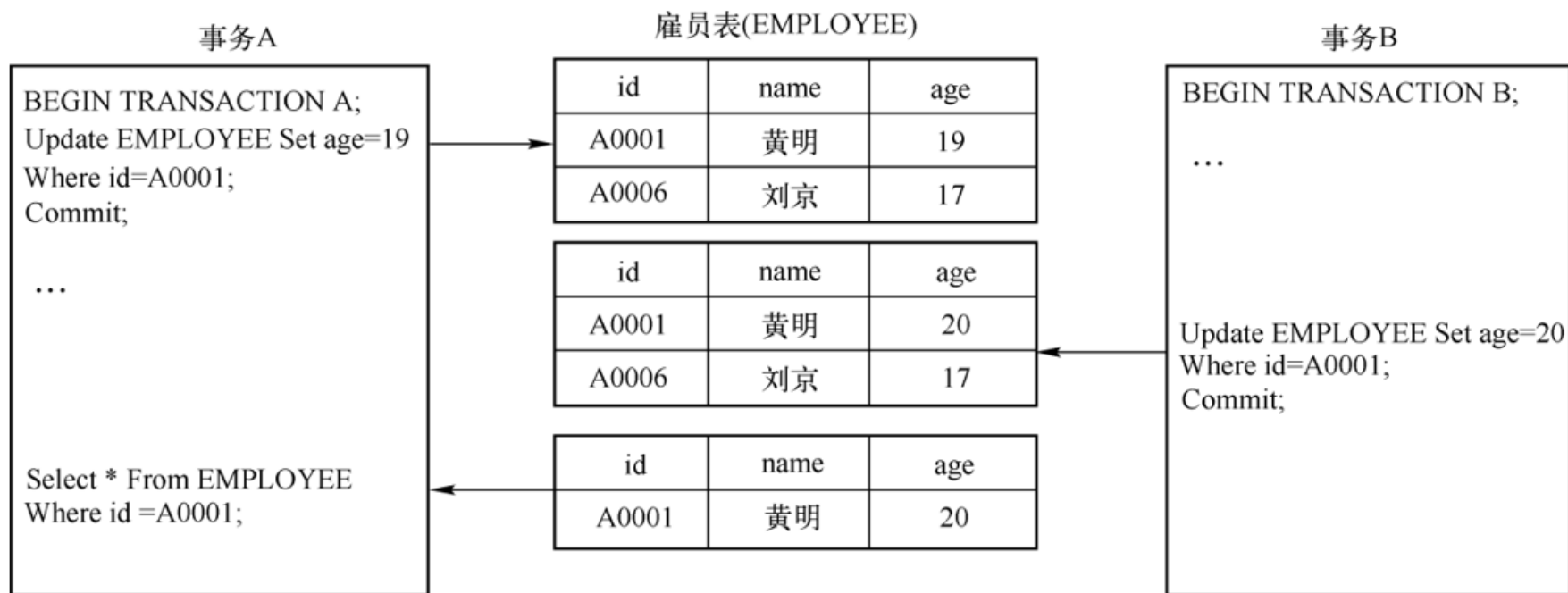
(3) 幻读 (Phantom Read) 是指在一个事务内多次查询同一范围的数据时，第二次查询发现有新的数据行被插入，导致第二次查询返回的结果集比第一次查询时更多。这种情况就好像出现了幻觉一样，所以称为“幻读”。



如何避免幻读？

幻读可能会导致数据的不一致性，因为同一范围的数据在不同时间点查询时，结果集发生了变化。为了避免幻读问题，数据库系统通常会采取事务隔离级别中的“可重复读（Repeatable Read）”或更高级别的隔离级别，确保在一个事务内多次查询同一范围的数据时，结果集是一致的，不会受到其他事务的影响。通过合适的并发控制机制和隔离级别，可以有效地避免幻读问题，保证数据操作的正确性和一致性。

(4) 丢失更新 (Lost Update) 是指当两个事务同时读取同一数据并进行修改时，后提交的事务可能会覆盖先提交的事务所做的修改，导致数据的更新操作丢失。具体来说，丢失更新的情况如下：事务T1读取数据D并进行修改。同时，事务T2也读取了数据D并进行修改。事务T2提交了修改，覆盖了事务T1所做的修改。



如何避免丢失更新？

丢失更新可能会导致数据的不一致性，因为后提交的事务覆盖了先提交的事务所做的修改。为了避免丢失更新问题，数据库系统通常会采取并发控制技术，如锁定机制或乐观并发控制，确保在多个事务同时修改同一数据时，能够正确处理更新操作，避免数据的丢失和覆盖。通过合适的并发控制机制，可以有效地避免丢失更新问题，保证数据操作的正确性和一致性。

2、并发事务调度

- 并发事务调度是数据库管理系统中用于控制并发事务执行顺序的机制。在多用户环境下，可能会有多个事务同时对数据库进行读取和修改操作，为了保证数据的一致性和完整性，数据库系统需要对这些并发事务进行合理的调度和管理。
- 并发事务调度的主要目标是保证事务的隔离性、一致性和持久性，同时尽可能提高系统的并发性能和吞吐量。

一.调度概念

调度：安排多个事务操作的执行序列。

事务调度的要求：事务内部操作的调度顺序应该与它们在事务中的顺序一致。

事务结束： COMMIT和ROLLBACK

WRITE事务写操作，简记为**W**，

READ事务读操作，简记为**R**，

WT(X)：事务T写数据库元素X，

RT(X)：事务T读数据库元素X，

S表示一个调度：

$S = R(A)W(A)R(C)W(C)R(B)W(B)$

T1	T2
Read(A)	
Write(A)	
	Read(C)
	Write(C)
	Commit
Read(B)	
Write(B)	

涉及两个事务的**并发**调度

串行调度

- 不同事务的活动在调度中是一个接一个执行的，没有交叉的运行。

T1	T2
READ(A) A:=A+A*0.1 WRITE(A) READ(B) B:=B+B*0.2 WRITE(B)	 READ(A) A:=A+10 WRITE(A) READ(B) B:=B-20 WRITE(B)

T1	T2
 READ(A) A:=A+A*0.1 WRITE(A) READ(B) B:=B+B*0.2 WRITE(B)	READ(A) A:=A+10 WRITE(A) READ(B) B:=B-20 WRITE(B)

- 两个串行调度的结果不同,但只要保持了数据库的一致性，最终的结果并不重要

可串行化调度

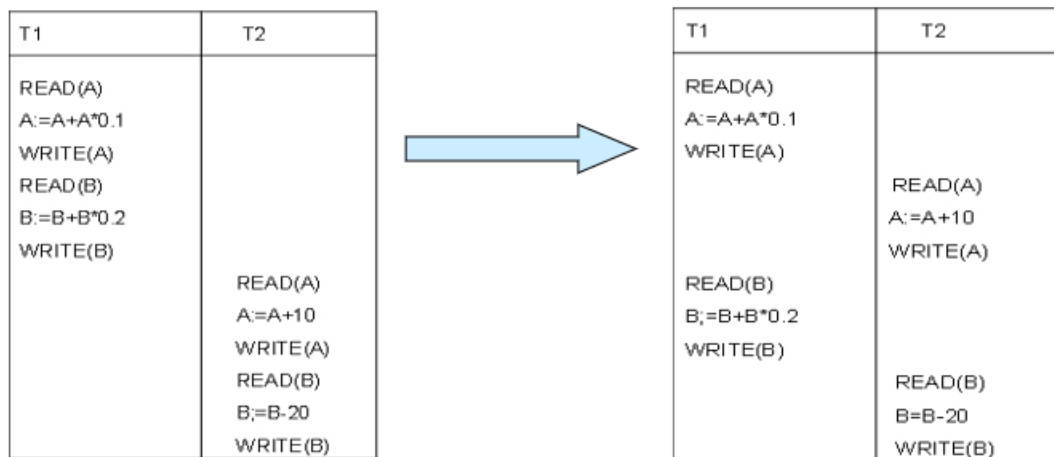
调度等价：如有两个调度S1和S2，在DB的任意相同初始状态下，所有读出的数据都是一样的，留给DB的最终状态也是一样的，则称S1和S2是调度等价的。

可串行化调度：对一事务集，如一个并发调度与一个串行调度等价，则称此并发调度是可串行化的(Serializable)。

注意：① 对n个事务有n! 种串行调度。每个串行调度执行结果可能不一样，可串行化只要求调度和其中某一个串行调度等价。

② 不同的可串行化调度是不一定等价的。

- **调度是可串行化的：**多个事务交叉调度的结果与某一个串行调度的结果相同
- DBMS认为**事务串行调度的结果保持了数据库的一致性，都是正确的**
- 如果调度是可串行化的，**系统认为其调度是一个正确的调度，保持了数据库的一致性**



- 如果并行调度与串行调度的结果相同，因此该调度是可串行的调度

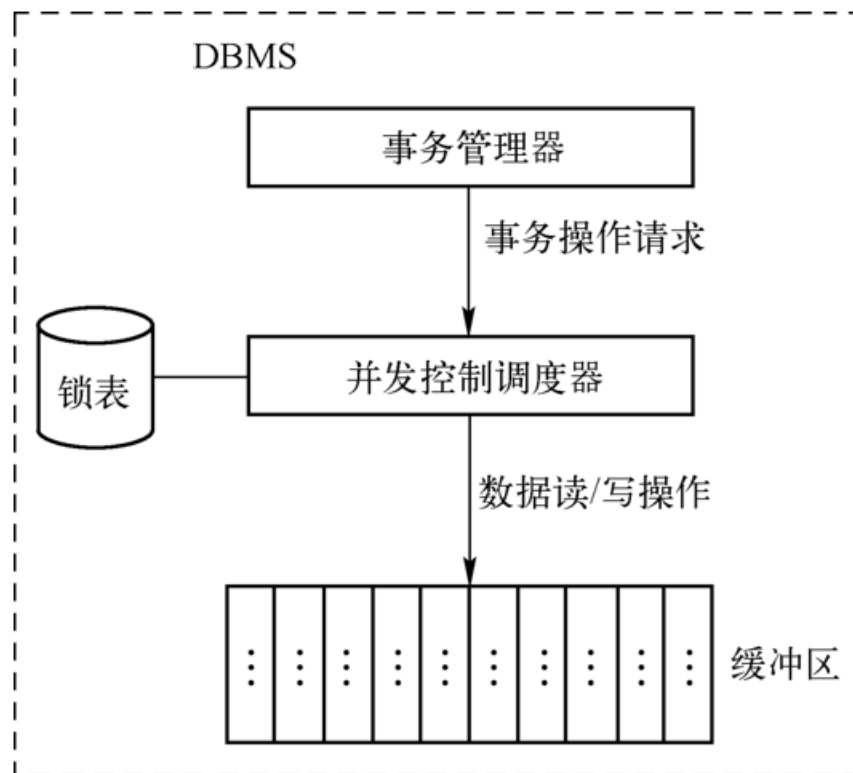
不可串行化调度

对于由多个事务组成的调度序列，如果这些事务的任何串行调度的结果都不与该调度序列的结果相等，则该调度是不可串行化的调度。

3、数据库的锁机制

为了实现可串行化调度，需要制定某种**规则（协议）**，只要事务按照该协议调度执行，就可以保证**调度可串行化**。

现有数据库主要采用**加锁方式**来保证调度执行的**可串行化**，即对敏感数据的读写进行**某种限制**，如同加了一把锁。



锁的基本类型

- 排它锁 (eXclusive, 也称 X锁, 用于写操作)

某数据对象在没有加任何锁的情况下, 一个事务可以对其加X锁, 而其他事务就不能对其再加任何锁。

- 共享锁 (Share, 也称 S锁, 用于读操作)

一个事务对某数据对象加了S锁后, 其他事务就不能对其加X锁, 但可以加S锁。

加锁的相容表

T2 T2申请锁	T1	X锁	S锁	无
	X锁	否	否	是
	S锁	否	是	是

是: 可以加锁 否: 不能加锁

(1) 一级封锁协议

任一事务在写某数据前，必须对其加上X锁，该事务结束后才释放。不采用S锁，读数据不用加锁。

S1	
T1	T2
Xlock(A)	
Write(A)	
	Xlock(A)被拒绝
Commit	
Ulock(A)	
	Xlock(A)
	Write(A)
	Commit
	Ulock(A)

消除了丢失更新

S2	
T1	T2
Xlock(A)	
Write(A)	
	Read(A)
	Xlock(B)
	B:=A+10
	Write(B)
	Commit
	Ulock(B)
Rollback	

产生读脏数据

S3	
T1	T2
Read(A)	
	Xlock(A)
	Read(A)
	A:=A2
	Write(A)
Read(A)	
	Commit
	Ulock(A)
	

产生不可重复读

(2) 二级封锁协议

满足一级封锁协议，且任一事务在读取某数据前，必须对其加上S锁，读完后就释放。

S2	
T1	T2
Xlock(A) Write(A)	Slock(A)被拒绝
Commit Unlock(A)	
	Slock(A) Read(A) B:=A+10 Unlock(A) Xlock(B) Write(B) Commit Unlock(B)

避免了读脏数据

S3	
T1	T2
Slock(A) Read(A) Unlock(A)	Xlock(A) Read(A) A:=A2 Write(A) Commit Ulock(A)
Slock(A) Read(A) Unlock(A)	

产生不可重复读

原因：过早释放S锁

(3) 三级封锁协议

满足一级封锁协议，且任一事务在读取某数据前，必须对其加上S锁，事务结束后就释放。

S3	
T1	T2
Slock(A) Read(A)	Xlock(A)被拒绝
Read(A) Commit Unlock(A)	
	Xlock(A) Read(A) A:=A2 Write(A) Commit Ulock(A)

该协议可以解决丢失更新、读脏数据以及不可重读的问题。

不同级别锁协议比较

加锁协议级别	排它锁	共享锁	不丢失更新	不脏读	可重复读
一级	全程加锁	不加	是	否	否
二级	全程加锁	开始时加锁， 读完数据释放锁定	是	是	否
三级	全程加锁	全程加锁	是	是	是

(4) 两阶段加锁协议

两阶段加锁事务：在一个事务中，如果加锁都在所有释放锁之前，则称这样的事务加锁规则称为两阶段加锁协议(2PL协议)。

- (1) 两阶段事务中，加锁阶段为其拥有的锁逐步增长的阶段，而解锁阶段为其拥有的锁逐步缩减的阶段，故称两阶段事务。
- (2) 两阶段事务中，如开始解锁，就不能再对任何对象加锁。
- (3) 如事务遵循两阶段加锁协议，同时遵循先加锁后操作原则，则该调度是冲突可串行化的。

T1	T2
XLock (A)	
Read (A)	
A:=A+A0.1	
Write (A)	
XLock (B)	
UNLock (A)	
Read (B)	
	XLock (A)
	Read (A)
	A:=A+10
	Write (A)
B:=B+B0.2	
Write (B)	
UNLock (B)	
	XLock (B)
	UNLock (A)
	Read (B)
	B:=B-20
	Write (B)
	UNLock (B)

两段锁协议的级联回滚现象

T1	T2	T3
XLock (A) Read (A) $A := A + A0.1$ Write (A) XLock (B) UNLock (A) Read (B)	XLock (A) Read (A) $A := A + 10$ Write (A) UNLock (A)	SLock (A) Read (A)
$B := B + B0.2$ Write (B) UNLock (B)		

- ① 每个事务都遵从两段锁协议；
- ② 若T1事务在**WRITE(B)**时刻发生故障，将导致事务T2、T3级联回滚。
- ③ 主要原因是锁的**过早释放**
 - **严格两阶段锁**：除要求满足两段锁协议规定外，还要求事务的**排它锁**必须在事务**提交之后释放**。
解决**级联回滚**问题；避免了**脏读**和**丢失更新**的问题。
思考：可能出现不可重复读吗？**Why?**
 - **强两阶段锁**：除要求满足两段锁协议规定外，还要求事务的**所有锁**都必须在事务**提交之后释放**。
进一步解决了**不能重复读**的问题

(5) (S,U,X) 协议

(S, U, X) 协议：除S, X两种锁外，又增加了U锁，即更新锁。

增设U锁原因：事务在做更新时，分两步：先读后写。即先读老内容，在内存中修改后；然后再写入修改后的内容。除最后的写入阶段外，为保证与其他事务共享数据，而新增更新锁。

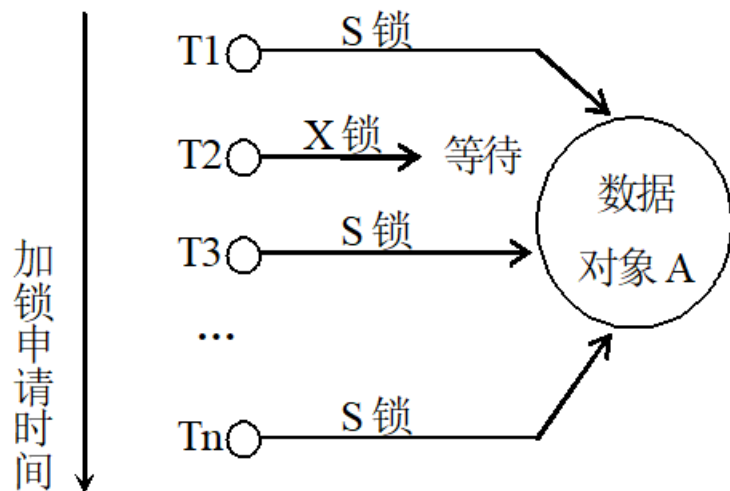
U锁的使用及作用：事务要更新数据对象时，先申请该对象的U锁。对象加了U锁，允许其他事务对它加S锁。在最后写入时，再将U锁升级为X锁。不必在全过程中加X锁，可提高并发度。

T2申请锁	T1 \ T2	S锁	X锁	U锁	无
	S锁	是	否	是	是
	X锁	否	否	否	是
	U锁	是	否	否	是

是：可以加锁 否：不能加锁

活锁与死锁

- 活锁**：当多个事务同时申请对数据的封锁时，由于选择策略的问题导致**长时间甚至永远的等待**，称为**活锁**。



注： T_i ($i=1,2,\dots,n$) 表示第 i 个事务

- 活锁的解决方法**
采用**先来先服务**（即队列方式）并结合优先级的选择策略。

- **死锁**：在多个事务竞争共享资源的情况下，出现的相互永远等待的封锁请求，若无外力作用，这些事务永远不能 向前推进，称为**死锁**。

例如：事务T1 和事务T2 并发执行，且都需要数据A 和B，假设数据A 和B 初始时为无锁状态。



事务隔离级别

- 数据库的隔离级别是指多个事务并发执行时，数据库系统为了保证事务之间的隔离性和一致性所采取的控制措施。
- 不同的隔离级别在事务并发执行时会采取不同的锁机制和控制策略，以确保事务操作的正确性和一致性。
- 隔离级别越高，数据库系统的并发性能通常会降低，因为需要更多的锁和控制机制来保证隔离性。
- 合理选择和配置隔离级别可以有效避免数据异常和并发问题，确保数据库系统的稳定性和可靠性。

常见的数据库隔离级别

- ① **读未提交(Read Uncommitted)**: 最低的隔离级别, 事务可以读取其他事务未提交的数据, 可能导致脏读、不可重复读和幻读问题。
- ② **读提交(Read Committed)**: 事务只能读取已提交的数据, 可以避免脏读问题, 但仍可能出现不可重复读和幻读问题。
- ③ **可重复读(Repeatable Read)**: 事务在执行过程中多次读取同一数据时, 保证数据的一致性不会发生变化, 可以避免不可重复读问题, 但仍可能出现幻读问题。
- ④ **序列化(Serializable)**: 最高的隔离级别, 通过对事务进行串行化执行, 确保事务之间不会相互影响, 避免脏读、不可重复读和幻读问题。

事务隔离级别

隔离级别	脏读	不可重复读	丢失更新	幻读
读未提交	可能	可能	可能	可能
读已提交	不可能	可能	可能	可能
可重复读	不可能	不可能	可能	可能
可串行化	不可能	不可能	不可能	不可能

下列有关数据库封锁协议描述正确的是：

A

如果事务在写某数据前对其加上X锁，该事务结束才解锁；但读数据时不用加S锁，则该事务不会产生丢失更新。

B

如果事务在写某数据前对其加上X锁，该事务结束后释放；读数据时加S锁，读完后即解锁，则该事务不会产生丢失更新和脏读。

C

如果事务在写某数据前对其加上X锁，该事务结束才解锁；读数据时加S锁，读完后立即解锁，则该事务不会产生脏读和不可重复读。

D

如果事务在写某数据前对其加上X锁，该事务结束才解锁；读数据时加S锁直到事务结束才解锁，则该事务不会产生不可重复读。

提交

有关数据库事务的死锁和活锁正确的是：

- ☐ A 数据库系统的事务并发执行时可能产生死锁，不会出现活锁。
- ☐ B 数据库并发执行的事务出现了死锁，也就是出现了死机。
- ☒ C 数据库系统使用先来先服务调度算法可以避免事务的活锁。
- ☐ D 数据库系统的事务并发执行时产生死锁和活锁是不可避免的。

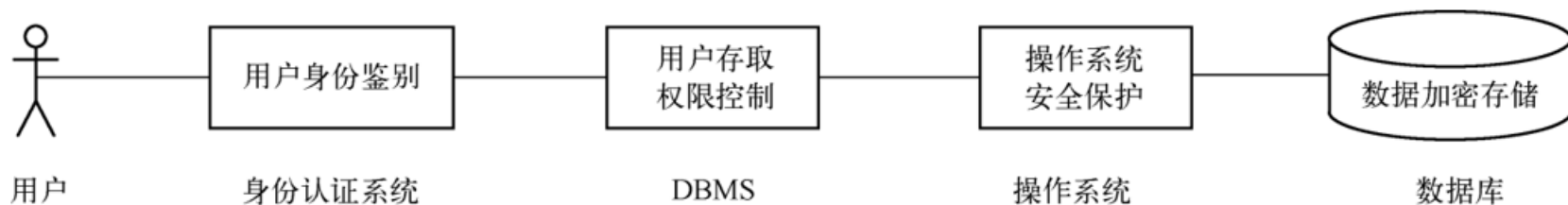
提交

5.6 数据库安全管理

一、数据库安全问题

- ① 黑客利用系统漏洞，攻击系统运行、窃取和篡改系统数据。
- ② 内部人员非法地泄露、篡改、删除系统的用户数据。
- ③ 系统运维人员操作失误导致数据被删除或数据库服务器系统宕机。
- ④ 系统故障导致数据库的数据损坏、数据丢失、数据库实例无法启动。
- ⑤ 意外灾害事件（火灾、水灾、地震等自然灾害）导致系统被破坏。

二、数据库系统安全模型



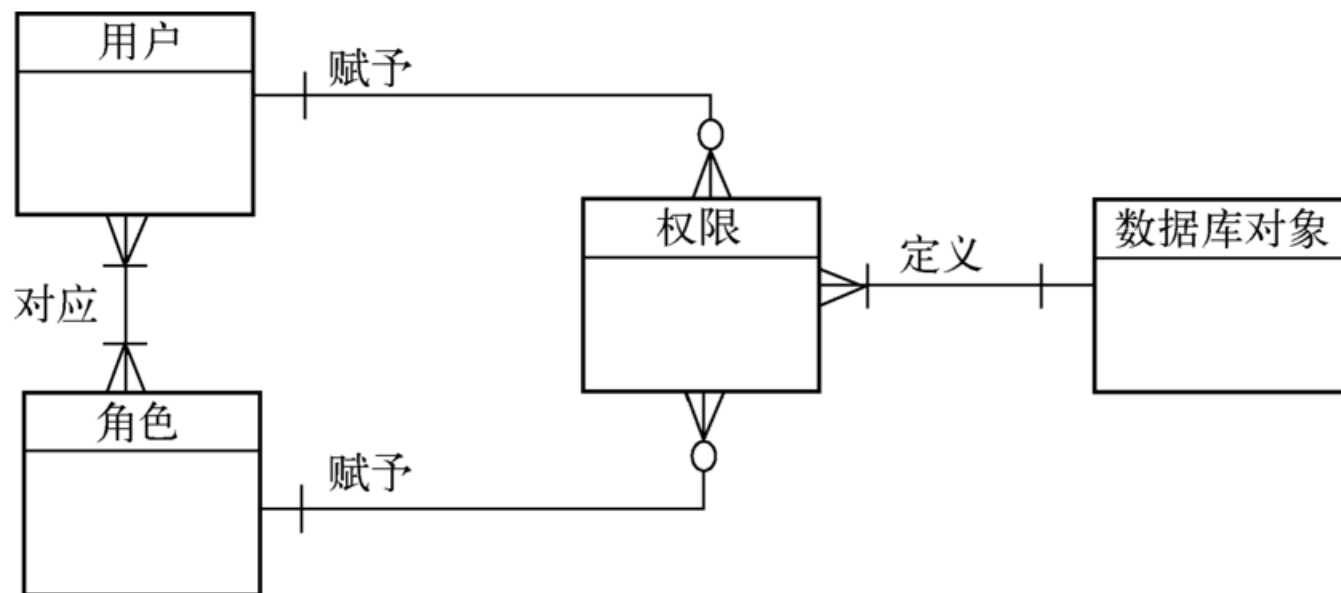
身份验证：用来确认登录用户是否是合法使用者

权限控制：通过权限机制控制用户对数据的访问

系统防护：OS系统安全机制防范非法系统访问

数据加密：通过加密算法对数据库中数据进行加密存储

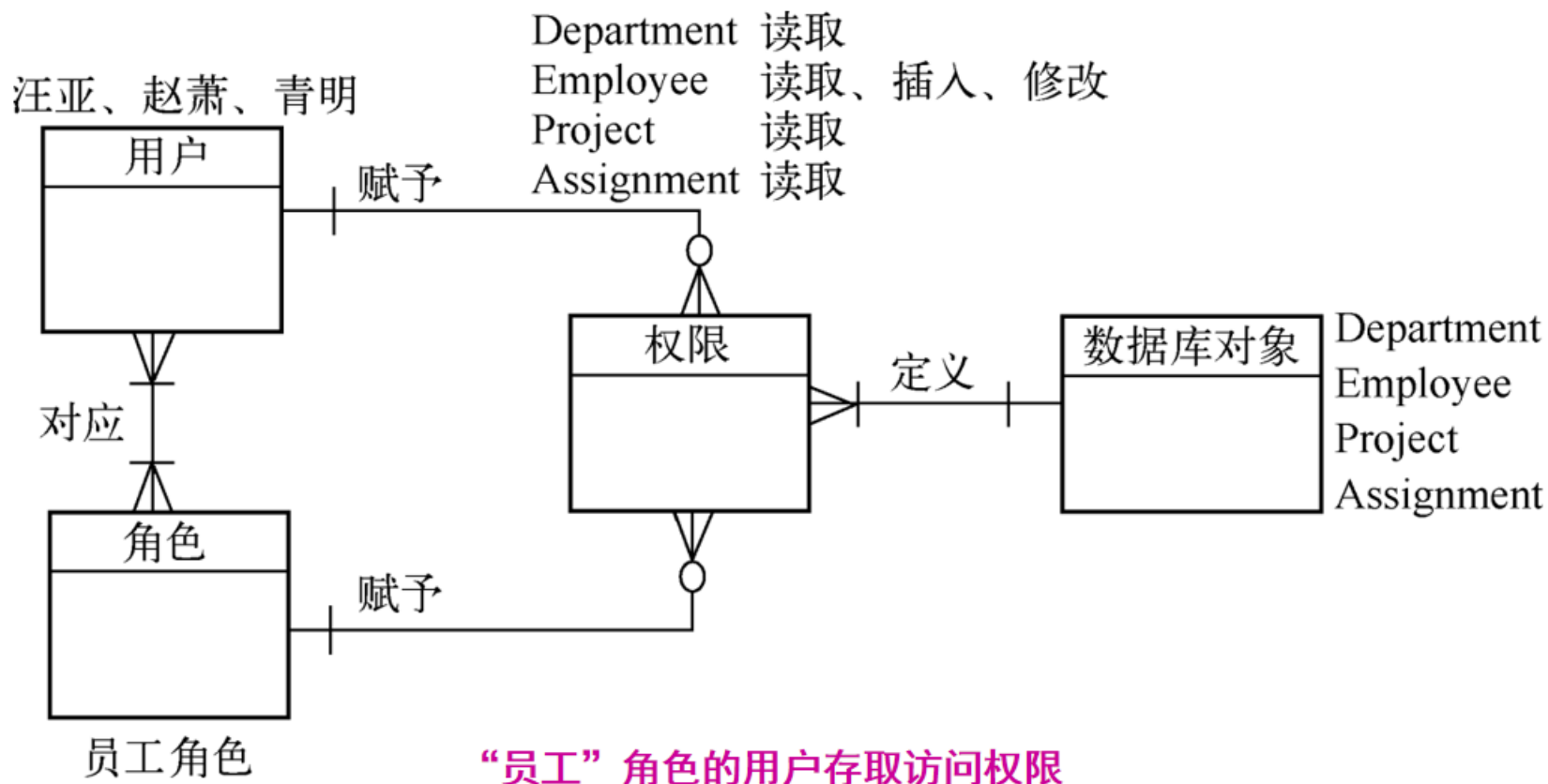
三、数据库存取权限控制安全模型



例：在3.7.1节的工程项目管理系统中，假定系统用户有三类角色：员工、经理和系统管理员。它们对数据库各个表对象的拥有权限见下表所示。

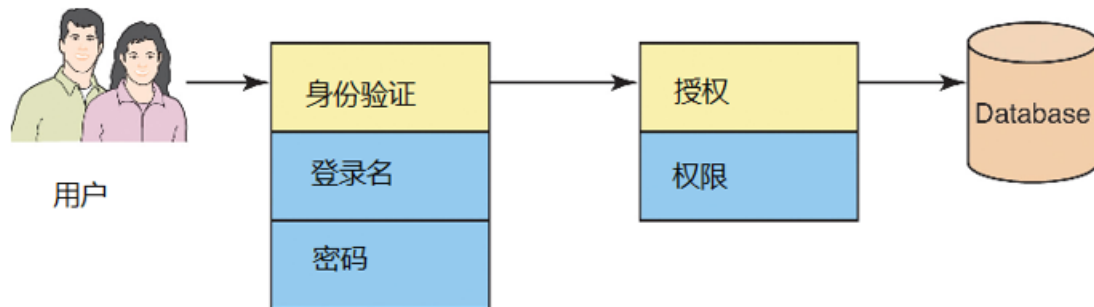
表	员工	经理	系统管理员
DEPARTMENT	读取	读取、插入、修改、删除	赋予权限、修改结构
EMPLOYEE	读取、插入、修改	读取、插入、修改、删除	赋予权限、修改结构
PROJECT	读取	读取、插入、修改、删除	赋予权限、修改结构
ASSIGNMENT	读取	读取、插入、修改、删除	赋予权限、修改结构

工程项目管理系统数据库存取权限控制安全模型设计



四、用户管理

用户要访问数据库，必须先在DBMS中创建其账号，并成为数据库的用户。此后，**用户每次访问数据库，都需要在DBMS进行身份验证**，只有合法用户才能进入系统，访问操作数据库对象。



用户管理——在数据库安全管理中，DBMS需要对每个用户进行管理，如用户创建、用户修改、用户删除管理等。

实现用户管理方式：

- ① 数据库服务器执行SQL语句管理用户
- ② 通过管理工具GUI操作管理用户

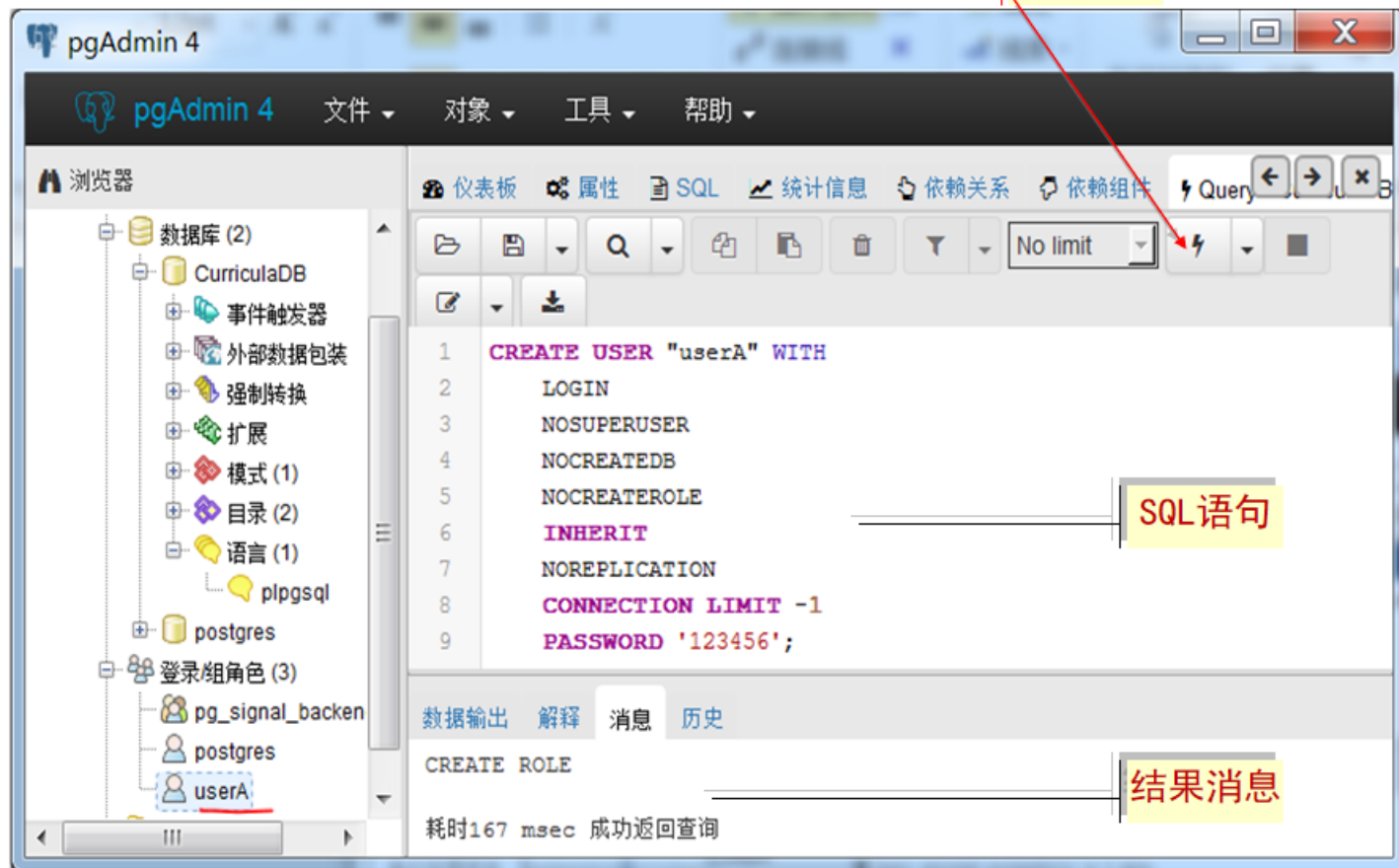
1. 用户创建SQL语句

CREATE USER <用户账号名> [[WITH] option [...]];

例：创建一个新用户，其账号名字为“userA”，密码为“123456”。该用户具有登录权限（Login）和角色继承权限系统权限，但它不是超级用户，不具有创建数据库权限、创建角色权限、数据库复制权限，此外数据库连接数不受限。

```
CREATE USER "userA" WITH  
LOGIN  
NOSUPERUSER  
NOCREATEDB  
NOCREATEROLE  
INHERIT  
NOREPLICATION  
CONNECTION LIMIT -1  
PASSWORD '123456';
```

用户创建SQL语句执行



2. 用户修改SQL语句

① 修改用户的属性

ALTER USER <用户名> [[**WITH**] option [...]];

② 修改用户的名称

ALTER USER <用户名> **RENAME TO** <新用户名>;

③ 修改用户的参数值

ALTER USER <用户名> **SET** <参数项> { **TO** | **=** } { value | **DEFAULT** };

④ 重置用户参数值

ALTER USER <用户名> **RESET** <参数项>;

例：修改用户“userA”的账号密码为“gres123”；
同时也限制该用户的数据库连接数为10。

```
ALTER USER "userA"  
CONNECTION LIMIT 10  
PASSWORD 'gres123';
```

用户修改SQL语句执行

The screenshot shows a database management tool interface. At the top, there is a toolbar with various icons and a search bar. Below the toolbar, the main area displays the SQL query being executed. The query is:

```
1 ALTER USER "userA"  
2 CONNECTION LIMIT 10  
3 PASSWORD 'gres123';  
4  
5
```

Below the query, there is a section for the execution results. The results show the command executed and the time taken:

```
ALTER ROLE  
耗时59 msec 成功返回查询
```

Annotations in the image point to specific elements:

- A red arrow points to the lightning bolt icon in the toolbar, labeled "运行按钮" (Run button).
- A yellow box labeled "SQL语句" (SQL statement) points to the query text.
- A yellow box labeled "结果消息" (Result message) points to the execution results.

3. 用户删除SQL语句

DROP USER <用户名>;

例: 在数据库中，删除用户“userA”，可以通过执行如下用户删除SQL语句实现用户删除。

DROP USER userA;

用户删除SQL语句执行

仪表板 属性 SQL 统计信息 依赖关系 依赖组件 Query - CurriculaDB on postgres@PostgreSQL 10 *

CurriculaDB on postgres@PostgreSQL 10

```
1 DROP USER "userA";
2
3
```

数据输出 解释 消息 Query History

DROP ROLE

耗时119 msec 成功返回查询

五、权限管理

数据库权限管理是指DBA管理员或数据库对象拥有者对其所拥有对象进行权限控制设置。

权限管理基本操作：

- 授予权限
- 收回权限
- 拒绝权限

权限类别：

- 数据库系统权限
- 数据库对象访问操作权限
- 数据库对象定义操作权限

1. 权限管理SQL语句

GRANT <权限名> **ON** <对象名> **TO** {数据库用户名|用户角色名};

REVOKE <权限名> **ON** <对象名> **FROM** {数据库用户名|用户角色名};

DENY <权限名> **ON** <对象名> **TO** {数据库用户名|用户角色名};

2. 权限管理实例

例 在3.7.1节的工程项目管理系统中，DBA管理员赋予员工用户（userA）对部门表（Department）、员工表（Employee）、项目表（Project）和任务表（Assignment）的读取数据权限。

对用户 “userA” 实现授权SQL程序如下

GRANT SELECT ON Department TO userA;

GRANT SELECT ON Employee TO userA;

GRANT SELECT ON Project TO userA;

GRANT SELECT ON Assignment TO userA;

用户授权SQL语句执行

运行按钮

The screenshot shows a SQL execution tool interface. At the top is a toolbar with icons for file operations, search, and execution. A lightning bolt icon is highlighted with a yellow box and labeled '运行按钮' (Run button). Below the toolbar is a text area containing four SQL statements, each on a new line and numbered 1 to 4. A yellow box labeled 'SQL语句' (SQL statement) points to the first statement. Below the text area is a tabbed interface with four tabs: '数据输出' (Data Output), '解释' (Explain), '消息' (Messages), and '历史' (History). The '消息' (Messages) tab is selected. Below the tabs, the text 'GRANT' is displayed. At the bottom, the execution result is shown: '耗时98 msec 成功返回查询' (Took 98 msec, successfully returned query). A yellow box labeled '结果消息' (Result message) points to this text.

```
1 GRANT SELECT ON Department TO "userA";
2 GRANT SELECT ON Employee TO "userA";
3 GRANT SELECT ON Project TO "userA";
4 GRANT SELECT ON Assignment TO "userA";
5
```

数据输出 解释 消息 历史

GRANT

耗时98 msec 成功返回查询

六、角色管理

在DBMS中，为了方便对众多用户及其权限进行管理，通常将一组具有相同权限的用户定义为**角色(Role)**。

角色管理内容：

- 创建角色
- 修改角色
- 删除角色

角色管理实现方式：

- 执行SQL语句管理角色
- 通过GUI操作管理角色

1. 角色管理SQL语句

① 创建角色的属性

Create ROLE <角色名> [[**WITH**] option [...]];

② 修改角色的属性

ALTER ROLE <角色名> [[**WITH**] option [...]];

③ 修改角色的名称

ALTER ROLE <角色名> **RENAME TO** <新角色名>;

④ 修改角色的参数值

ALTER ROLE <角色名> **SET** <参数项> { **TO** | **=** } { value | **DEFAULT** };

④ 删除指定角色

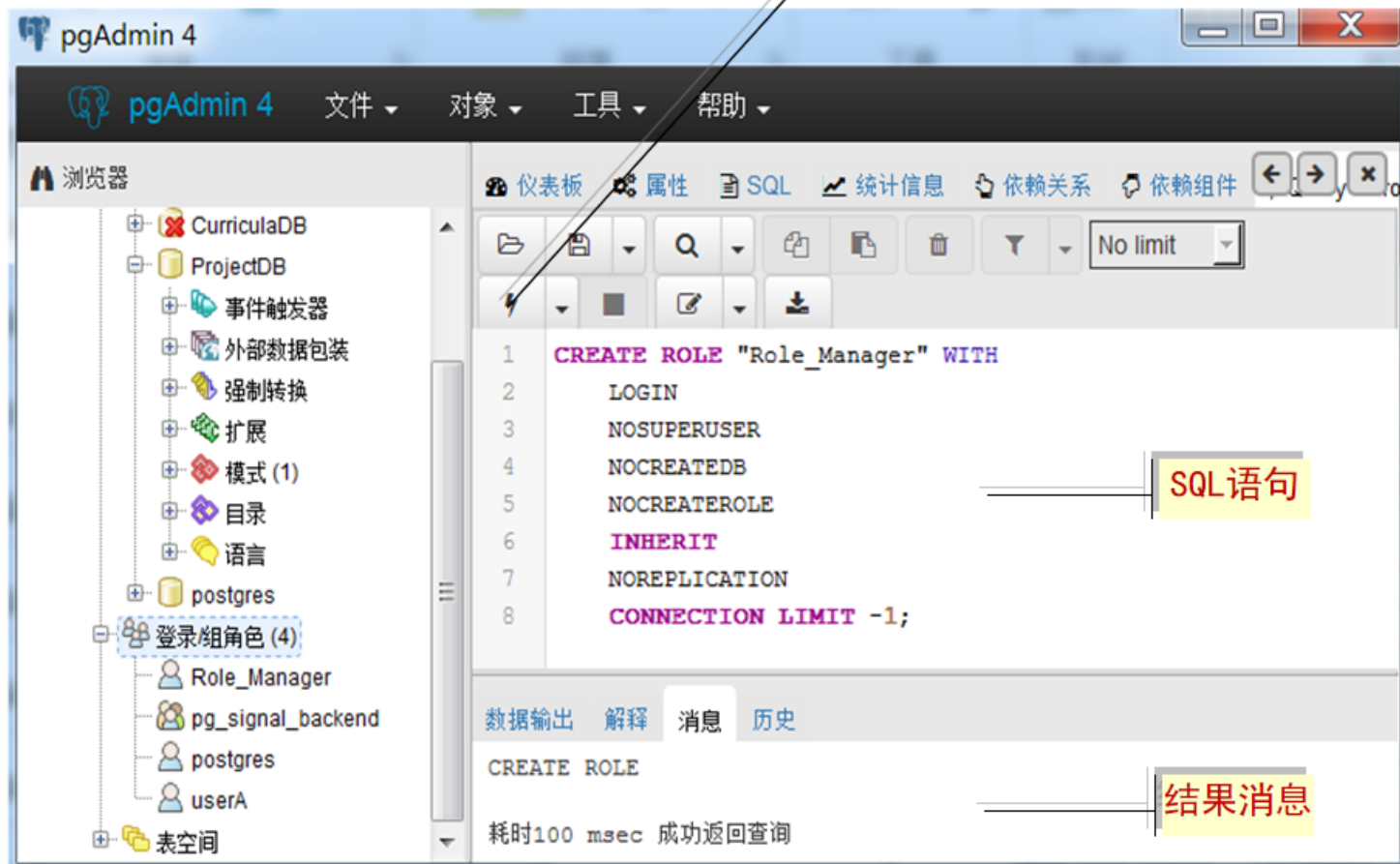
DROP ROLE <角色名>;

2. 角色管理实例

例：在工程项目管理系统中，假定需要在ProjectDB数据库内创建经理角色Role_Manager。该角色具有登录权限（Login）和角色继承权限系统权限，但它不是超级用户，不具有创建数据库权限、创建角色权限、数据库复制权限，此外数据库连接数不受限。

```
CREATE ROLE "Role_Manager" WITH  
LOGIN  
NOSUPERUSER  
NOCREATEDB  
NOCREATEROLE  
INHERIT  
NOREPLICATION  
CONNECTION LIMIT -1;
```

角色创建SQL语句执行



3. 角色权限授予

例：在创建好经理角色Role_Manager后，还需要赋予该角色对数据库表Department、Employee、Project、Assignment的插入、修改、删除、查询权限。

```
GRANT SELECT,INSERT,UPDATE,DELETE ON Department TO "Role_Manager";
```

```
GRANT SELECT,INSERT,UPDATE,DELETE ON Employee TO "Role_Manager";
```

```
GRANT SELECT,INSERT,UPDATE,DELETE ON Project TO "Role_Manager";
```

```
GRANT SELECT,INSERT,UPDATE,DELETE ON Assignment TO "Role_Manager";
```

角色授权SQL语句执行

The screenshot shows the pgAdmin 4 interface. On the left is the '浏览器' (Browser) pane showing a tree structure of the database. The '登录/组角色 (4)' (Logins/Groups Roles (4)) folder is expanded, and 'Role_Manager' is selected. The main pane shows the 'Query - ProjectDB on postgres@PostgreSQL 9.6' window. The SQL editor contains four GRANT statements. A yellow callout box labeled '运行按钮' (Run Button) points to the lightning bolt icon in the toolbar. Below the editor, the '消息' (Messages) tab is active, showing the output 'GRANT' and '耗时117 msec 成功返回查询' (Took 117 msec, successfully returned query). A yellow callout box labeled 'SQL语句' (SQL Statement) points to the SQL editor, and another labeled '结果消息' (Result Message) points to the Messages tab.

pgAdmin 4

pgAdmin 4 文件 对象 工具 帮助

浏览器

- 外部数据包装
- 强制转换
- 扩展
- 模式 (1)
- 目录
- 语言
- postgres
- 登录/组角色 (4)
 - Role_Manager
 - pg_signal_backend
 - postgres
 - userA
- 表空间

仪表板 属性 SQL 统计信息 依赖关系 依赖组件 Query - ProjectDB on postgres@PostgreSQL 9.6

ProjectDB on postgres@PostgreSQL 9.6

```
1 GRANT SELECT,INSERT,UPDATE,DELETE ON Department TO "Role_Manager";
2 GRANT SELECT,INSERT,UPDATE,DELETE ON Employee TO "Role_Manager";
3 GRANT SELECT,INSERT,UPDATE,DELETE ON Project TO "Role_Manager";
4 GRANT SELECT,INSERT,UPDATE,DELETE ON Assignment TO "Role_Manager";
5
6
```

数据输出 解释 消息 历史

GRANT

耗时117 msec 成功返回查询

运行按钮

SQL语句

结果消息

5.7. 备份与恢复

一、数据库备份

数据库备份是指将数据库中的数据和结构复制到另一个位置或存储介质，以防止数据丢失或损坏的过程。合理制定备份策略、选择合适的备份类型和工具，定期进行备份操作，并测试备份的可恢复性，可以有效应对数据丢失或损坏的风险，确保数据库系统的稳定性和可靠性。

数据库备份常用方法：

- ① **全量备份（Full Backup）**：全量备份是指将数据库中的所有数据和结构完整地备份到另一个位置或介质中。全量备份通常是定期进行的，可以保证数据库在某一时间点的完整性和一致性。
- ② **增量备份（Incremental Backup）**：增量备份是指在全量备份的基础上，只备份自上次备份以来发生变化的数据和日志。增量备份可以减少备份时间和存储空间，但恢复时需要依次恢复全量备份和增量备份。

数据库备份常用方法：

- ③ 差异备份（Differential Backup）：差异备份是指在全量备份的基础上，备份自上次全量备份以来发生变化的数据。与增量备份不同的是，差异备份只需要恢复最近的一次全量备份和差异备份即可。
- ④ 事务日志备份：是一种只备份自上一次日志备份至今的事务日志数据的方式。事务日志备份所需要的时间和空间比差异数据库备份更少，而且它可以支持事务的回滚操作；使用事务日志备份可以将数据库恢复到故障点时刻的状态。

数据库备份常用方法：

- ⑤ 热备份（Online Backup）：热备份是指在数据库运行时进行备份操作，不会中断数据库的正常运行。热备份通常需要数据库支持相应的备份工具和机制。
- ⑥ 冷备份（Cold Backup）：冷备份是指在数据库停止运行时进行备份操作，可以确保备份的数据完整性和一致性，但会中断数据库的正常服务。
- ⑦ 文件备份：是指将数据库系统中的系统文件、数据文件和日志文件复制到另一个位置或存储介质，以保证数据库数据的安全性和可靠性。

数据库备份设备

数据库备份设备是指用于存储数据库备份数据的硬件设备或存储介质，确保备份数据的安全性和可靠性。常见的数据库备份设备包括以下几种：

- ① **磁带备份设备**：磁带备份设备是传统的备份存储介质之一，具有较大的存储容量和较低的成本，适合长期存储大量备份数据。磁带备份设备通常需要专门的磁带库管理软件来管理备份数据。
- ② **磁盘阵列备份设备**：是一种快速、可靠的备份存储介质，具有较高的读写速度和较低的故障率，适合快速备份和恢复数据库数据。磁盘阵列备份设备可以是外部硬盘、网络存储设备或云存储服务。

数据库备份设备

- ③ **网络存储设备**：网络存储设备是一种集中存储数据的设备，可以**通过网络**连接到数据库服务器进行备份和恢复操作。网络存储设备具有高可用性和可扩展性，适合大规模数据库系统的备份需求。
- ④ **云存储服务**：云存储服务是一种**基于云计算技术**提供的备份存储服务，**用户**可以将数据库备份数据上传到云端存储，实现数据的远程备份和恢复。云存储服务具有高可靠性、灵活性和可扩展性，适合跨地域或跨机房的备份需求。
- ⑤ **虚拟磁带库**：虚拟磁带库是一种**基于硬盘存储**的虚拟备份设备，可以模拟磁带备份设备的功能，提供类似于磁带备份的备份和恢复操作。虚拟磁带库可以**提高备份和恢复的效率，减少备份时间和成本**。

PostgreSQL备份数据库

(1) pg_dump备份数据库

pg_dump是 PostgreSQL客户端应用程序，须具有对要备份的**所有表的读访问权限**；pg_dump只备份某个数据库的数据，它**不会导出角色和表空间相关的信息**，因为这些信息是整个数据库集群共用的，不属于某个单独的数据库。

① 首先使用下列命令进入postgresql12的bin目录

```
cd \program files\PostgreSQL\12\bin
```

② 执行下列命令完成数据库备份

```
pg_dump -h localhost -U postgres -p 5432 -d coursedb -c -C -f f:\databackup\coursedb.backup
```

```
C:\Program Files\PostgreSQL\12\bin>pg_dump -h localhost -U postgres -p 5432 -d coursedb -c -C -f f:\databackup\coursedb.backup  
□令:
```

pg_dump命令参数

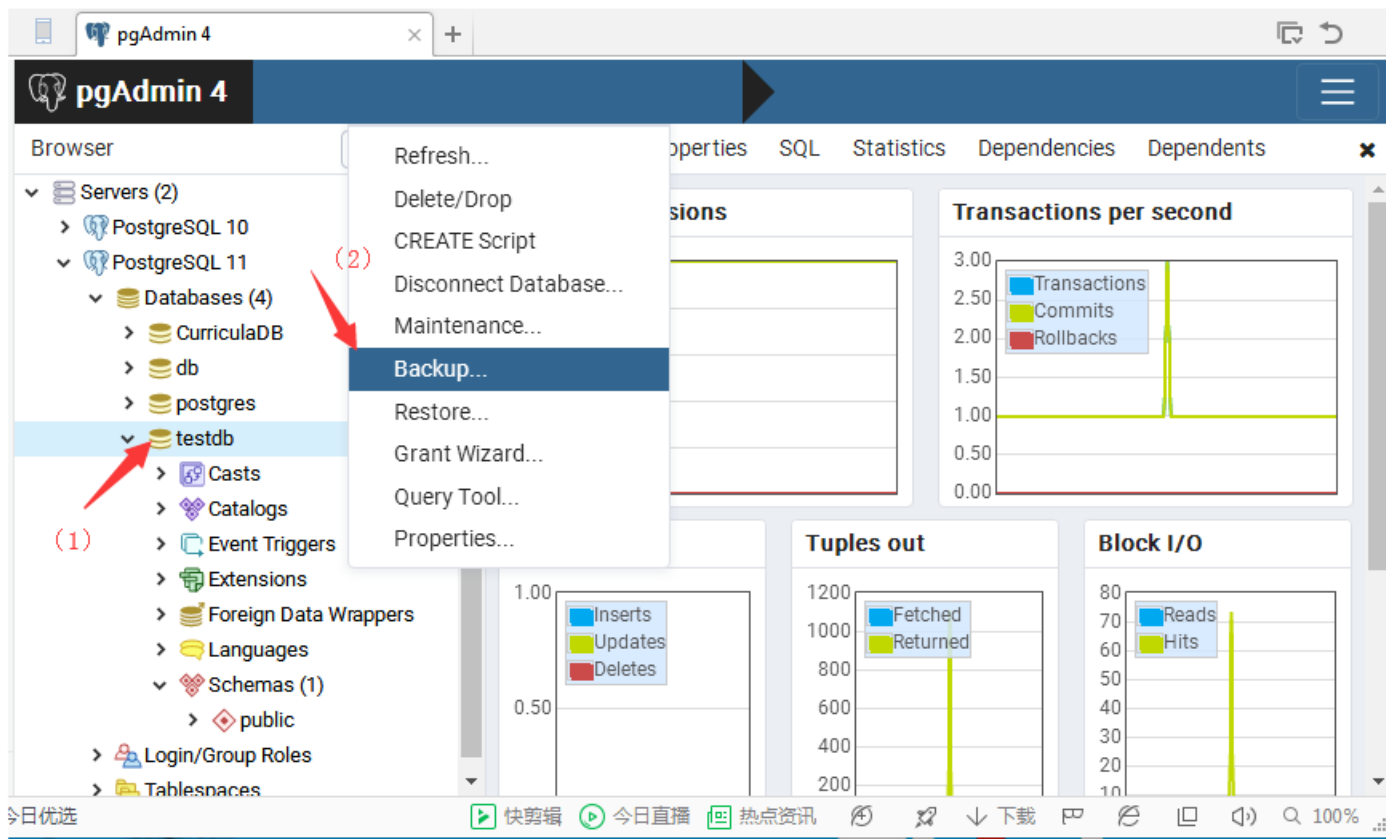
- h: 数据库服务器地址;
- p: 数据库端口号;
- U: U 大写,表示用户名;
- d: 数据库名称;
- f: 备份文件路径以及备份文件名称;
- c: 在重新创建之前,先删除数据库对象;
- C: 在转储中包括命令,以便创建数据库;
- t: 只转储指定名称的表;
- w: 永远不提示输入口令
- W: 强制口令提示 (自动)

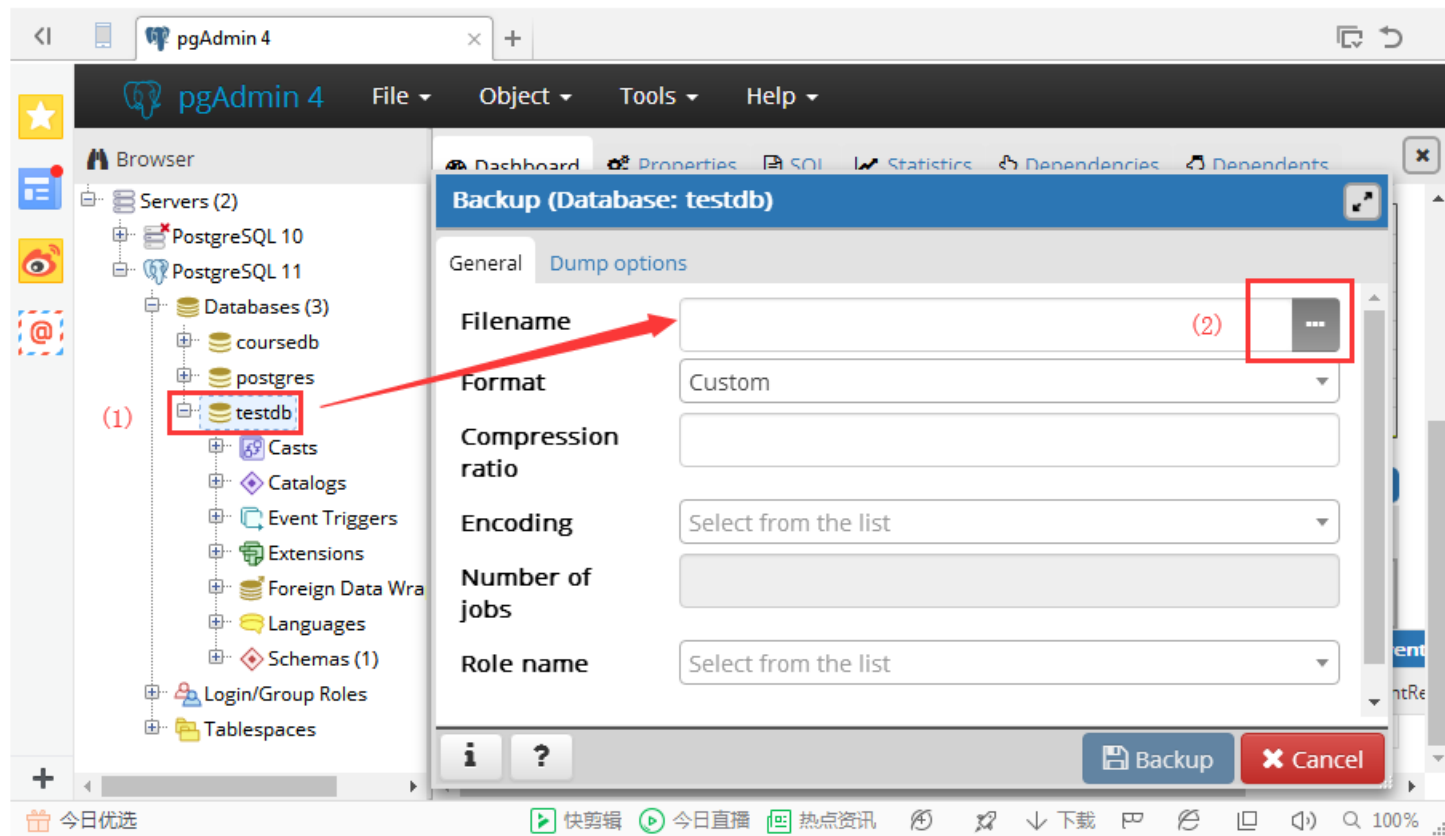
(2) 使用pg_dumpall备份数据库

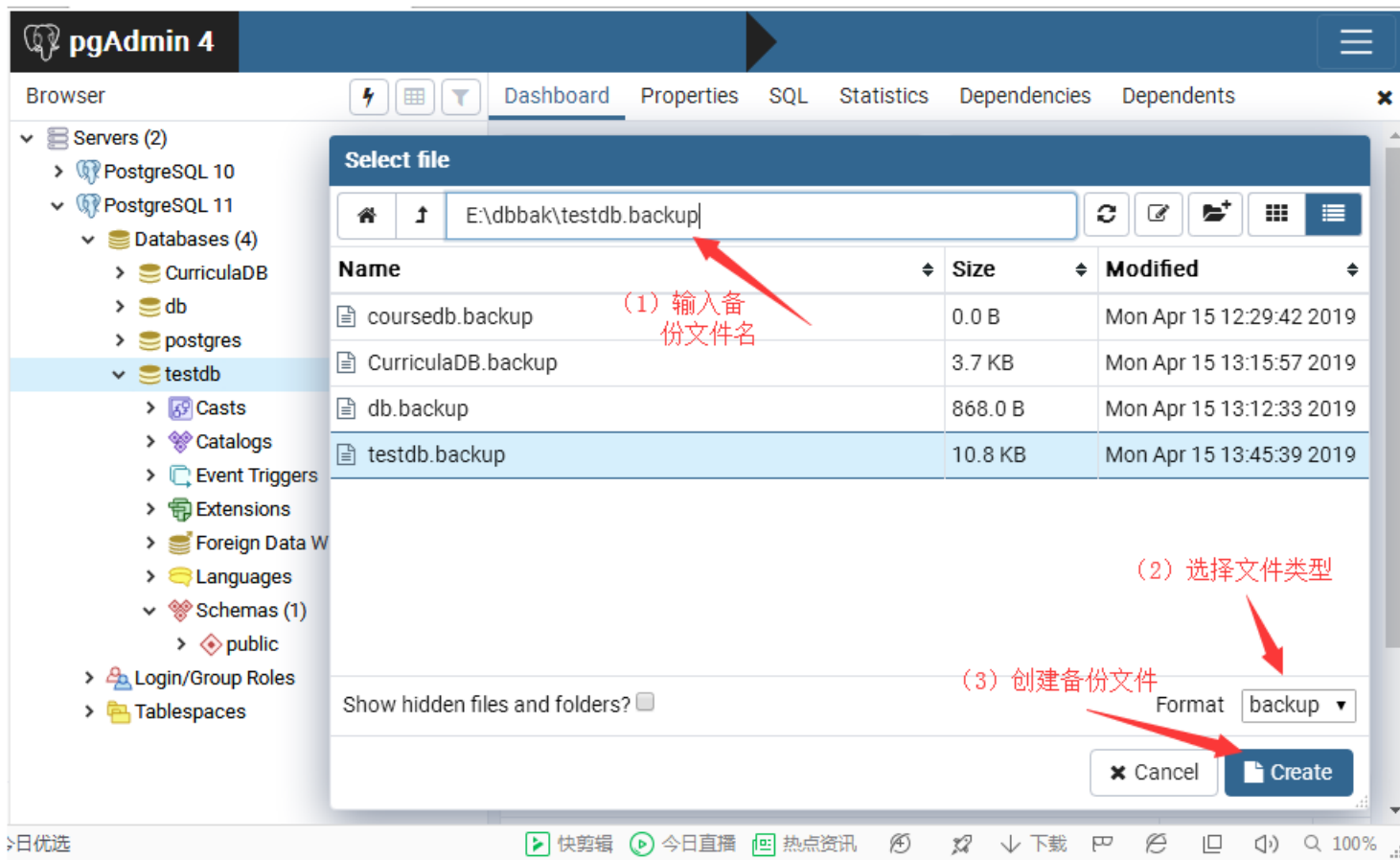
pg_dumpall命令可以备份数据库集群的全部内容，包含了表空间、角色的定义，这个命令是重新创建角色、表空间、空数据，然后再为每个数据库调用pg_dump。

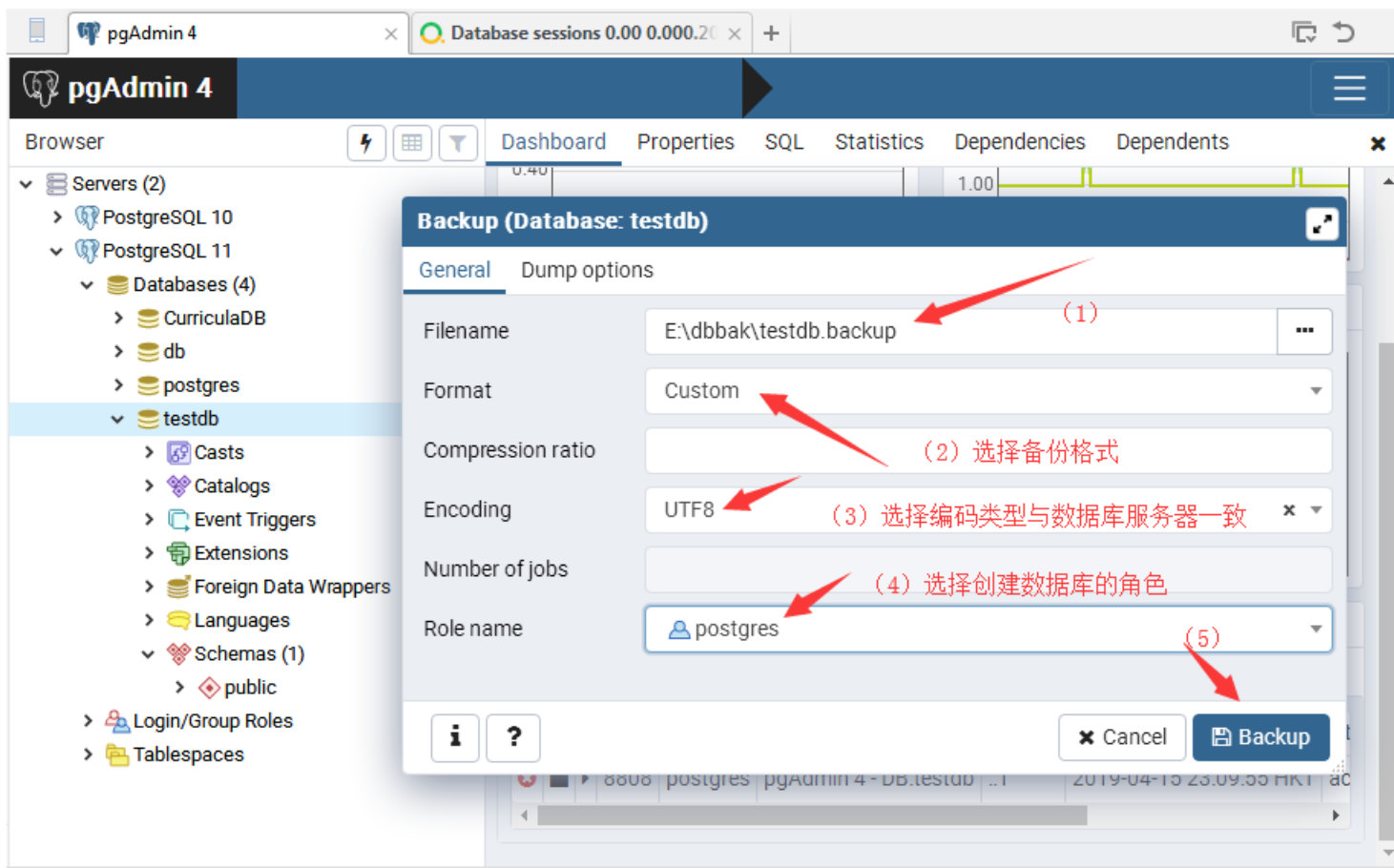
```
C:\Program Files\PostgreSQL\12\bin>pg_dumpall -h localhost -U postgres -p 5432 -f f:\databackup\alldb.backup
[Progress indicators: square box, diamond, colon]
C:\Program Files\PostgreSQL\12\bin>_
```

(3) 使用pgAdmin4实现postgreSQL数据库备份









pgAdmin 4

Browser

- Servers (2)
 - PostgreSQL 10
 - PostgreSQL 11
 - Databases (4)
 - CurriculaDB
 - db
 - postgres
 - testdb
 - Casts
 - Catalogs
 - Event Triggers
 - Extensions
 - Foreign Data Wrappers
 - Languages
 - Schemas (1)
 - public
 - Login/Group Roles
 - Tablespaces

Dashboard Properties SQL Statistics Dependencies Dependents

Tuples in

Tuples out

Block I/O

Backing up an object on the server

Server activity

Sessions

Backing up an object on the server 'PostgreSQL 11 (localhost:5433)' from database 'testdb'

Mon Apr 15 2019 23:31:45 GMT+0800 (China Standard Time)

1.35 seconds

OK

More details...

Stop Process

Successfully completed.

二、数据库恢复

数据库恢复是指在数据库系统发生故障、数据丢失或损坏时，通过备份数据或日志记录将数据库恢复到正常运行状态的过程。

1、数据库恢复场景

(1) 事务故障的数据恢复

是指在数据库系统中**发生事务执行过程中出现故障或错误，导致数据不一致或损坏时**，通过事务恢复机制将数据库恢复到一致性状态的过程。事务故障数据恢复通常包括以下几个方面：

- ① **事务回滚**：当事务执行过程中发生错误或异常时，数据库系统会进行事务回滚操作，将未提交的事务操作全部回滚，恢复数据库到事务开始之前的状态，确保数据的一致性和完整性。
- ② **事务重做**：在事务提交后，如果数据库系统发生故障或崩溃，可能会导致已提交的事务操作未能持久化到磁盘上。数据库系统会通过事务重做操作，重新执行已提交的事务操作，将数据恢复到提交事务后的状态，确保数据的持久性和一致性。

③ **事务日志恢复**：数据库系统会通过事务日志记录事务操作的执行过程和结果，当数据库系统发生故障或崩溃时，可以通过事务日志中的信息进行事务恢复操作，包括回滚未提交的事务、重做已提交的事务等，确保数据库的一致性和完整性。

④ **检查点恢复**：数据库系统会定期生成检查点，记录数据库的状态和日志信息，当数据库系统发生故障或崩溃时，可以通过检查点信息进行数据库恢复操作，加快恢复速度并减少数据丢失的风险。

(2) 系统崩溃的数据恢复

是指在数据库系统或计算机系统**发生意外崩溃、断电或异常关闭**等情况下，通过系统恢复机制将数据恢复到一致性状态的过程。系统崩溃**可能导致数据丢失、事务未提交、日志未写入磁盘等问题**，需要通过系统崩溃的数据恢复操作来确保数据的完整性和可靠性。系统崩溃的数据恢复通常包括以下几个步骤：

- ① **系统恢复启动**：当系统发生崩溃时，数据库系统或操作系统会自动**启动系统恢复机制，检查系统状态和日志信息**，确定恢复的起始点和方式。
- ② **日志重做**：系统会首先进行**日志重做操作**，将**已提交的事务操作和未持久化到磁盘的日志信息重新执行**，确保数据的持久性和一致性。

- ③ **事务回滚**：对于未提交的事务操作，系统会进行事务回滚操作，将未提交的事务操作全部回滚，恢复数据库到事务开始之前的状态，确保数据的一致性和完整性。
- ④ **检查点恢复**：系统会根据最近的检查点信息，恢复数据库到检查点时的状态，减少恢复的时间和数据丢失的风险。
- ⑤ **数据一致性检查**：在系统崩溃的数据恢复过程中，需要对恢复的数据进行一致性检查，确保数据的完整性和正确性。可以通过查询数据、执行事务操作、检查日志等方式验证数据恢复的正确性。

(3) 存储介质损坏的数据恢复

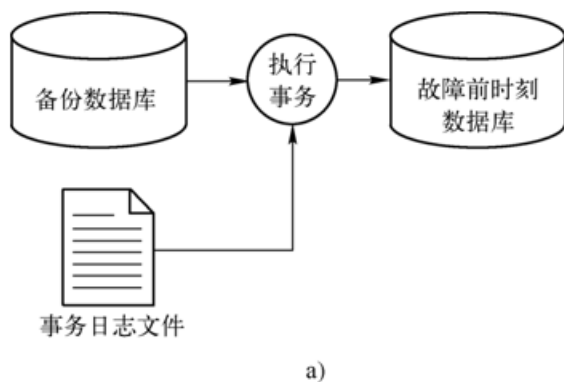
是指在硬盘、固态硬盘、光盘等**存储介质发生物理损坏或逻辑损坏导致数据丢失或无法访问**时，通过专业的数据恢复工具和技术将损坏的存储介质中的数据进行恢复的过程。存储介质损坏可能包括磁盘坏道、芯片损坏、文件系统损坏、数据丢失等情况，需要采取相应的数据恢复方法来尝试恢复丢失的数据。存储介质损坏的数据恢复通常包括以下几个步骤：

- ① **存储介质评估**：首先需要对损坏的存储介质进行评估，确定损坏的原因和程度，包括物理损坏和逻辑损坏的情况，为后续数据恢复操作做准备。
- ② **数据恢复工具**：根据存储介质的损坏情况选择合适的数据恢复工具和技术，如数据恢复软件、数据恢复设备等，用于尝试恢复存储介质中的数据。

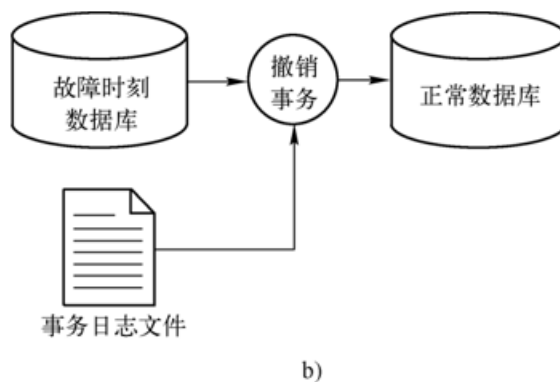
- ③ **数据镜像**：在进行数据恢复操作之前，通常会先对损坏的存储介质进行数据镜像，将存储介质中的数据复制到其他存储介质上，以避免进一步损坏原始数据。
- ④ **数据恢复操作**：根据存储介质的损坏情况和数据恢复工具的支持，进行数据恢复操作，尝试恢复存储介质中的丢失数据，包括修复文件系统、恢复损坏文件、重建数据等操作。
- ⑤ **数据验证**：在数据恢复操作完成后，需要对恢复的数据进行验证，确保数据的完整性和正确性，避免数据丢失或损坏的风险。

2、数据库恢复技术

数据库恢复技术是**利用数据库备份文件和数据库事务日志文件**来实现数据库恢复处理的；根据用户恢复要求，一般采用**前滚（redo重做）事务方式**或**回滚（rollback）事务方式**恢复数据库。



a) 前滚(redo)事务恢复方式



b) 回滚事务恢复方式

3、PostgreSQL数据库的恢复方法

(1) 使用psql命令恢复数据库

psql是postgresql数据库提供的连接数据库shell命令：

psql -h IP地址 -p 端口 -U 数据库名 -f 需执行的文件

例如：**psql -h 127.0.0.1 -U postgres -p 5432 -f f:\databackup\coursedb.backup**

例如: `psql -h 127.0.0.1 -U postgres -p 5432 -f f:\databackup\coursedb.backup`

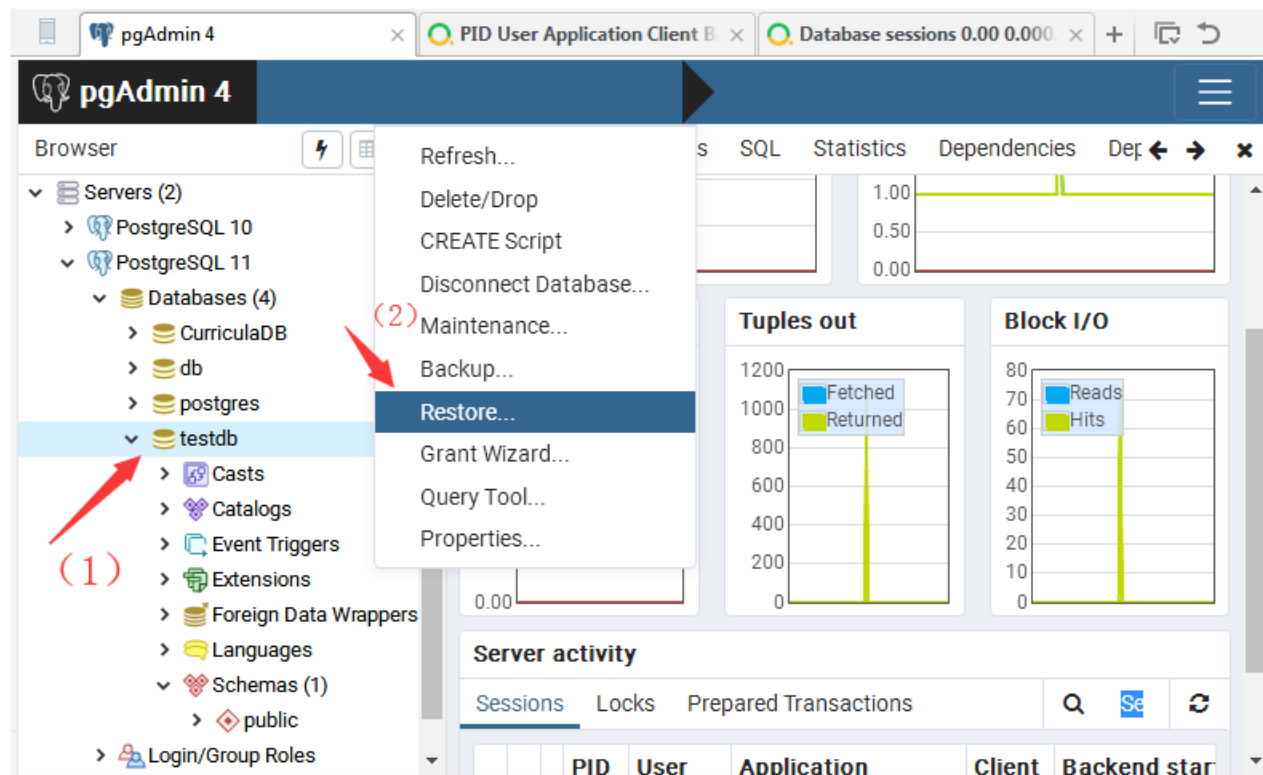
```
命令提示符
C:\Program Files\PostgreSQL\12\bin>psql -h 127.0.0.1 -U postgres -p 5432 -f f:\databackup\coursedb.backup
用户 postgres 的口令:
SET
SET
SET
SET
SET
SET
set_config
-----
(1 行记录)

SET
SET
SET
SET
DROP DATABASE
CREATE DATABASE
ALTER DATABASE
您现在已经连接到数据库 "coursedb", 用户 "postgres".
SET
SET
SET
SET
set_config
-----
(1 行记录)

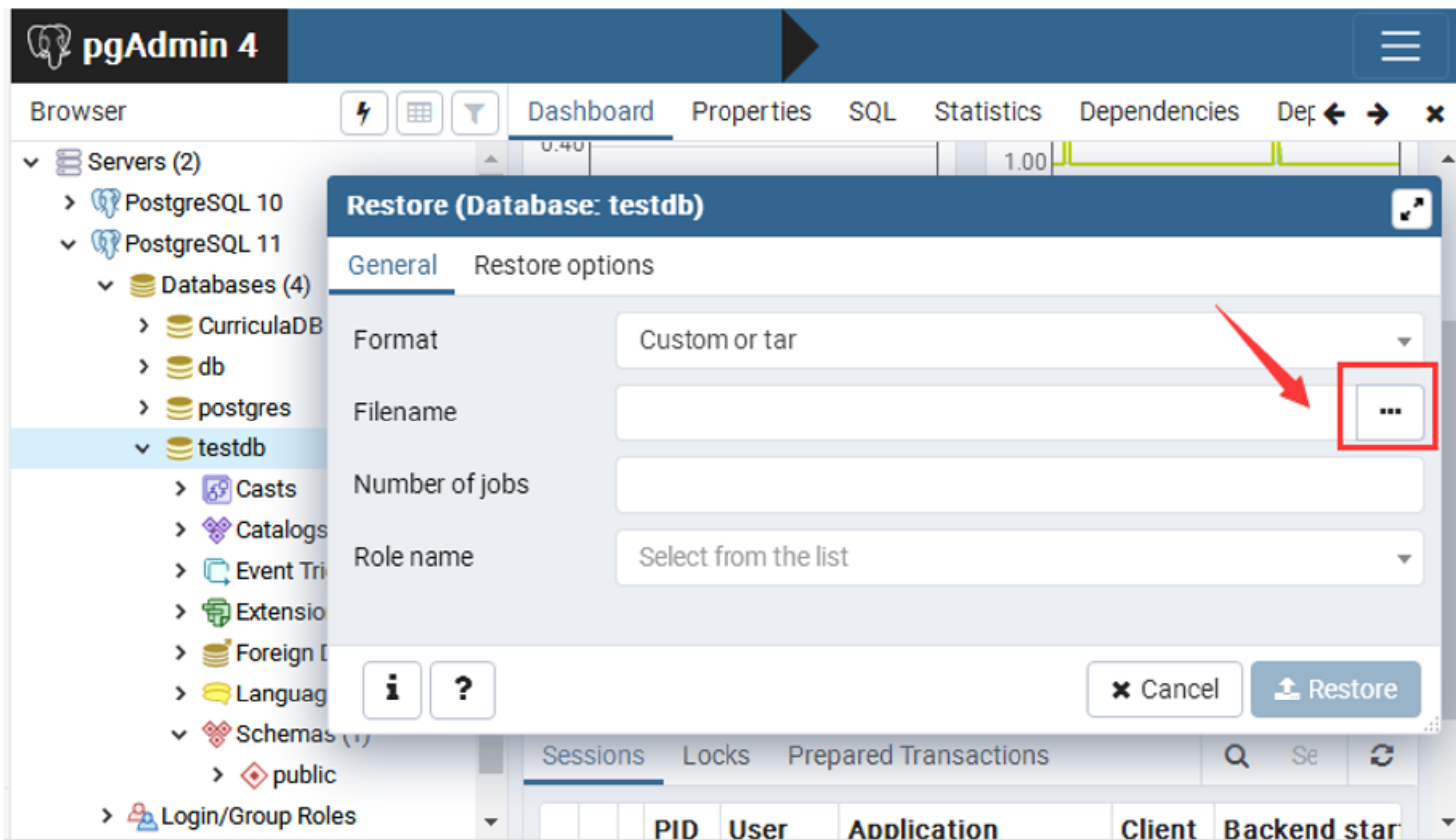
SET
SET
SET
SET
CREATE FUNCTION
ALTER FUNCTION
SET
SET
CREATE TABLE
ALTER TABLE
COPY 0
CREATE TRIGGER

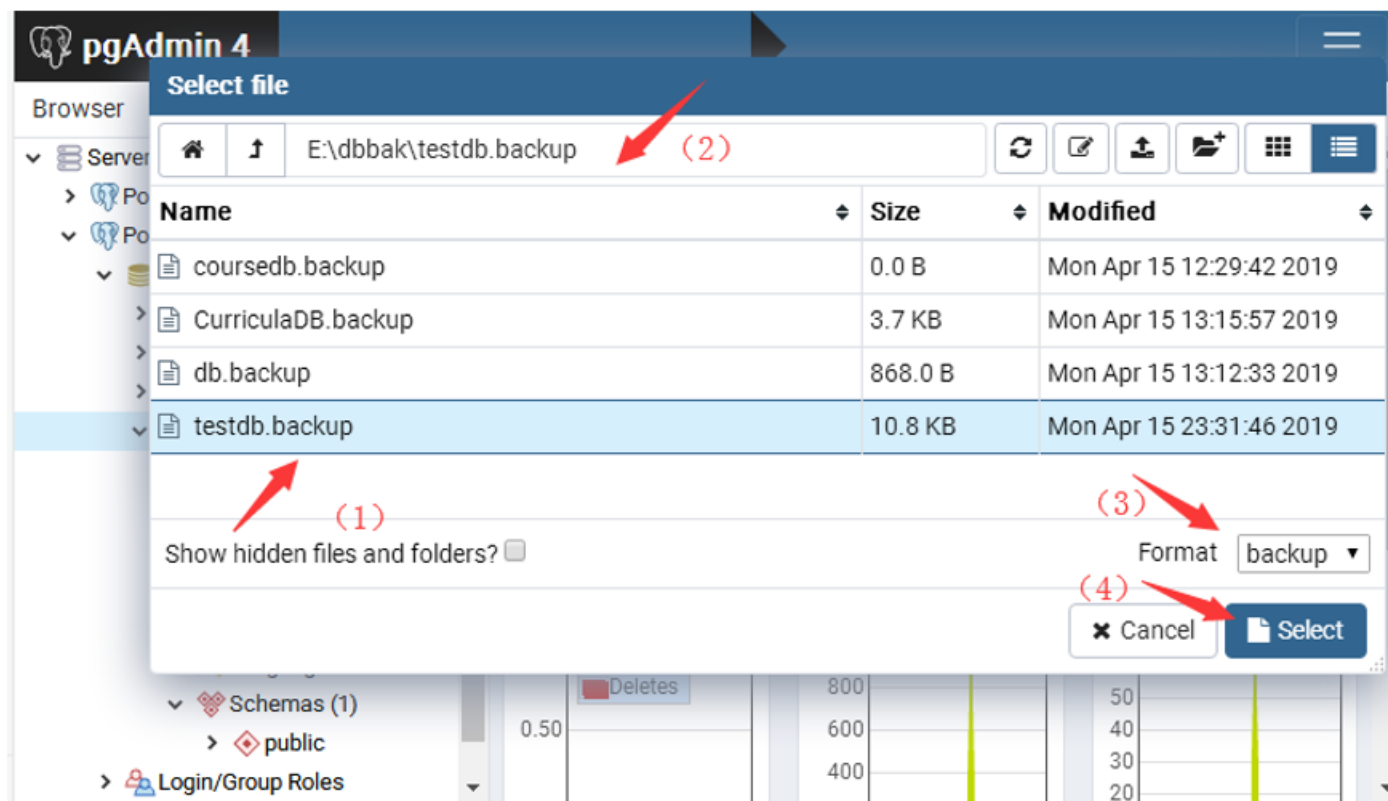
C:\Program Files\PostgreSQL\12\bin>
```

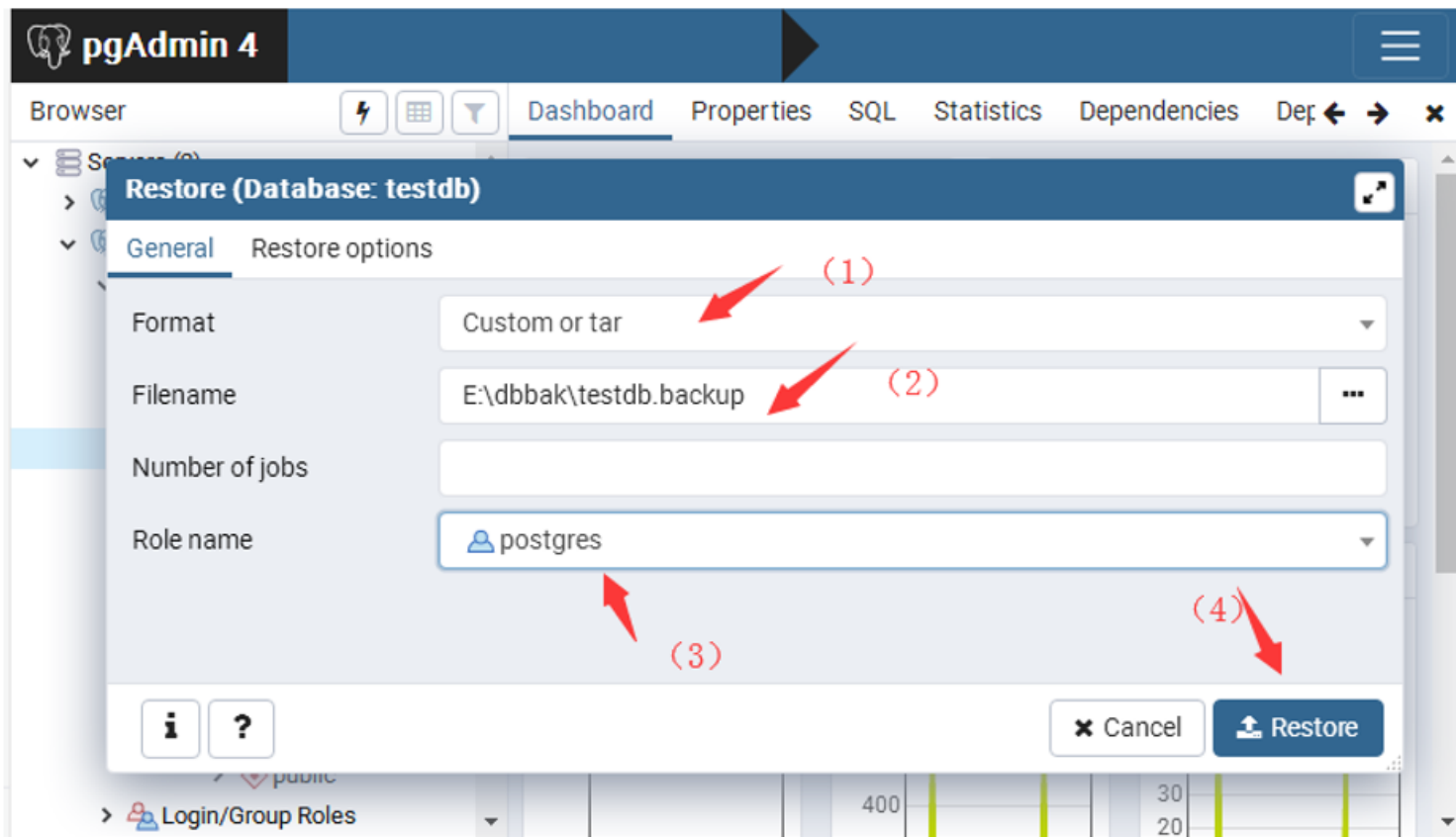
(2) 使用pgAdmin4实现postgreSQL数据库恢复



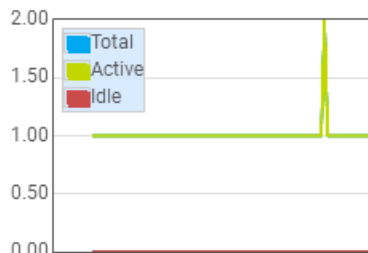
请务必注意：需要恢复的数据库一定是新建的空数据库，否则有很大可能会报错！



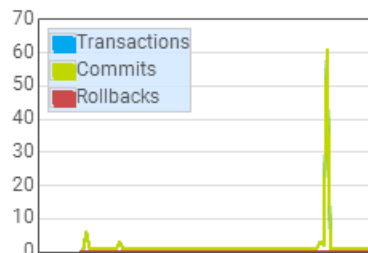




Database sessions



Transactions per second



Tuples in



Tuples out



Block I/O



Restoring backup on the server

Restoring backup on the server 'PostgreSQL 11 (localhost:5433)'

Mon Apr 15 2019 23:52:46 GMT+0800 (China Standard Time)

0.68 seconds

OK

More details...

Stop Process



Successfully completed.

谢谢大家！